



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

1. 已知一组字符及其出现频率如下表所示：

字符	A	B	C	D	E	F
频率	5	9	12	13	16	45

(1) 构造对应的哈夫曼树

(2) 给出每个字符的哈夫曼编码

(3) 计算该哈夫曼树的带权路径长度 WPL 224

2. 某森林F对应的二叉树为T，T的先序遍历序列是a, b, d, c, e, g, f。中序序列是b, d, a, e, g, c, f。则，森林T中有几棵树？ 3

3. 若一棵度为7的树有7个度为2的结点，有6个度为3的结点，有5个度为4的结点，有4个度为5的结点，有3个度为6的结点，有2个度为7的结点，该树一共有（ ）个叶子结点。 78

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

8.4 生成树和最小生成树

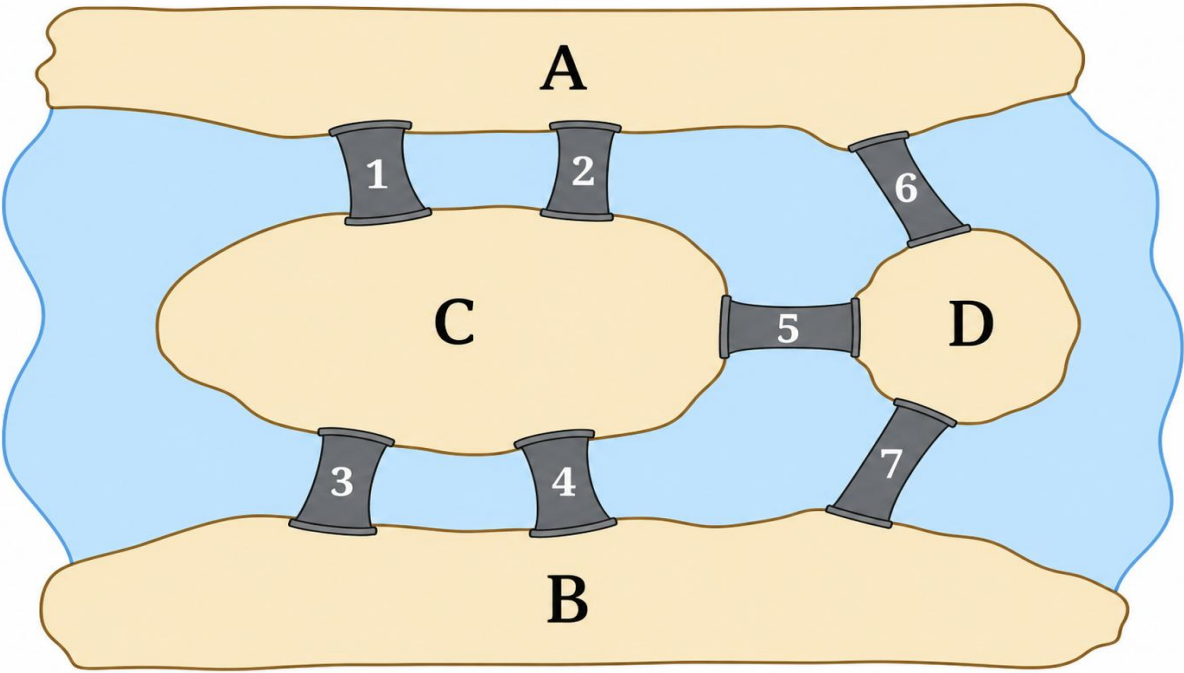
8.5 最短路径

8.6 拓扑排序

8.7 AOE网与关键路径

图是现实问题的抽象，例如七桥问题

柯尼斯堡七桥问题示意图



陆地 = 顶点，桥 = 边

是否存在一条路线，可以不重复地走完所有 7 座桥？

图论的起源

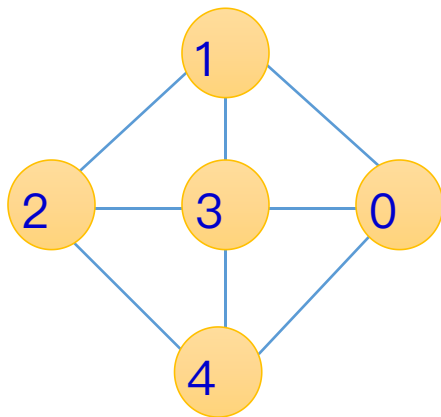
图的应用：地图导航



8.1 图的基本概念

8.1.1 图的定义

 (Graph) G 由顶点集合 $V(G)$ 和边集合 $E(G)$ 构成。



二元组

● 数据集合 $\Leftrightarrow$ 顶点集合

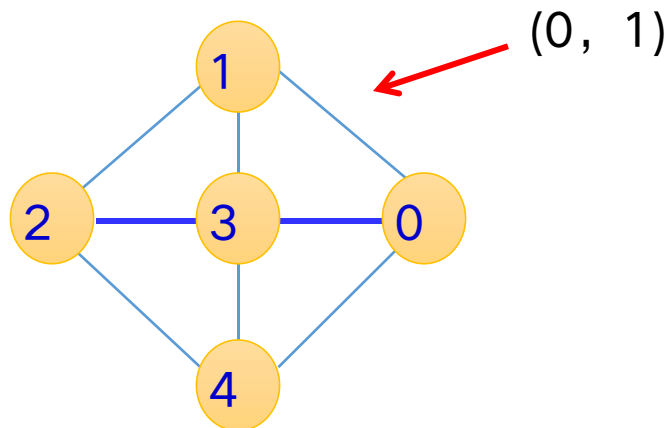
● 关系 $r_j \Leftrightarrow$ 边集合

↑
序偶

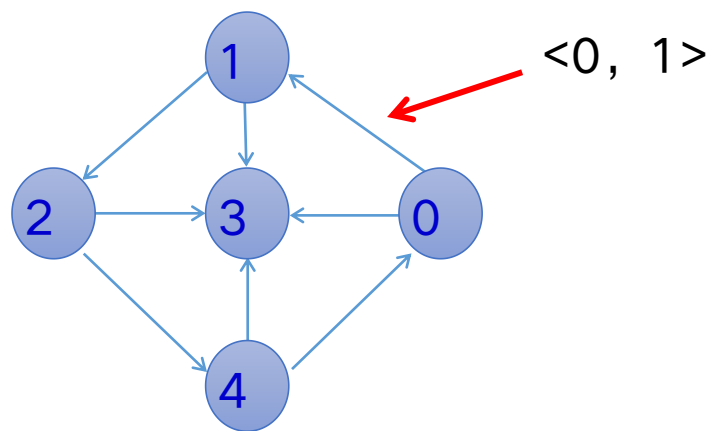
↑
边

说明：对于 n 个顶点的图，对每个顶点连续编号，即顶点的编号为 $0 \sim n-1$ 。通过编号唯一确定一个顶点。

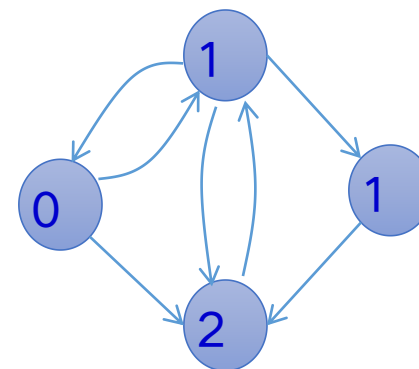
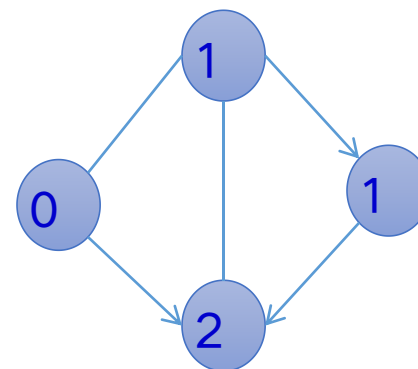
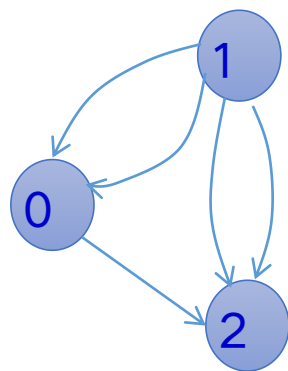
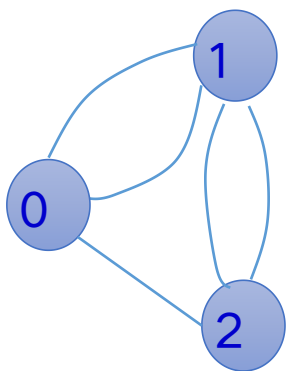
在图G中，如果代表边的顶点对是无序的，则称G为**无向图**。用圆括号序偶表示无向边。



如果表示边的顶点对是有序的，则称G为**有向图**。用尖括号序偶表示有向边。



数据结构中讨论的图都是简单图，不包含如下多重图等



图抽象数据类型 = 逻辑结构 + 基本运算（运算描述）

图的基本运算如下：

- ① CreateGraph(&g): 创建图。
- ② DestroyGraph(&g): 销毁图。
- ③ DispGraph(g): 输出图。输出边和顶点
- ④ DFS(g, v): 从顶点 v 出发深度优先遍历。
- ⑤ BFS(g, v): 从顶点 v 出发广度优先遍历。
- ...



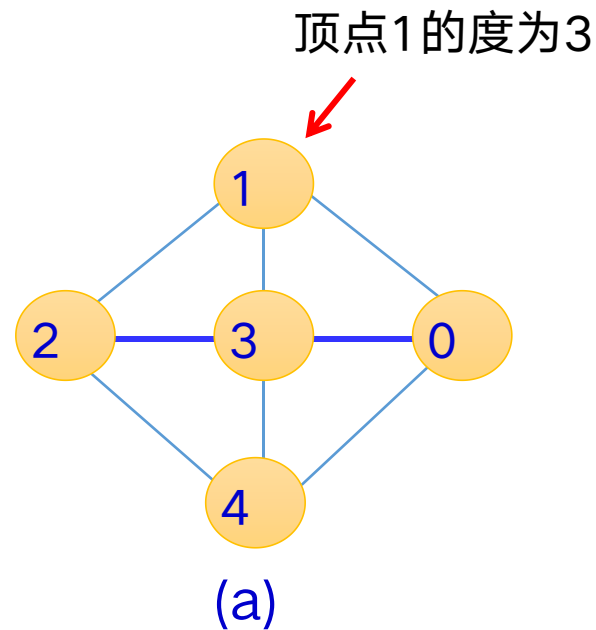
8.1.2 图的基本术语

1、端点和邻接点

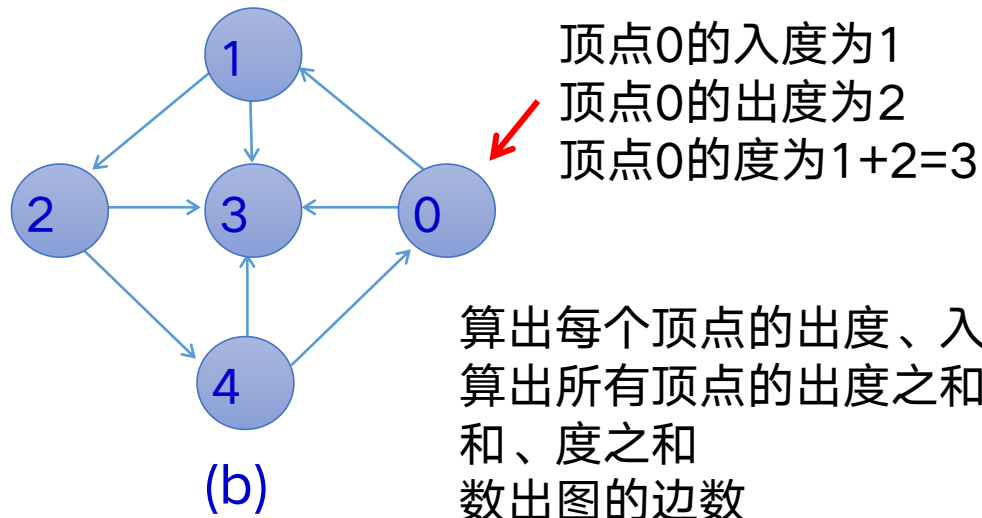
- 无向图中，若存在一条边 (i, j) $\Rightarrow$ 顶点 i 和顶点 j 为端点，它们互为邻接点。
- 有向图中，若存在一条边 $\langle i, j \rangle$ $\Rightarrow$ 顶点 i 为起始端点（简称为起点），顶点 j 为终止端点（简称终点），它们互为邻接点。

2、顶点的度、入度和出度

- 无向图中，以顶点为端点的边数称为该顶点的度。
- 有向图中，以顶点为终点的入边的数目，称为该顶点的入度。以顶点为始点的出边的数目，称为该顶点的出度。一个顶点的入度与出度的和为该顶点的度。



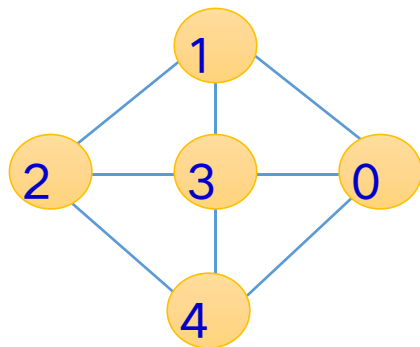
算出每个顶点的度
算出所有顶点的度之和
数出图的边数



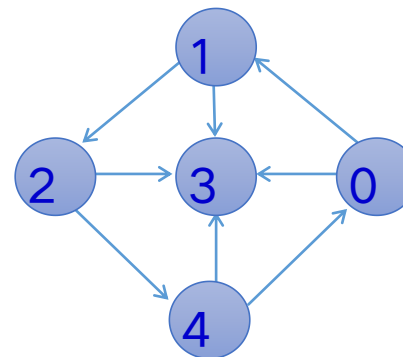
算出每个顶点的出度、入度、度
算出所有顶点的出度之和、入度之和、度之和
数出图的边数

若一个图中有 n 个顶点和 e 条边，每个顶点的度为 d_i ($0 \leq i \leq n-1$)，则有：

$$e = \frac{1}{2} \sum_{i=0}^{n-1} d_i$$



$n=5, e=8$
 $d_0=3, d_1=3, d_2=3,$
 $d_3=4, d_4=3$
 $(d_0+d_1+d_2+d_3+d_4)/2 =$
8



$n=5, e=8$
 $d_0=3, d_1=3, d_2=3,$
 $d_3=4, d_4=3$
 $(d_0+d_1+d_2+d_3+d_4)/2 =$
8

在有 n 个顶点的有向图中，每个顶点的度最大可达（ ）

A. n

B. $n-1$

C. $2n$

D. $2n - 2$

每个顶点可以有 $n-1$ 个入度和 $n-1$ 个出度，所以答案：D

在有向图中，所有顶点的入度之和等于所有顶点的出度之和的____倍。

A. 1 B. $1/2$ C. 2 D. 4

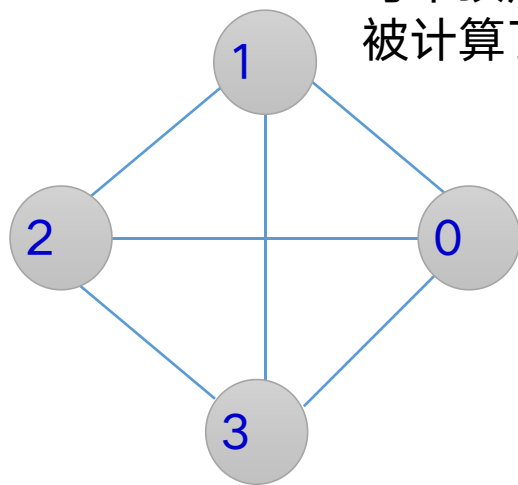
答案：A

3、完全图

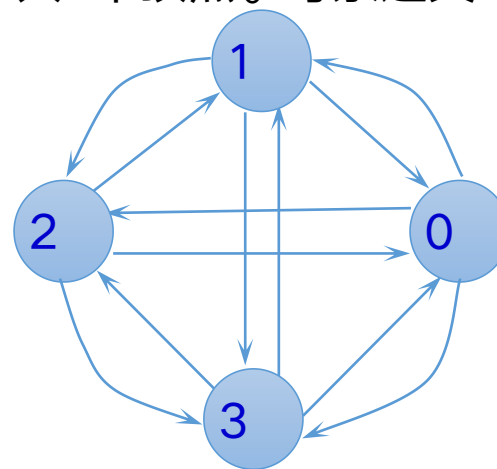
每个顶点有 $(n-1)$ 条边，一共 n 个顶点。每条边又被计算了两次，所以 $n(n-1)/2$

- 无向图中，每两个顶点之间都存在着一条边，称为**完全无向图**，包含有 $n(n-1)/2$ 条边。
- 有向图中，每两个顶点之间都存在着方向相反的两条边，称为**完全有向图**，包含有 $n(n-1)$ 条边。

每个顶点有 $2(n-1)$ 条边，一共 n 个顶点。每条边又被计算了两次，所以 $n(n-1)$



完全无向图： $n=4$ ，
 $e=n(n-1)/2=6$



完全有向图： $n=4$ ，
 $e=n(n-1)=12$

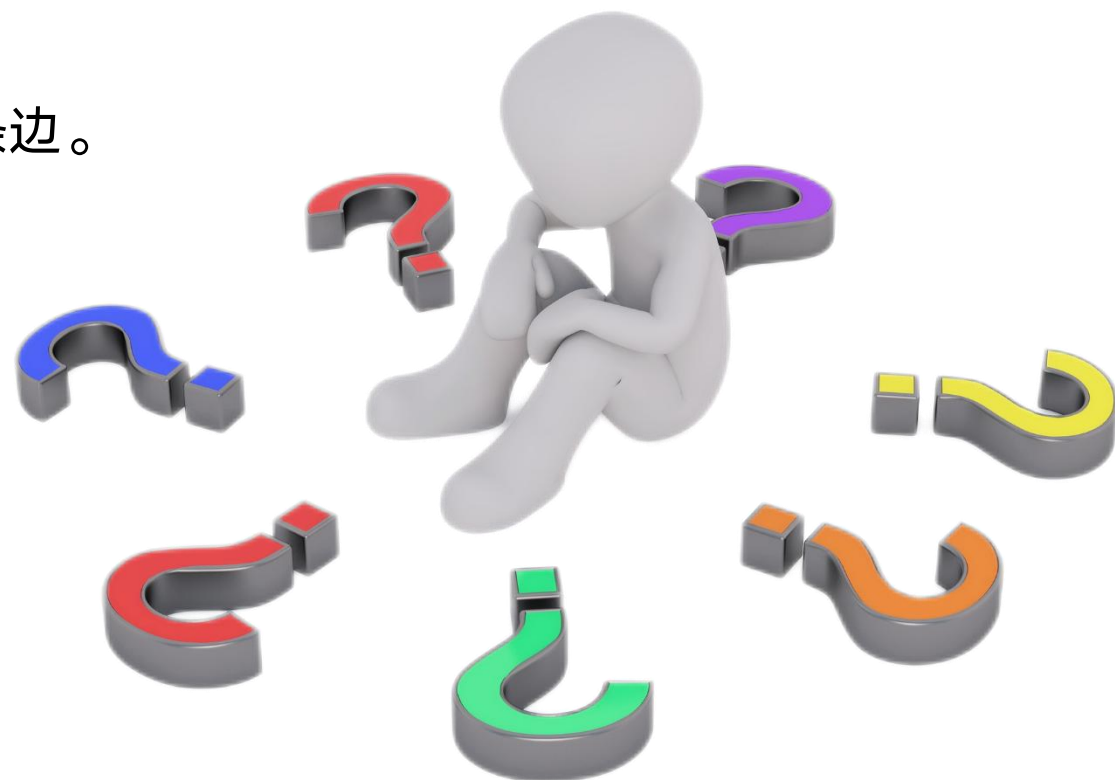
示例

设G是一个含有6个顶点的无向图，该图至多有（ ）条边。

A. 5 B. 6 C. 7 D. 15

边数最多时为完全无向图，有 $n(n-1)/2=15$ 条边。

答案为D。



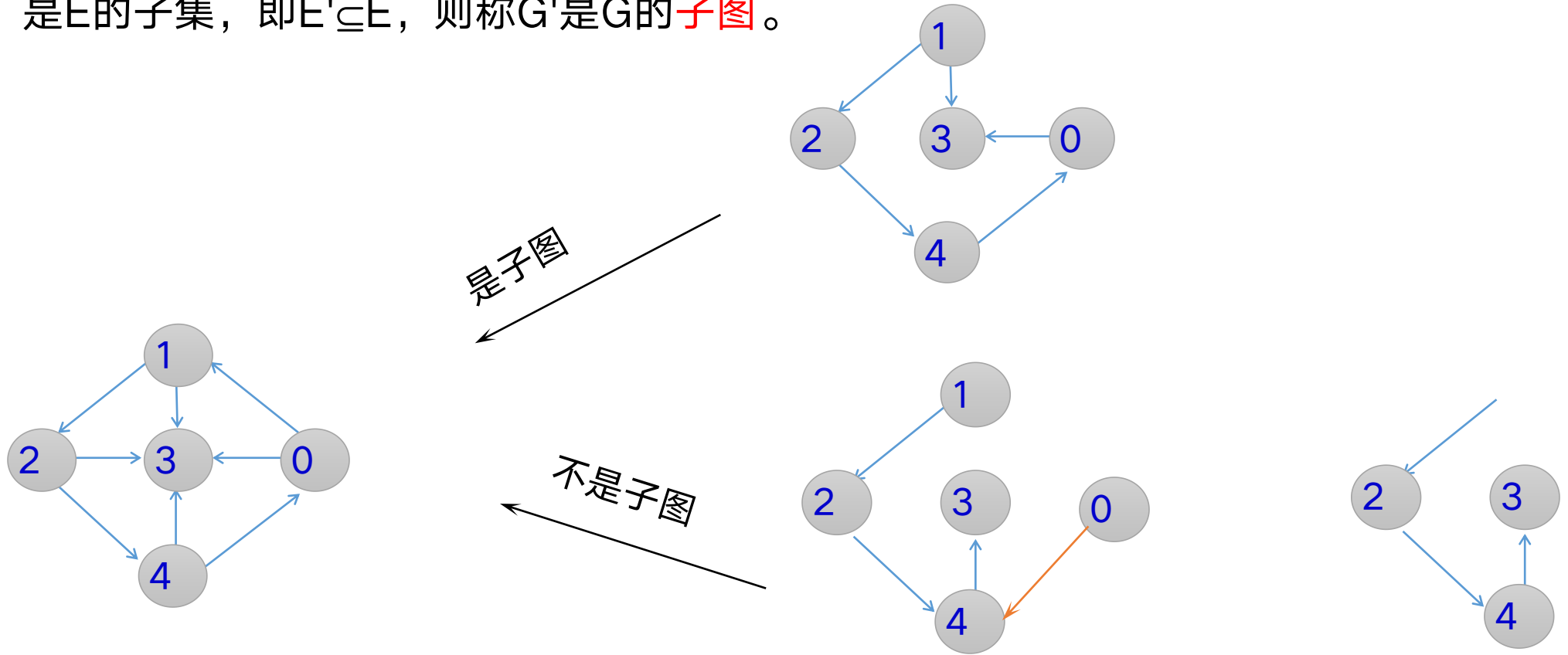
4、稠密图、稀疏图

- 当一个图接近完全图时，则称为稠密图。
- 相反，当一个图含有较少的边数（即当 $e \ll n(n-1)$ ）时，则称为稀疏图。

定性而不是定量描述

5、子图

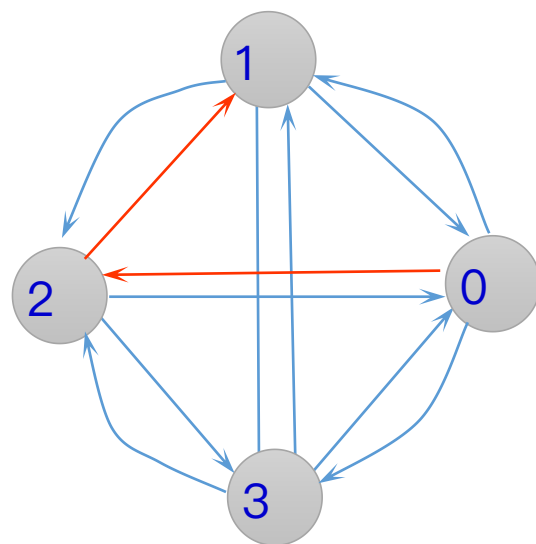
设有两个图 $G=(V, E)$ 和 $G'=(V', E')$ ，若 V' 是 V 的子集，即 $V' \subseteq V$ ，且 E' 是 E 的子集，即 $E' \subseteq E$ ，则称 G' 是 G 的**子图**。



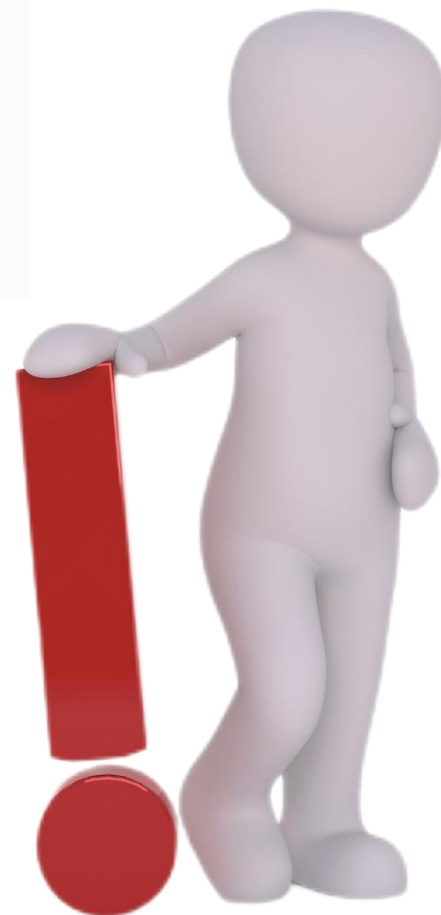
提问：任意 V' 是 V 的子集，且 E' 是 E 的子集，都能得到一个图 G' 是 G 的子图？

6、路径和路径长度

- 在一个图 $G=(V, E)$ 中, 从顶点 i 到顶点 j 的一条**路径** $(i, i_1, i_2, \dots, i_m, j)$ 。其中, 所有的 $(i_x, i_y) \in E(G)$, 或者 $\langle i_x, i_y \rangle \in E(G)$
- **路径长度**是指一条路径上经过的边的数目。
- 若一条路径上除开始点和结束点可以相同外, 其余顶点均不相同, 则称此路径为**简单路径**。

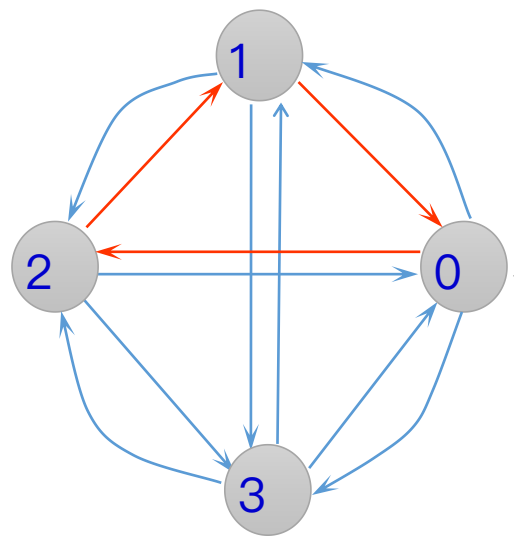


0到1的一条简单路径为
 $0 \rightarrow 2 \rightarrow 1$, 长度为2

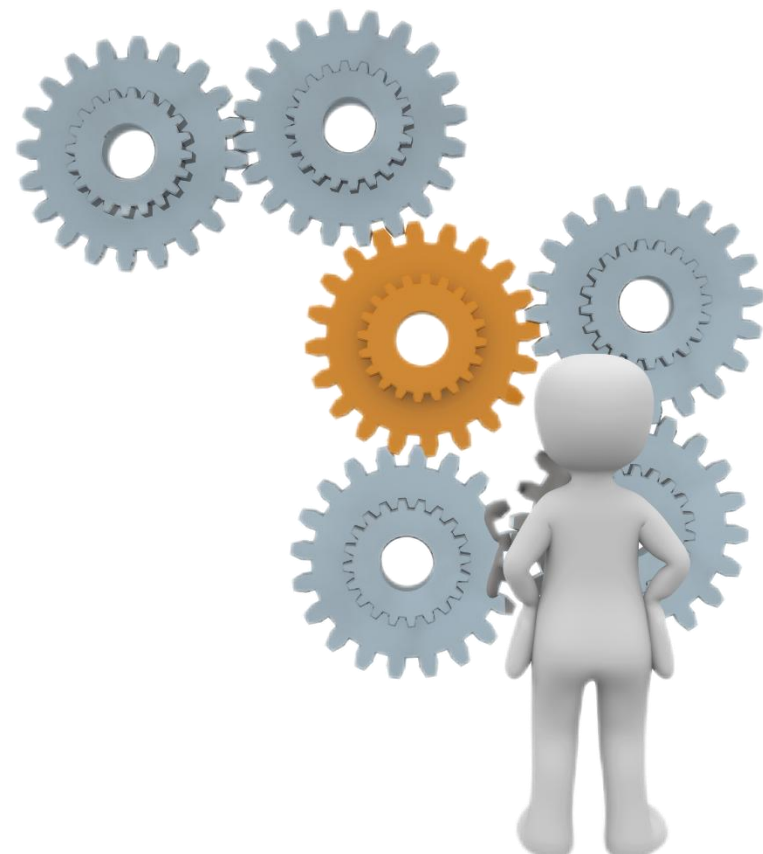


7、回路或环

若一条路径上的开始点与结束点为同一个顶点，则此路径被称为回路或环。开始点与结束点相同的简单路径被称为简单回路或简单环。



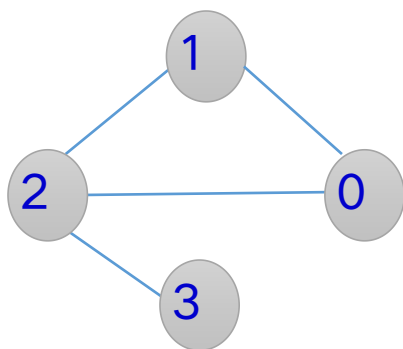
(0, 2, 1, 0)就是一条简单回路，其长度为3。



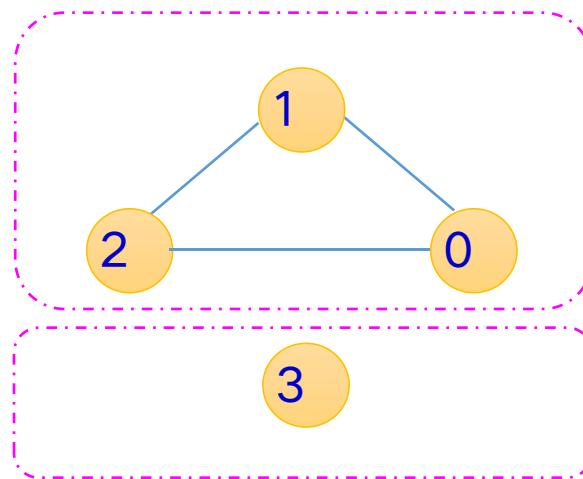
8、连通、连通图和连通分量

无向图中：

- 若从顶点 i 到顶点 j 有路径，则称顶点 i 和 j 是**连通**的。
- 若图中任意两个顶点都连通，则称为**连通图**，否则称为**非连通图**。
- 无向图 G 中的极大连通子图称为 G 的**连通分量**。任何连通图的连通分量只有一个，即本身，而非连通图有多个**连通分量**。

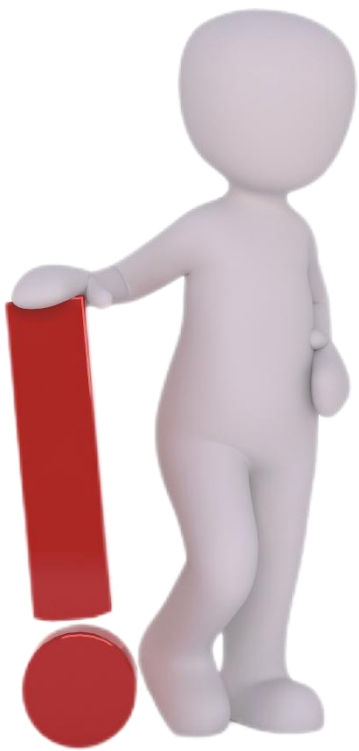


一个连通图



两个连通分量

一个非连通图





设G是一个连通无向图，有15条边，则该图至少有()个顶点。

A. 5 B. 6 C. 7 D. 15

设G是一个非连通无向图，有15条边，则该图至少有()个顶点。

A. 5 B. 6 C. 7 D. 15

- 要使顶点个数最少并且为非连通无向图，图由两个连通分量构成：完全无向图+单个顶点。
- 完全无向图： $n(n-1)/2=15 \Rightarrow n=6$ 。
- 单个顶点：1个顶点。
- 对应的图有 $6+1=7$ 个顶点。

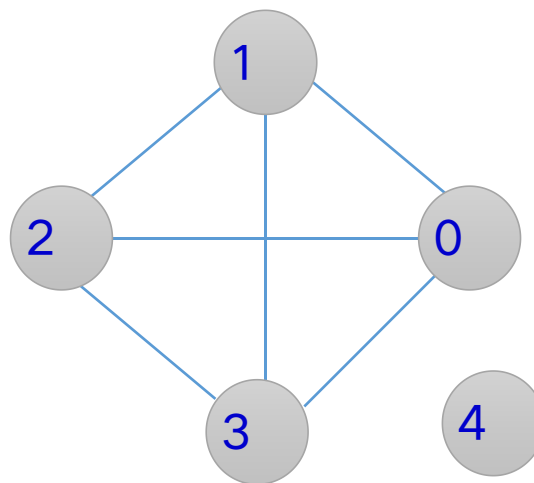
答案为B/C。



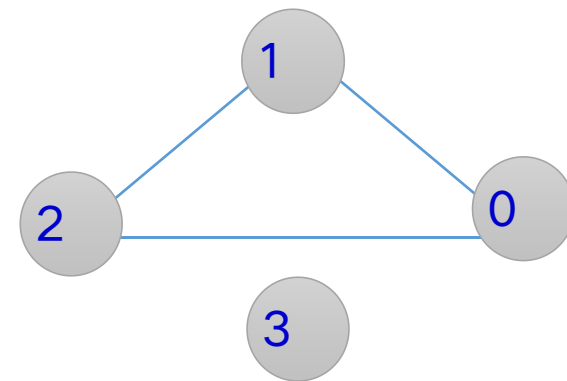
对于 n 个顶点， m 条边的无向图 G ，说法正确的是_____。

- A. 若 $m > n$ ，则 G 必连通
- B. 若 $m < n$ ，则 G 必不连通
- C. 若 $m \geq n$ ，则 G 中必含回路
- D. 若 $m < n$ ，则 G 中必不含回路

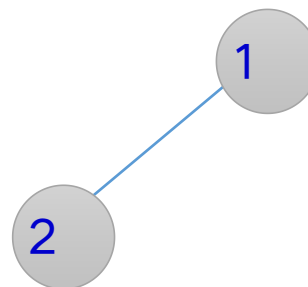
答案： C



A



D



B

已知无向图 G 如下：

顶点： $\{1, 2, 3, 4, 5\}$

边： $(1,2), (1,3), (2,3), (2,4), (3,5), (4,5)$

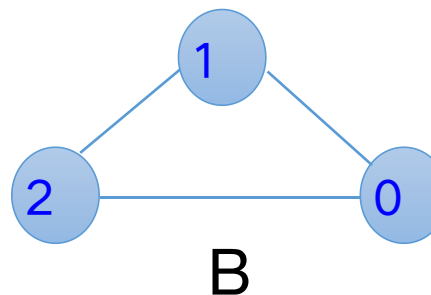
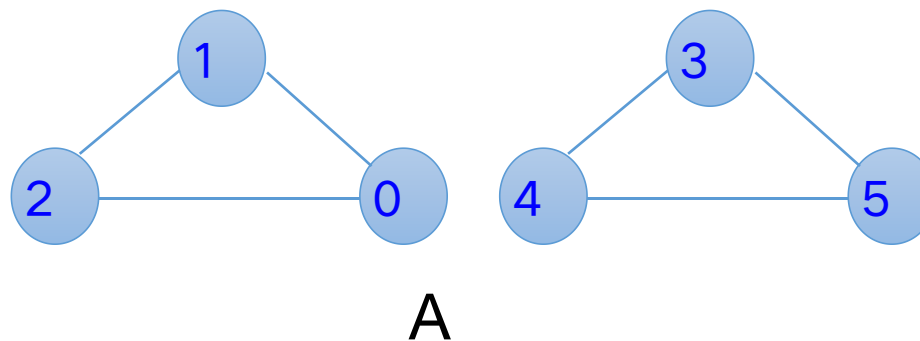
- ① 顶点1的度为 _____，顶点5的度为 _____；
- ② 该图是否为连通图：_____；
- ③ 该图共有 _____条边，各顶点度数之和为_____.

答案：

- (1) 2
- (2) 2
- (3) 是
- (4) 6
- (5) 12

一个有 n 个顶点和 n 条边的无向图一定是()

- A. 连通的
- B. 不连通的
- C. 无环的
- D. 有环的



答案: D

(2022考研) 对于无向图 $G = (V, E)$ ，下选项中正确的是 ()。

A. 当 $|V| > |E|$ ， G 一定连通

考虑孤点的情况

B. 当 $|V| < |E|$ ， G 一定连通

考虑孤点的情况，5个点，6条边

C. 当 $|V| = |E| - 1$ ， G 一定不连通

4个点，5条边

D. 当 $|V| > |E| + 1$ ， G 一定不连通

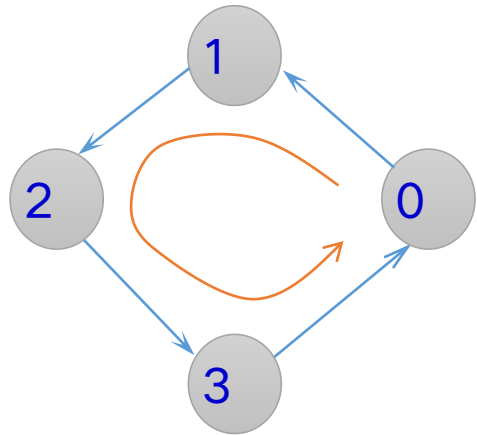
一个有 n 个顶点的连通无向图，至少要有 $n-1$ 条边。比如树就是边数最少的连通图

答案：D

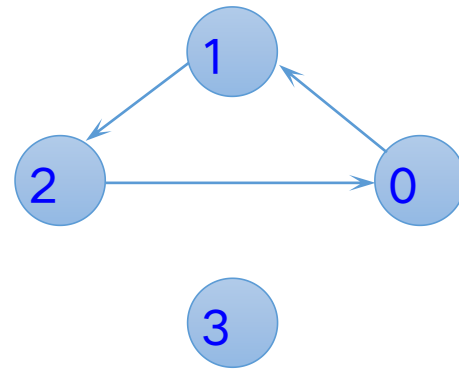
9、强连通图和强连通分量

有向图中：

- 若从顶点 i 到顶点 j 有路径，则称从顶点 i 到 j 是**连通**的。
- 若图 G 中的任意两个顶点 i 和 j 都连通，即从顶点 i 到 j 和从顶点 j 到 i 都存在路径，则称图 G 是**强连通图**。



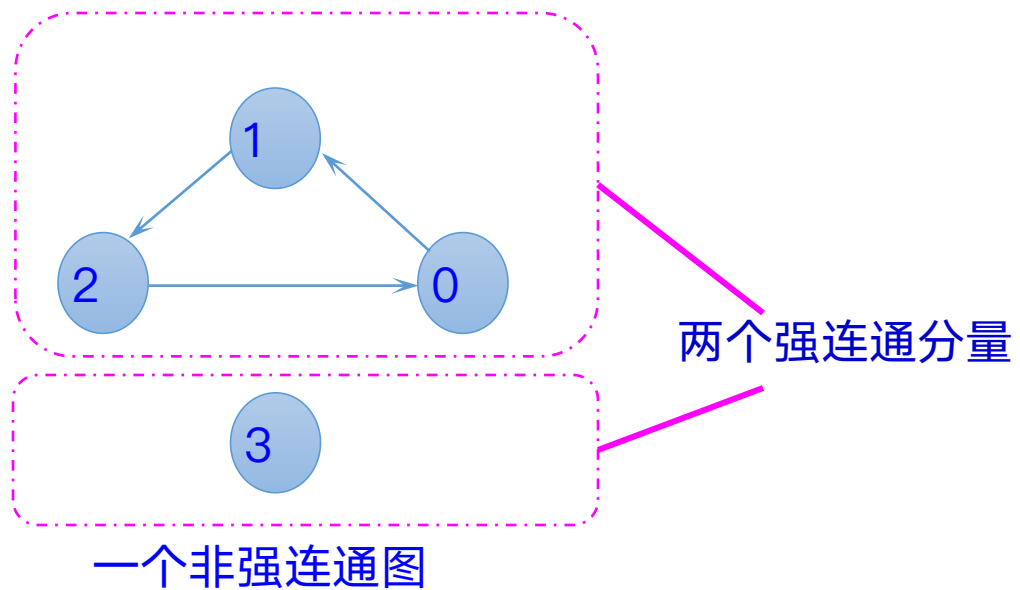
一个强连通图



一个非强连通图

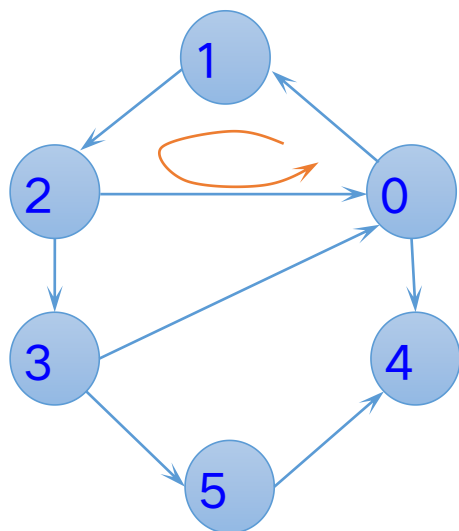


- 有向图G中的极大强连通子图称为G的**强连通分量**。
- 强连通图只有一个强连通分量，即本身。非强连通图有多个强连通分量。

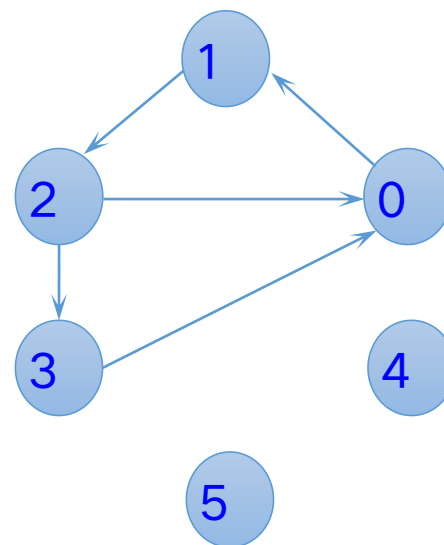


在一个非强连通中找**强连通分量**的方法：

- ① 在图中找有向环。
- ② 扩展该有向环：如果某个顶点到该环中任一顶点有路径，并且该环中任一顶点到这个顶点也有路径，则加入这个顶点。



一个非强连通图



3个强连通分量

找到这个图的所有强连通分量

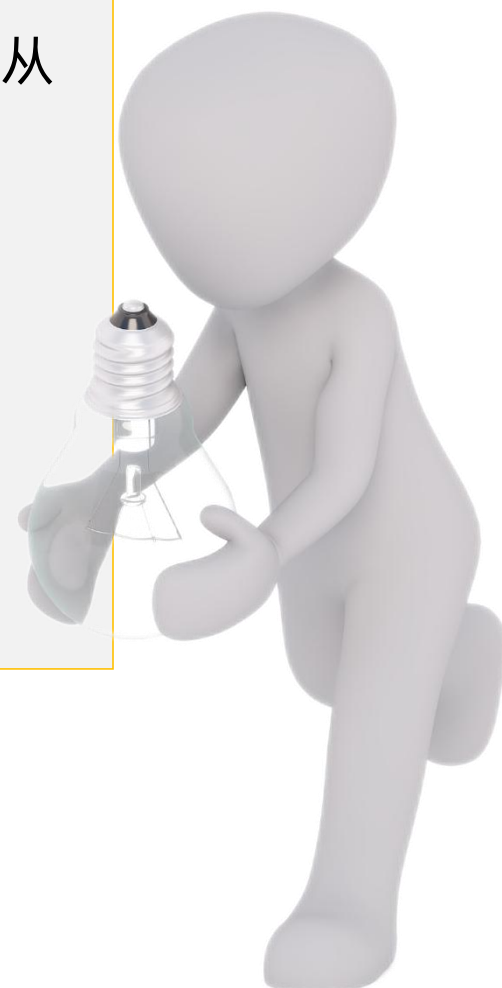
【例8.1】 n ($n>2$) 个顶点的强连通图至少有多少条边？这样的有向图是什么形状？

根据强连通图的定义可知，图中的任意两个顶点和 j 都连通，即从顶点 i 到顶点 j 和从顶点 j 到顶点 i 都存在路径。

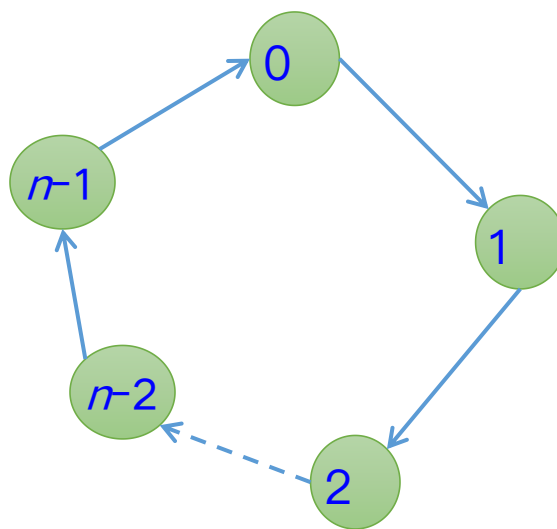
这样，每个顶点的度 $d_i \geq 2$ ，设图中总的边数为 e ，有：

$$e = \frac{1}{2} \sum_{i=0}^{n-1} d_i \geq \frac{1}{2} \sum_{i=0}^{n-1} 2 = n$$

即： $e \geq n$ 。因此， n 个顶点的强连通图至少有 n 条边。

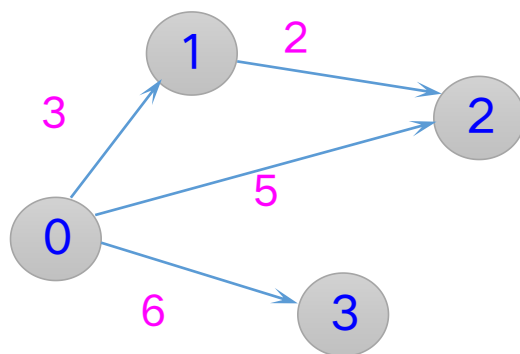


而只有 n 条边的强连通图是环形的，即顶点0到1有一条有向边，顶点1到2有一条有向边， $\dots$ ，顶点 $n-1$ 到0有一条有向边，如下图所示。

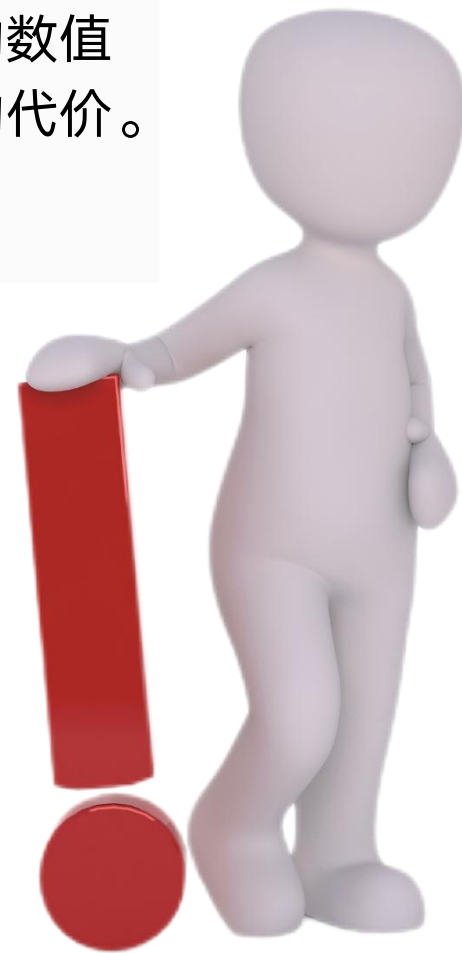


10、权和网

- 图中每一条边都可以附带有一个对应的数值，这种与边相关的数值称为**权**。权可以表示从一个顶点到另一个顶点的距离或花费的代价。
- 边上带有权的图称为**带权图**，也称作**网**。



一个网



提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

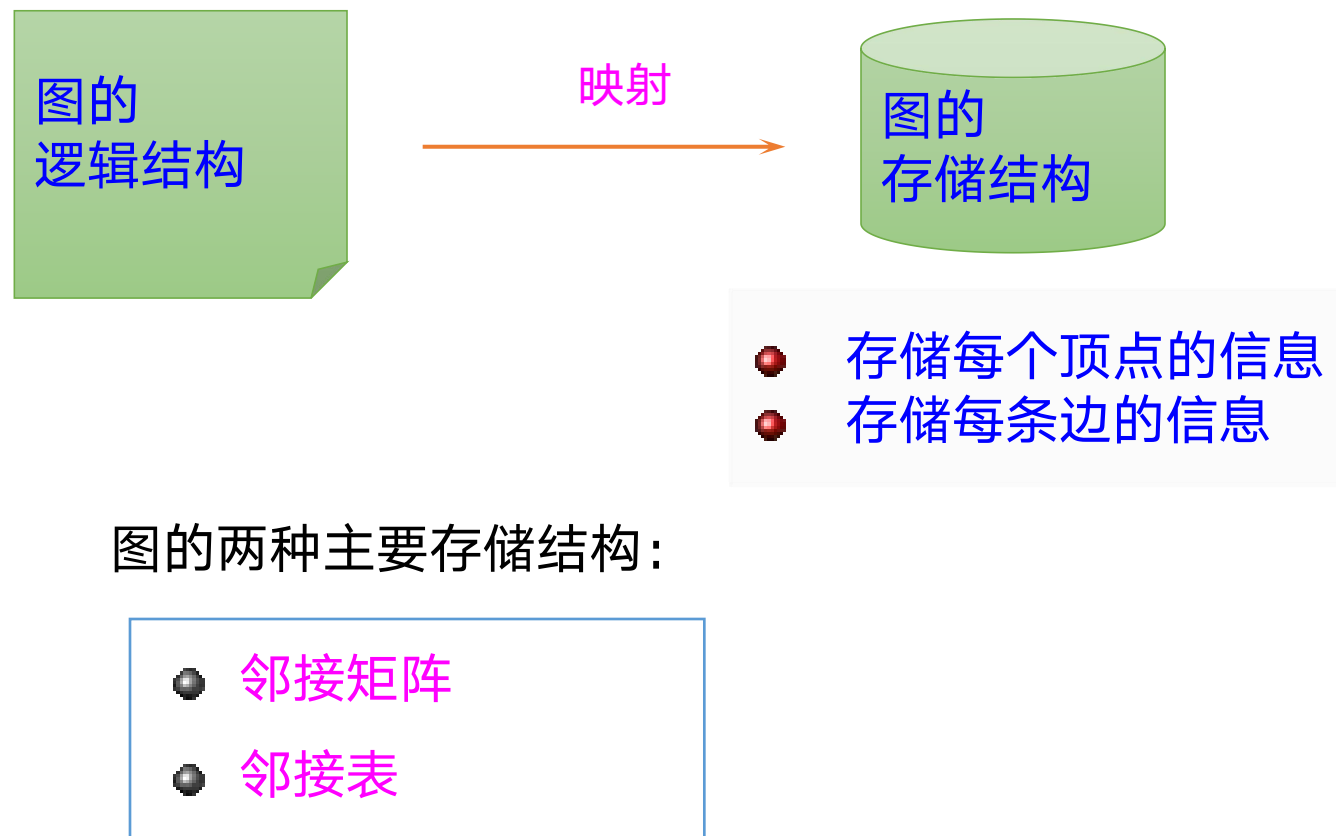
8.4 生成树和最小生成树

8.5 最短路径

8.6 拓扑排序

8.7 AOE网与关键路径

8.2 图的存储结构和基本运算算法



8.2.1 邻接矩阵存储方法

邻接矩阵是表示顶点之间相邻关系的矩阵。设 $G=(V, E)$ 是具有 n ($n>0$) 个顶点的图，顶点的编号依次为 $0 \sim n-1$ 。

G 的邻接矩阵 A 是 n 阶方阵，其定义如下：

(1) 如果 G 是**无向图**，则：

$A[i][j]=1$ ：若 $(i, j) \in E(G)$

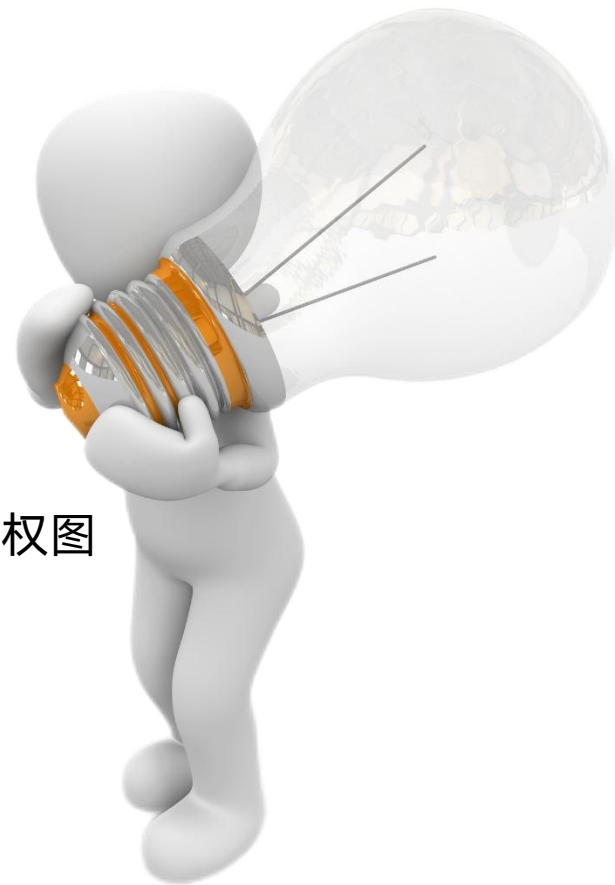
0：其他

(2) 如果 G 是**有向图**，则：

$A[i][j]=1$ ：若 $\langle i, j \rangle \in E(G)$

0：其他

不带权图



(3) 如果G是带权无向图，则：

$A[i][j] = w_{ij}$: 若 $i \neq j$ 且 $(i, j) \in E(G)$

0: $i = j$

∞ : 其他

(4) 如果G是带权有向图，则：

$A[i][j] = w_{ij}$: 若 $i \neq j$ 且 $\langle i, j \rangle \in E(G)$

0: $i = j$

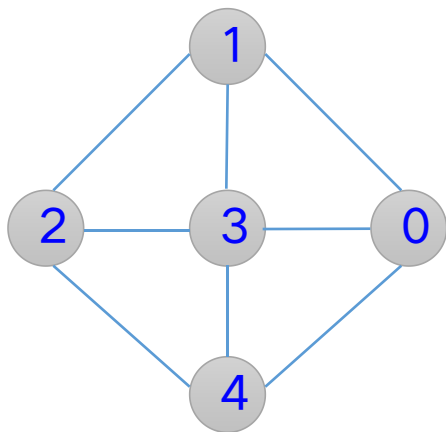
∞ : 其他

带权图



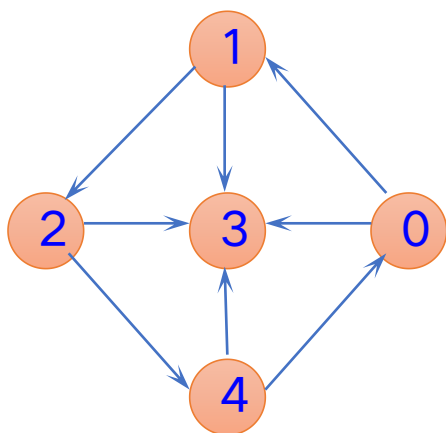
邻接矩阵演示

无向图：写出邻接矩阵，并且写出每个顶点的度



$$A_1 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

对称



$$A_2 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

不对称

有向图：写出邻接矩阵，并且写出每个顶点的出度、入度和度

邻接矩阵的主要特点：

- 一个图的邻接矩阵表示是唯一的。
- 特别适合于稠密图的存储。

邻接矩阵的存储空间为 $O(n^2)$



图的邻接矩阵存储类型定义如下：

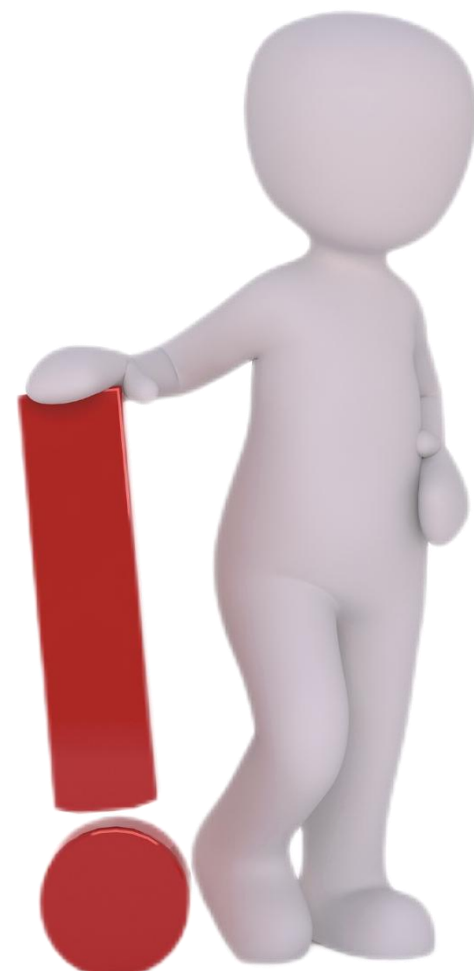
```
#define MAXV <最大顶点个数>

typedef struct
{
    int no;           //顶点编号
    InfoType info;    //顶点其他信息，如城市名称
} VertexType;

typedef struct        //图的定义
{
    int edges[MAXV][MAXV]; //邻接矩阵
    int n, e;             //顶点数，边数
    VertexType vexs[MAXV]; //存放顶点信息
} MatGraph;
```

声明顶点的类型

声明的邻接矩阵类型



【2013 统考题】 设图的邻接矩阵如下所示，各顶点的度依次是 ()

A. 1, 2, 1, 2

B. 2, 2, 1, 1

C. 3, 4, 2, 3

D. 4, 4, 2, 2

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

答案：C

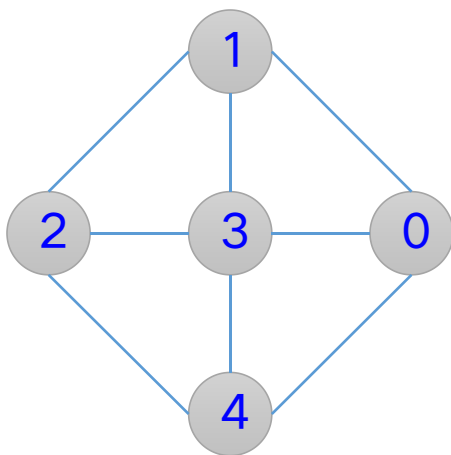
习题5.对于无向图的邻接矩阵, 说法正确的是_____。

- A.第 i 行上的非零元素个数和第 i 列的非零元素个数一定相等
- B.矩阵中的非零元素个数等于图中的边数
- C.第 i 行和第 i 列上非零元素总数等于顶点 i 的度数
- D.矩阵中的非全零行的行数等于图中的顶点数

答案: A

8.2.2 邻接表存储方法

- 对图中每个顶点建立一个单链表，将顶点的所有邻接点链起来。



顶点0的
单链表



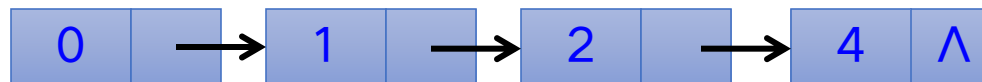
顶点1的
单链表



顶点2的
单链表



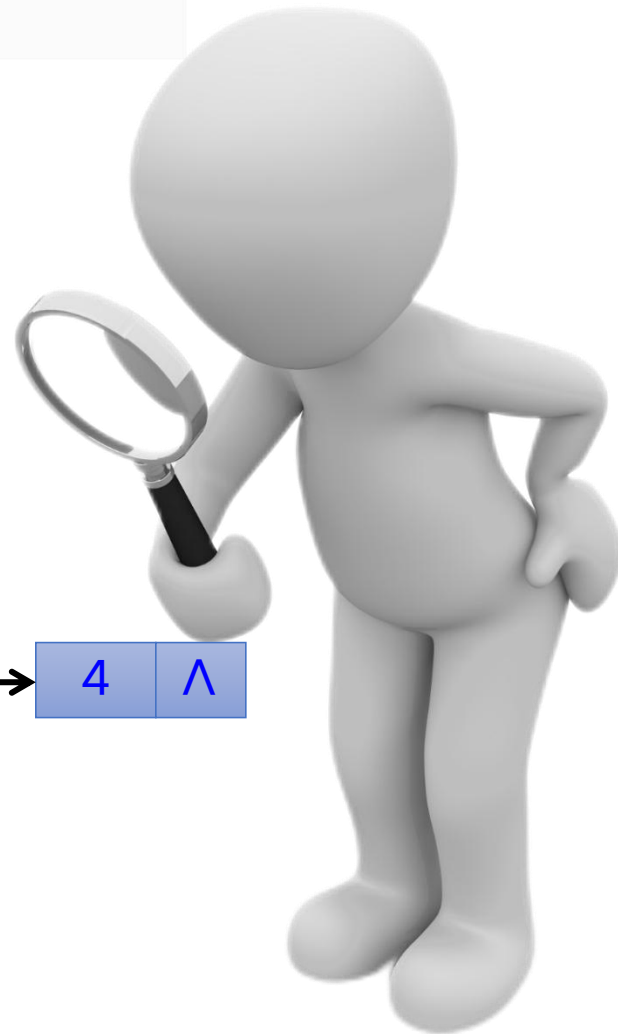
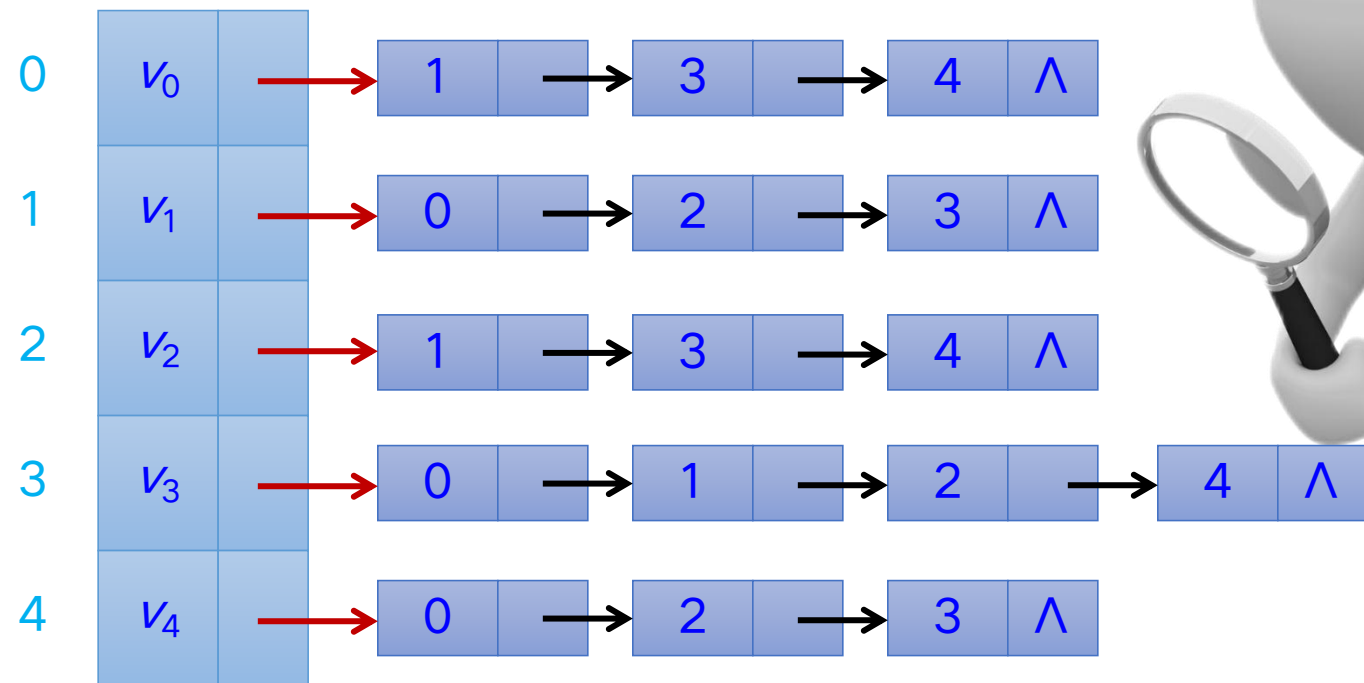
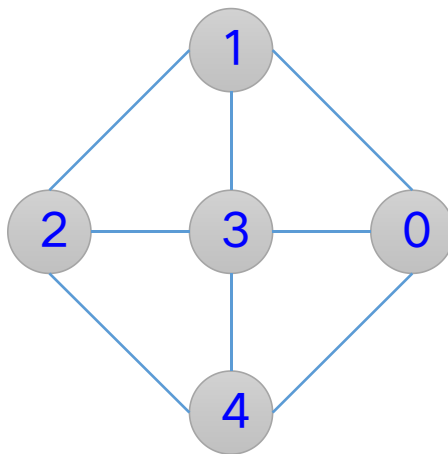
顶点3的
单链表



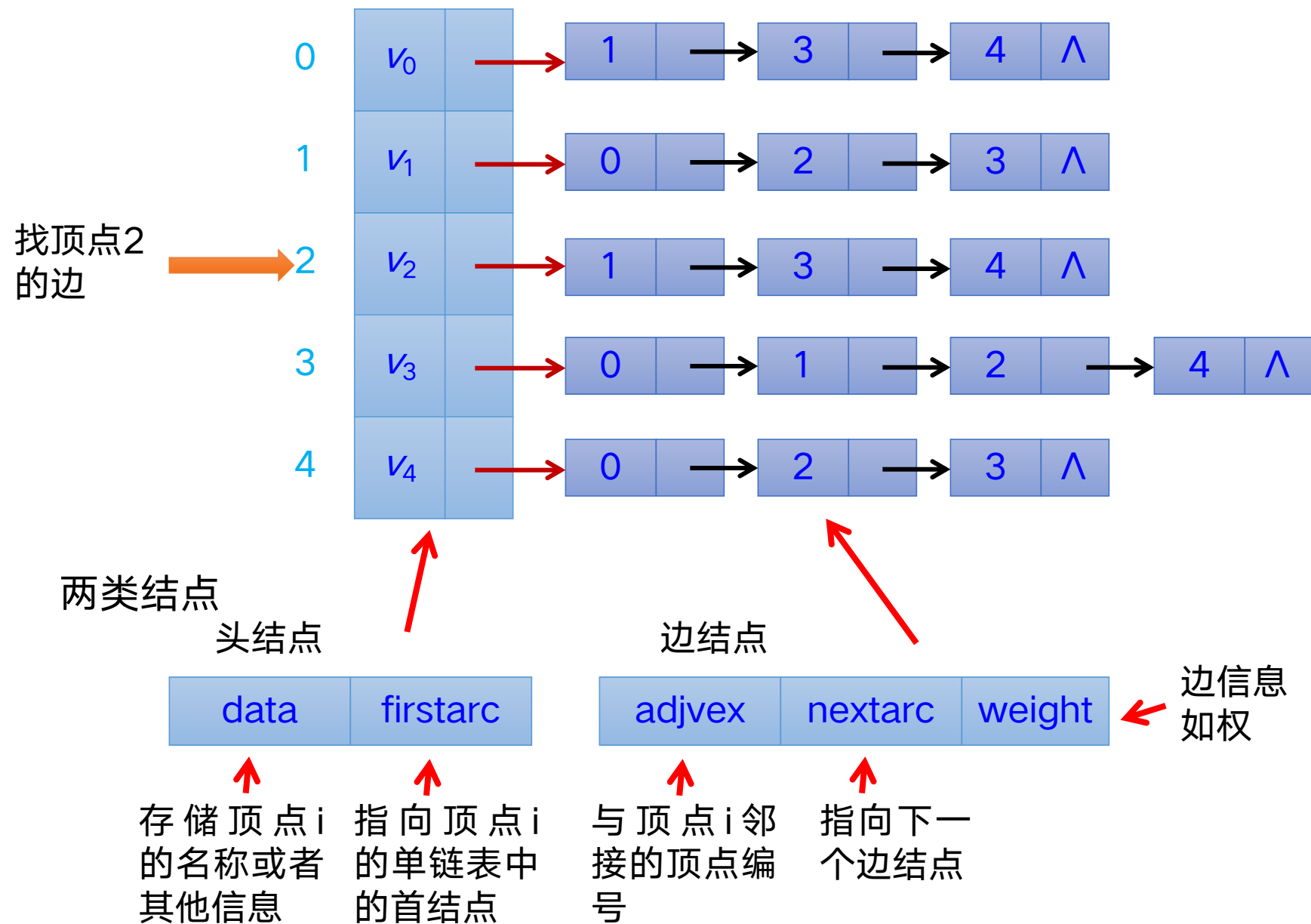
顶点4的
单链表



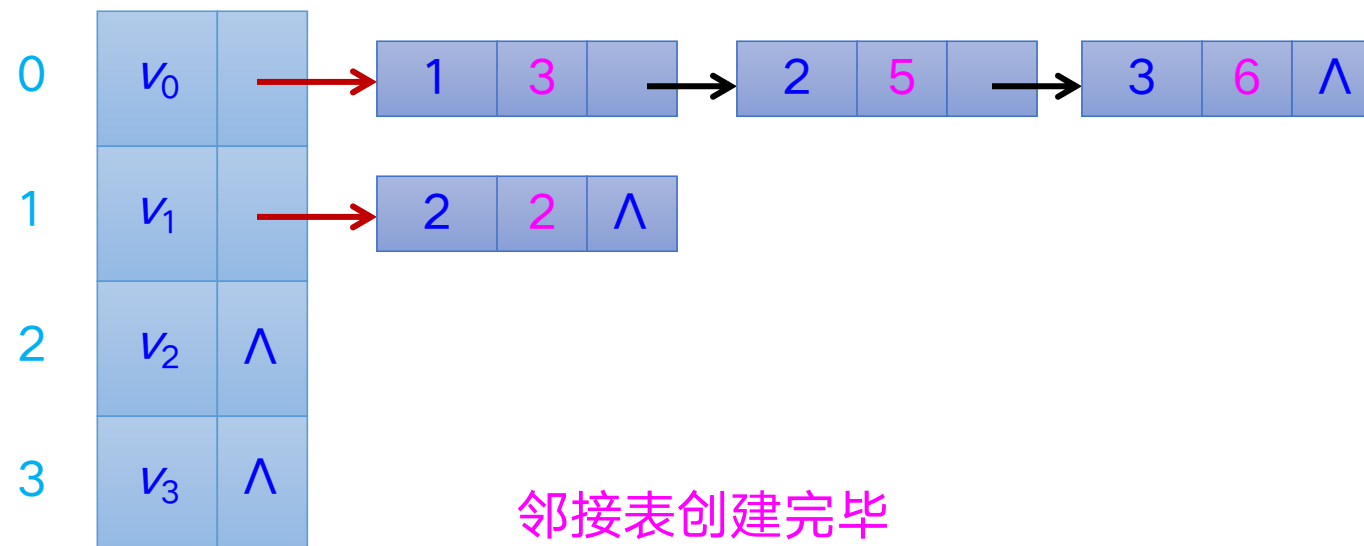
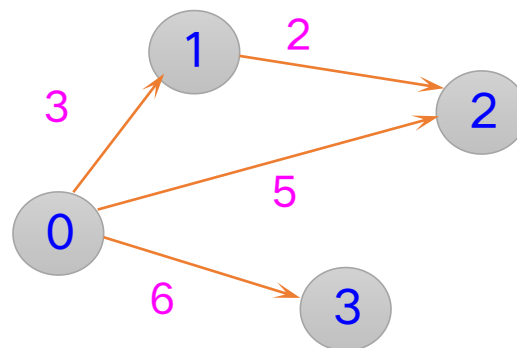
- 每个单链表上添加一个表头结点（表示顶点信息）。并将所有表头结点构成一个数组，下标为*i*的元素表示顶点*i*的表头结点。



图的邻接表存储方法是一种顺序分配与链式分配相结合的存储方法。

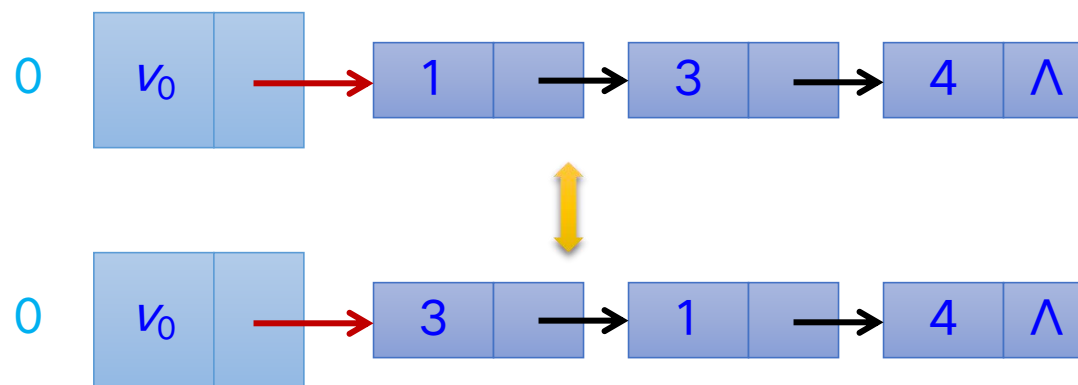
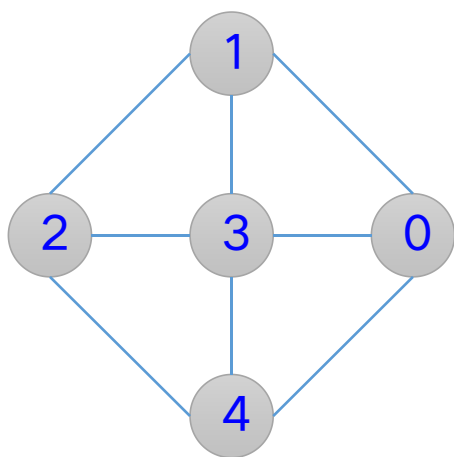


一个网



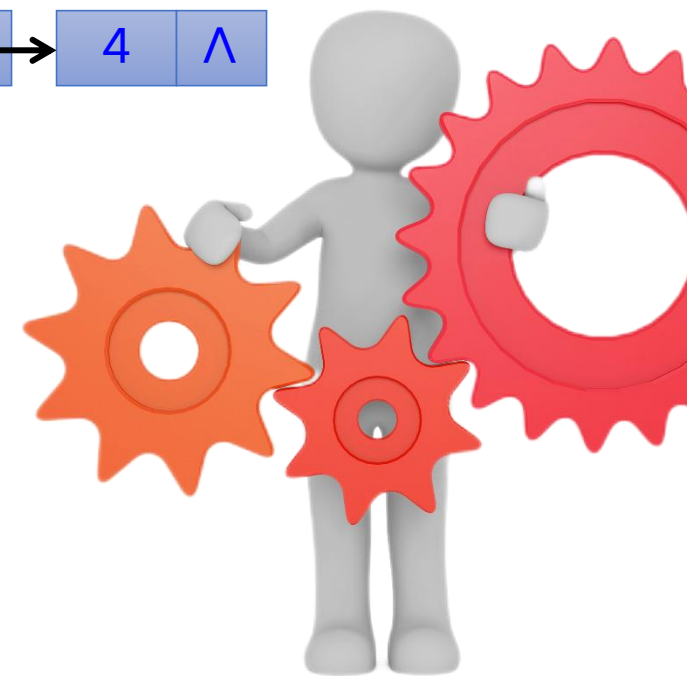
邻接表的特点如下：

- 邻接表表示不唯一。



- 特别适合于稀疏图存储。

邻接表的存储空间为 $O(n+e)$



图的邻接表存储类型定义如下：

```
typedef struct ANode
{ int adjvex;           //该边的终点编号
  struct ANode *nextarc; //指向下一条边的指针
  InfoType weight;      //该边的权值等信息
} ArcNode;
```

```
typedef struct Vnode
{ Vertex data;          //顶点信息
  ArcNode *firstarc;    //指向第一条边
} VNode;
```

```
typedef struct
{ VNode adjlist[MAXV]; //邻接表
  int n, e;            //图中顶点数 $n$ 和边数 $e$ 
} AdjGraph;
```

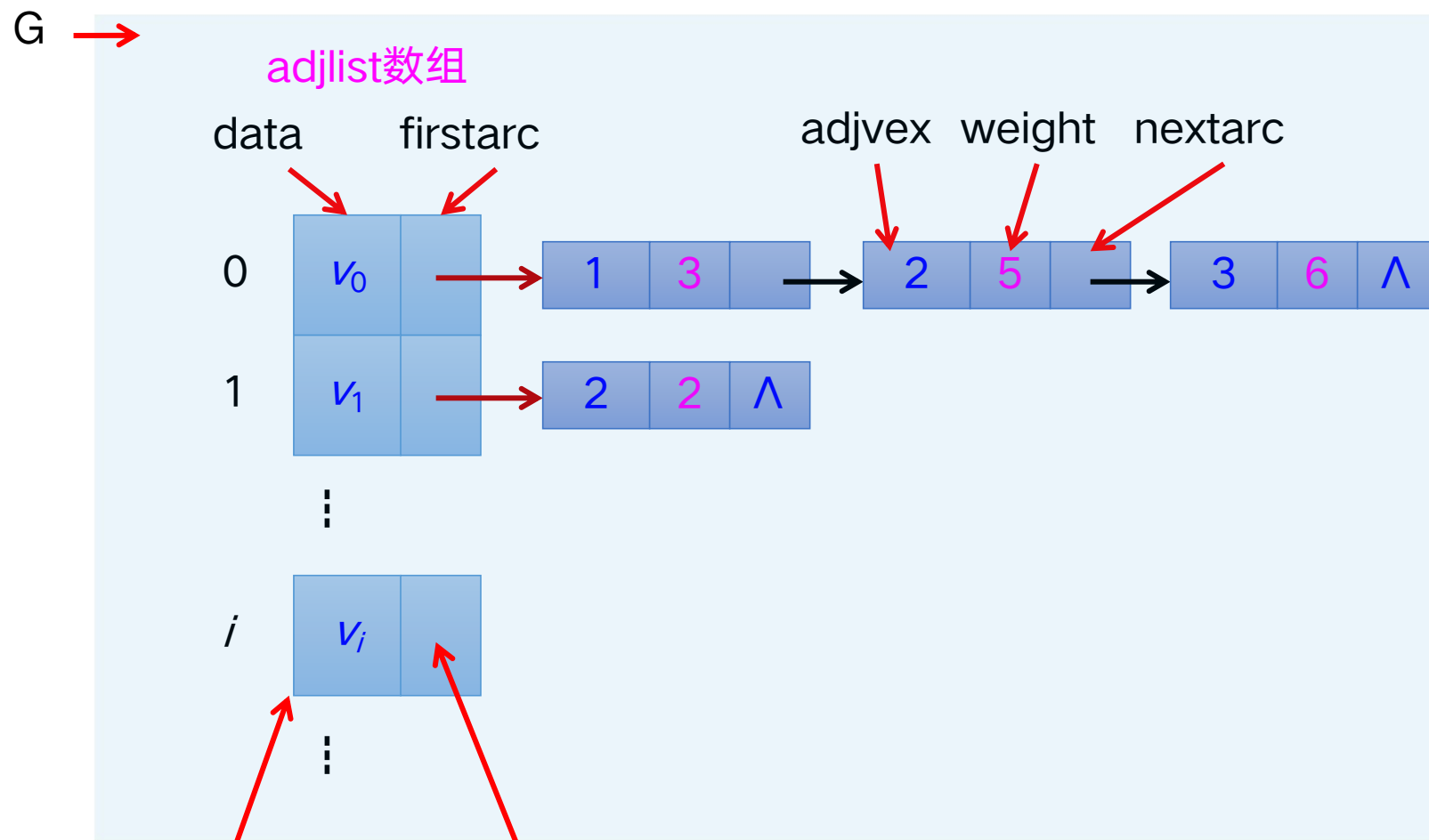
声明边结
点类型

声明邻接
表头结点
类型

声明图邻
接表类型



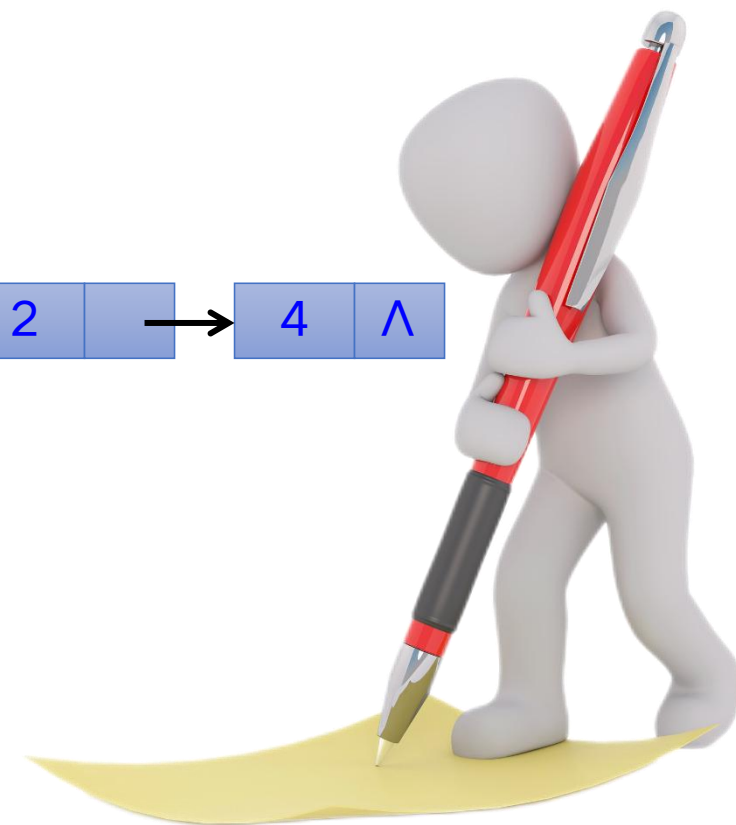
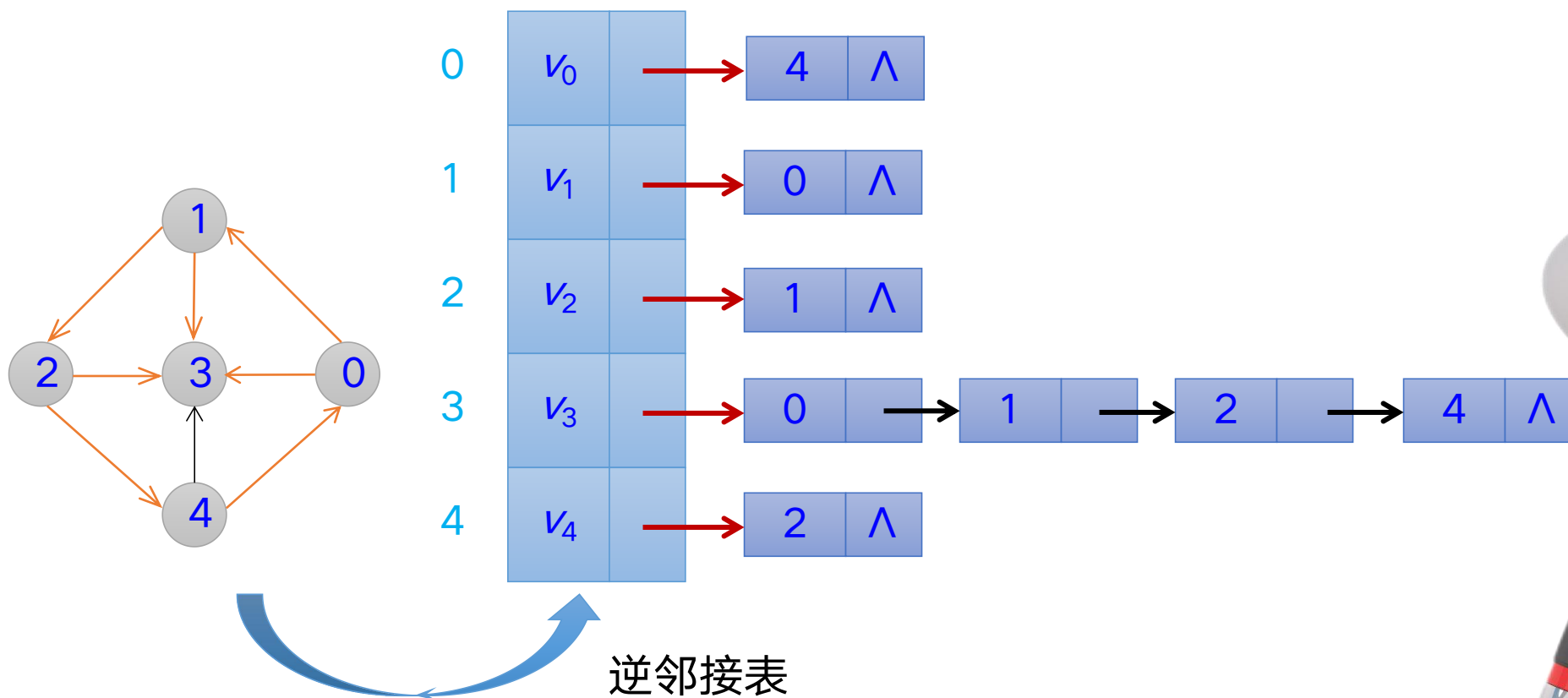
一个邻接表通常用指针引用：



引用头结点: $G \rightarrow \text{adjlist}[i]$

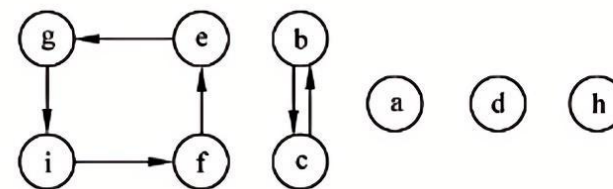
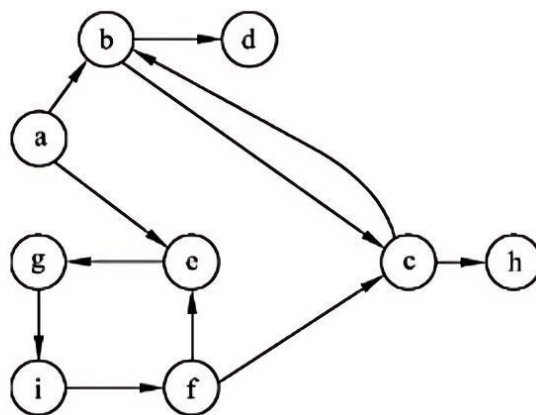
引用头结点的指针域: $G \rightarrow \text{adjlist}[i].\text{firstarc}$

逆邻接表：就是在有向图的邻接表中，对每个顶点，单链表链接的是起点。

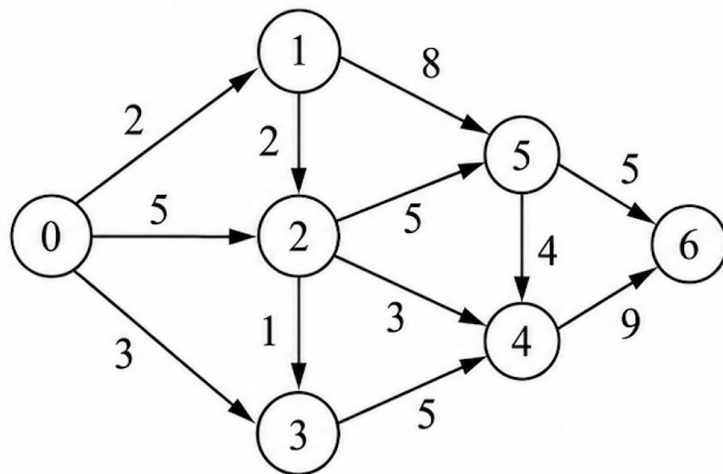


一个有向图 G 的邻接表存储如下图所示，给出该图的所有强连通分量。

0	a	→	1	→	4	∧
1	b	→	2	→	3	∧
2	c	→	1	→	7	∧
3	d	→	∧			
4	e	→	6	→	∧	
5	f	→	2	→	4	∧
6	g	→	8	→	∧	
7	h	→	∧			
8	i	→	5	→	∧	



有一个带权有向图如图 8.21 所示, 回答以下问题:



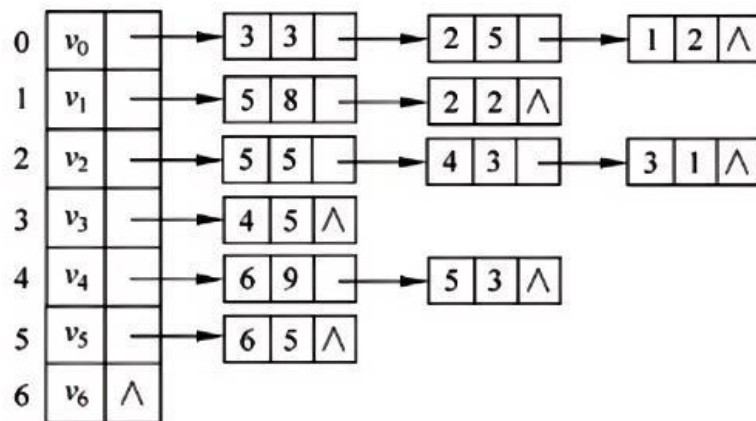
(1) 给出该图的邻接矩阵表示。

(2) 给出该图的邻接表表示。

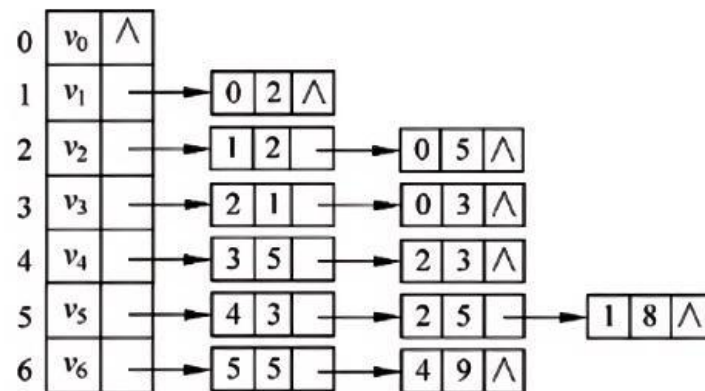
(3) 给出该图的逆邻接表表示。

0	2	5	3	∞	∞	∞
∞	0	2	∞	∞	8	∞
∞	∞	0	1	3	5	∞
∞	∞	∞	0	5	∞	∞
∞	∞	∞	∞	0	3	9
∞	∞	∞	∞	∞	0	5
∞	∞	∞	∞	∞	∞	0

(1)



(2)



(3)

8.2.3 图基本运算算法设计

- 图的基本运算算法主要有创建图、输出图和销毁图。
- 下面讨论邻接表的相关算法设计。



(1) 创建图的运算算法

根据邻接矩阵数组 A 、顶点个数 n 和边数 e 来建立图的邻接表 G （采用邻接表指针方式）。

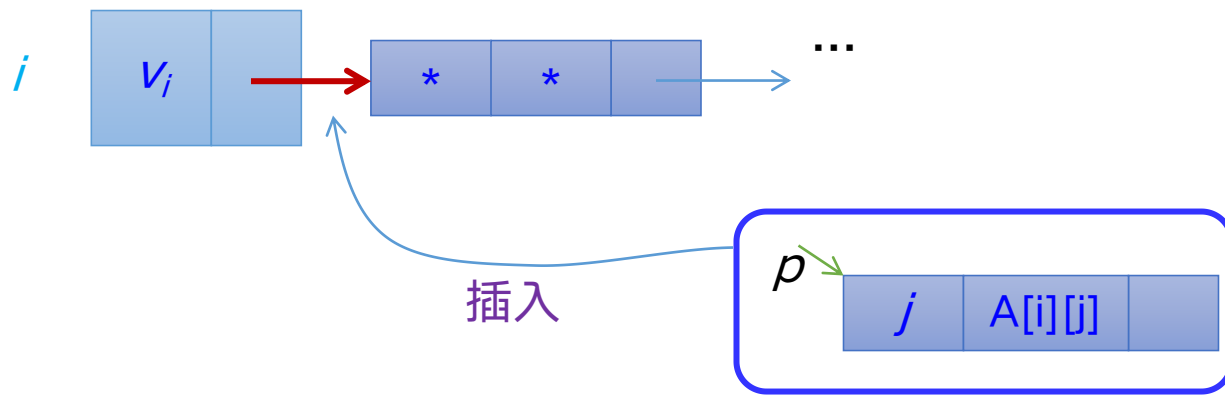
```
void CreateAdj(AdjGraph *&G, int A[MAXV][MAXV], int n, int e)
//创建图的邻接表
{ int i, j;
  ArcNode *p;
  G=(AdjGraph *)malloc(sizeof(AdjGraph));
  for (i=0;i<n;i++)          //给邻接表中所有头结点的指针域置初值
    G->adjlist[i].firstarc=NULL;
```




```

for (i=0;i<n;i++)           //遍历邻接矩阵中每个元素
    for (j=n-1;j>=0;j--)
        if (A[i][j]!=0 && A[i][j]!=INF) //存在一条边
        { p=(ArcNode *)malloc(sizeof(ArcNode)); //创建一个结点p
          p->adjvex=j;           //存放邻接点
          p->weight=A[i][j];     //存放权
          p->nextarc=G->adjlist[i].firstarc; //采用头插法插入结点p
          G->adjlist[i].firstarc=p;
        }
    G->n=n; G->e=e;
}

```

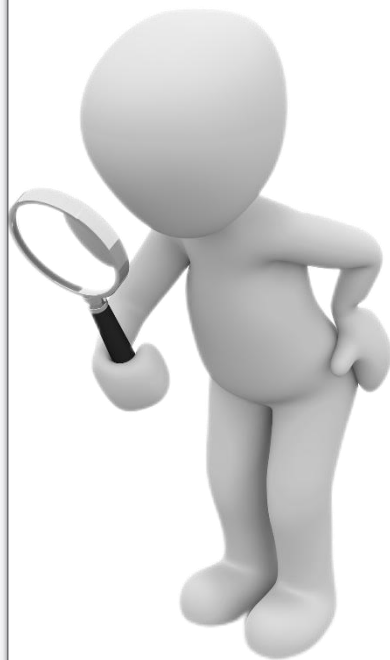


(2) 输出图的运算算法

```
void DispAdj(AdjGraph *G) //输出邻接表G
{ int i;
  ArcNode *p;
  for (i=0;i<G->n;i++) // 依次遍历邻接表中的每个顶点
  { p=G->adjlist[i].firstarc; // p 指向第 i 个顶点的第一条边
    printf("%3d: ", i); // 输出当前顶点的编号
    while (p!=NULL) // 遍历顶点 i 的所有邻接点
    { printf("%3d[%d]→", p->adjvex, p->weight); // 输出邻接点编号和边的权值
      p=p->nextarc; // 指向下一条边
    }
    printf(" ^\n");
  }
}
```

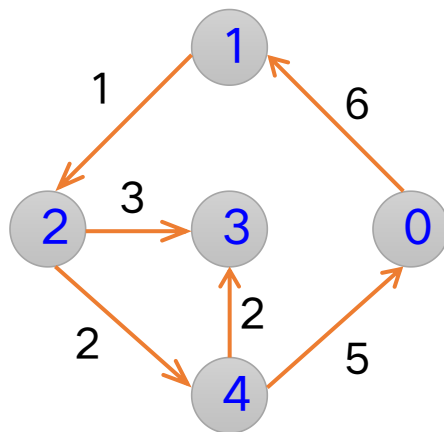
(3) 销毁图的运算算法

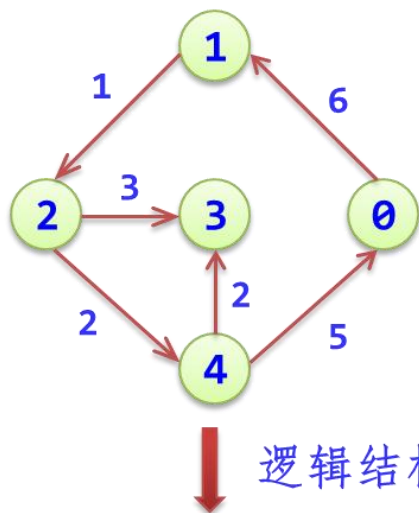
```
void DestroyAdj(AdjGraph *&G)           //销毁邻接表
{   int i; ArcNode *pre, *p;
    for (i=0;i<G->n;i++)
    {   pre=G->adjlist[i].firstarc;       //扫描所有的单链表
        if (pre!=NULL)                  //p指向第i个单链表的首结点
        {   p=pre->nextarc;
            while (p!=NULL)
            {   free(pre);
                pre=p; p=p->nextarc;
            }
            free(pre);                   //释放第i个单链表的所有边结点
        }
    }
    free(G);                             //释放头结点数组
}
```



操作：将图的两存储结构的声明代码存放在graph.h头文件中。将图的基本运算算法代码存放在graph.cpp文件中。

编写程序创建以下图的邻接表 G ，输出邻接表并销毁





逻辑结构信息

```
int n=5,e=6;
int A[][MAXV]=
    {{0,6,INF,INF,INF},
     {INF,0,1,INF,INF},
     {INF,INF,0,3,2},
     {INF,INF,INF,0,INF},
     {5,INF,INF,2,0}};
```

```
#include "graph.cpp" //包含图基本运算函数
void main()
{ AdjGraph *G;
  int n=5,e=6;
  int A[][MAXV]=
      {{0,6,INF,INF,INF},
       {INF,0,1,INF,INF},
       {INF,INF,0,3,2},
       {INF,INF,INF,0,INF},
       {5,INF,INF,2,0}};
  CreateAdj(G,A,n,e); //创建图的邻接表
  printf("G的邻接表:\n"); DispAdj(G);
  printf("销毁图G的邻接表\n");
  DestroyAdj(G);
}
```



```
F:\清华大学数据结...
G的邻接表:
0:  1[6]→^
1:  2[1]→^
2:  3[3]→ 4[2]→^
3:  ^
4:  0[5]→ 3[2]→^
销毁图G的邻接表
Press any key to continue
```

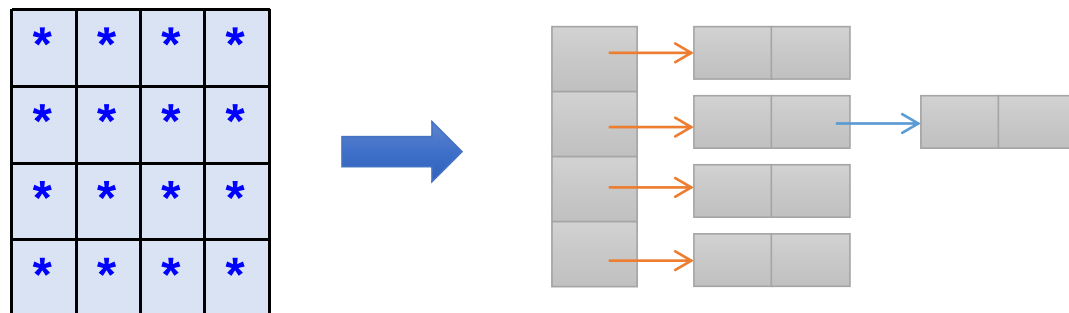


【例8-2】 对于具有 n 个顶点的图 G :

- (1) 设计一个将邻接矩阵转换为邻接表的算法;
- (2) 设计一个将邻接表转换为邻接矩阵的算法;
- (3) 分析上述两个算法的时间复杂度。

解

(1) 算法(1)



在图 G 的邻接矩阵 g 中查找值不为0、不为 ∞ 的元素，找到这样的元素如 $g.edges[i][j]$ ，表示存在一条边，创建一个adjvex域为 j 的边结点，采用头插法将它插入到第 i 个单链表中。

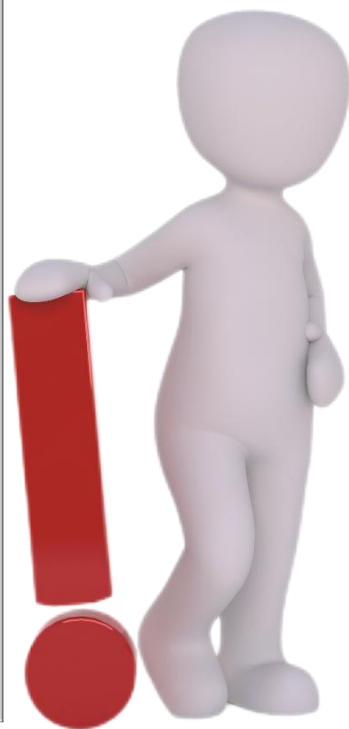



```

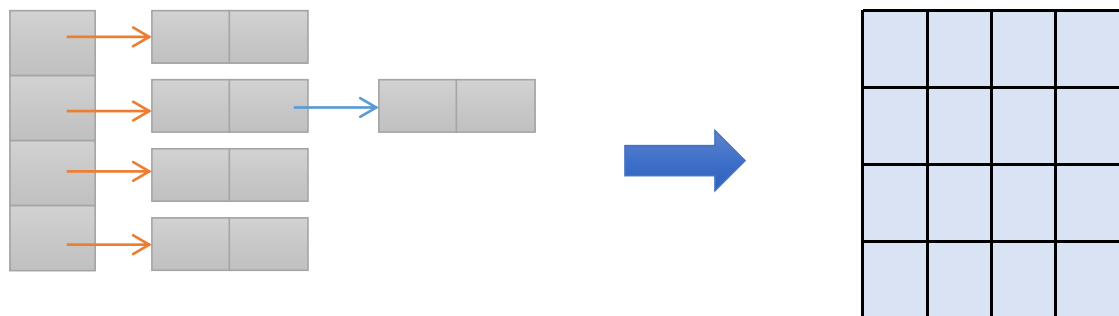
void MatToList(MatGraph g, AdjGraph *&G)
//将邻接矩阵g转换成邻接表G
{  int i, j;
    ArcNode *p;
    G=(AdjGraph *)malloc(sizeof(AdjGraph));
    for (i=0;i<g.n;i++)          //将邻接表中所有头结点的指针域置初值
        G->adjlist[i].firstarc=NULL;
    for (i=0;i<g.n;i++)          //检查邻接矩阵中每个元素
        for (j=g.n-1;j>=0;j--)
            if (g.edges[i][j]!=0 && g.edges[i][j]!=INF)    //存在一条边
            {  p=(ArcNode *)malloc(sizeof(ArcNode));        //建一个边结点p
                p->adjvex=j; p->weight= g.edges[i][j];
                p->nextarc=G->adjlist[i].firstarc;  //采用头插法插入结点p
                G->adjlist[i].firstarc=p;
            }
    G->n=g.n; G->e=g.e;
}

```

时间复杂度为 $O(n^2)$



算法(2)

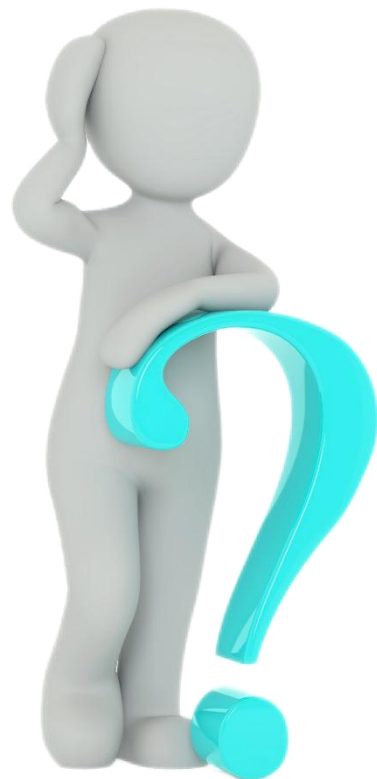


初始时将邻接矩阵 g 中所有边对应的元素值设置为0，扫描邻接表 G 的所有单链表：

通过第 i 个单链表查找顶点 i 的相邻结点 p ，将邻接矩阵 g 的元素 $g.edges[i][p \rightarrow adjvex]$ 修改为 $p \rightarrow weight$ 。




```
void ListToMat(AdjGraph *G, MatGraph &g)
//将邻接表G转换成邻接矩阵g
{   int i;
    ArcNode *p;
    for (i=0;i<G->n;i++)           //扫描所有的单链表
    {   p=G->adjlist[i].firstarc;   //p指向第i个单链表的首结点
        while (p!=NULL)           //扫描第i个单链表
        {   g.edges[i][p->adjvex]=p->weight;
            p=p->nextarc;
        }
    }
    g.n=G->n; g.e=G->e;
}
```



```

void ListToMat(AdjGraph *G, MatGraph &g)
//将邻接表G转换成邻接矩阵g
{ int i;
  ArcNode *p;
  for (i=0;i<G->n;i++)          //扫描所有的单链表
  { p=G->adjlist[i].firstarc;    //p指向第i个单链表的首结点
    while (p!=NULL)             //扫描第i个单链表
    { g.edges[i][p->adjvex]=p->weight;
      p=p->nextarc;
    }
  }
  g.n=G->n; g.e=G->e;
}

```

算法(2)中虽有两重循环，但只对邻接表的所有头结点和边结点访问一次。

对于无向图，访问次数为 $n+2e$ ，对于有向图，访问次数为 $n+e$ ，所以算法(2)时间复杂度为 $O(n+e)$ 。其中 e 为图的边数。



8.2.4 图的其他存储方法

1. 十字链表

十字链表是**有向图**的另外一种存储结构，它是邻接表和逆邻接表的结合。

2. 邻接多重表

- 邻接多重表是**无向图**的另外一种存储结构，与十字链表类似。
- 与无向图的邻接表相比，对图的边的操作（标记或删除）更便利。





数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

➤ 第二章

- 线性表的逻辑结构

有穷性、一致性、序列性、一对一关系

- 线性表的存储结构

顺序表：基本运算实现与应用

增、删、改、查

链表：基本运算实现与应用

双链表、循环链表

顺序表与链表的优缺点比较

- 有序表

顺序表：
移动元素
修改数量

链表：
先关联后继结点
头插法、尾插法

一、顺序表和单链表

1、线性表两类存储结构的比较

① 顺序表

优点

- 存储密度大：无须为表示线性表中元素之间的逻辑关系而增加额外的存储空间。
- 具有随机存取特性。

缺点

- 插入和删除操作需要移动大量元素。
- 初始空间大小分配难以掌握。

一、顺序表和单链表

1、线性表两类存储结构的比较

② 链表

优点

- 由于采用结点的动态分配方式，具有良好的适应性。
- 插入和删除操作只需修改相关指针域，不需要移动元素。

缺点

- 存储密度小：为表示线性表中元素之间的逻辑关系而需要增加额外的存储空间（指针域）。
- 不具有随机存取特性。

1. 已知一个顺序表中，第一个元素的存储地址是20，每个元素长度为5，则第8个元素的存储地址是（ ） B

A. 50 B. 55 C. 60 D. 不能确定

2. 一个长度为 n 的顺序表向第 i 个元素之前插入一个新的元素时需要向后移动_____个元素。
B

A. $n-i$ B. $n-i+1$ C. $n-i-1$ D. 不能确定

3. 以下关于有序表的叙述正确的是（ ）。 D

- A. 有序表只能采用顺序表存储
- B. 有序表中元素之间的关系是非线性关系
- C. 有序表只能采用链表存储
- D. 有序表既可以采用顺序表存储，也可以采用链表存储



课堂回顾



➤ 图的遍历

- 广度优先遍历

先进先出的特性，队列

- 深度优先遍历

递归思想，栈

示例

已知一无向图 $G = (V, E)$ ，其中
 $V = \{a, b, c, d, e\}$, $E = \{(a, b), (a, d), (a, c), (d, c), (b, e)\}$ 。现用某一种图遍历方法从顶点 a 开始遍历图，得到的序列为 $abecd$ ，请问采用的是什麼遍历方法？

示例

已知图的邻接矩阵表示如下，试写出：

- (1) 以顶点V1为出发点的深度优先遍历序列；
- (2) 以顶点V1为出发点的广度优先遍历序列。

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
V1	0	1	1	1	0	0	0	0	0	0
V2	0	0	0	1	1	0	0	0	0	0
V3	0	0	0	1	0	1	0	0	0	0
V4	0	0	0	0	0	1	1	0	1	0
V5	0	0	0	0	0	0	1	0	0	0
V6	0	0	0	0	0	0	0	1	1	0
V7	0	0	0	0	0	0	0	0	1	0
V8	0	0	0	0	0	0	0	0	0	1
V9	0	0	0	0	0	0	0	0	0	1
V10	0	0	0	0	0	0	0	0	0	0

V1 V2 V4 V6 V8 V10 V9 V7 V5 V3

V1 V2 V3 V4 V5 V6 V7 V9 V8 V10

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

▶ 8.3 图的遍历

8.4 生成树和最小生成树

8.5 最短路径

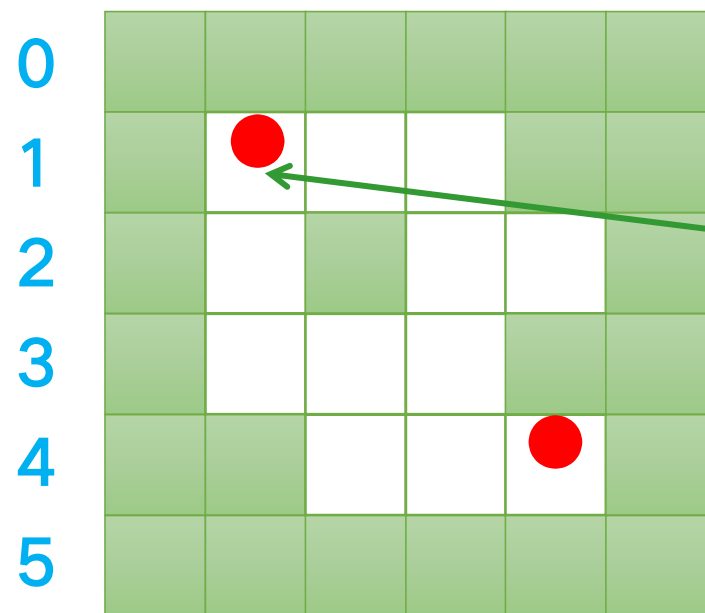
8.6 拓扑排序

8.7 AOE网与关键路径

3. 图遍历方法求解迷宫问题

创建迷宫问题的邻接表：

0 1 2 3 4 5



头结点用二维数组表示

(0,0)

Λ

.....

(1,1)

(1,2)

(2,1)

Λ

(1,2)

(1,1)

(1,3)

Λ

(1,3)

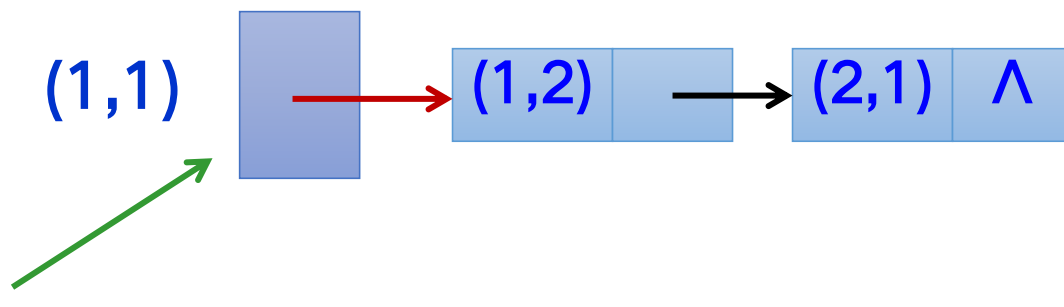
(1,2)

Λ

.....



邻接表设计:



```
typedef struct Vnode
{
    ArcNode *firstarc; //指向
    第一个可走的方块
} VNode;
```

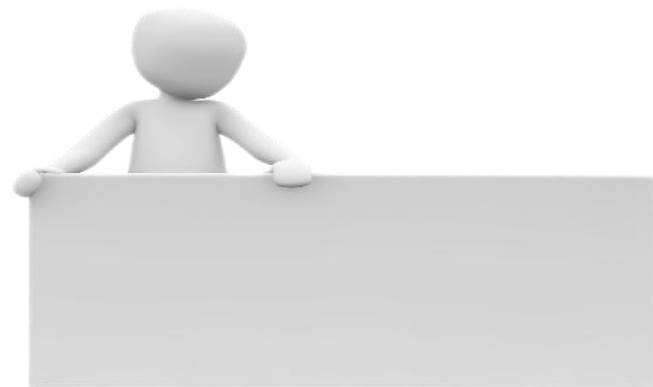
```
typedef struct ANode
{ int i, j; //行号 列号
    struct ANode *nextarc; //指
    向下一个节点
} ArcNode;
```

```
typedef struct
{
    VNode adjlist[M+2][N+2]; //头节点数组
} AdjGraph; //迷宫图的邻接表类型
```

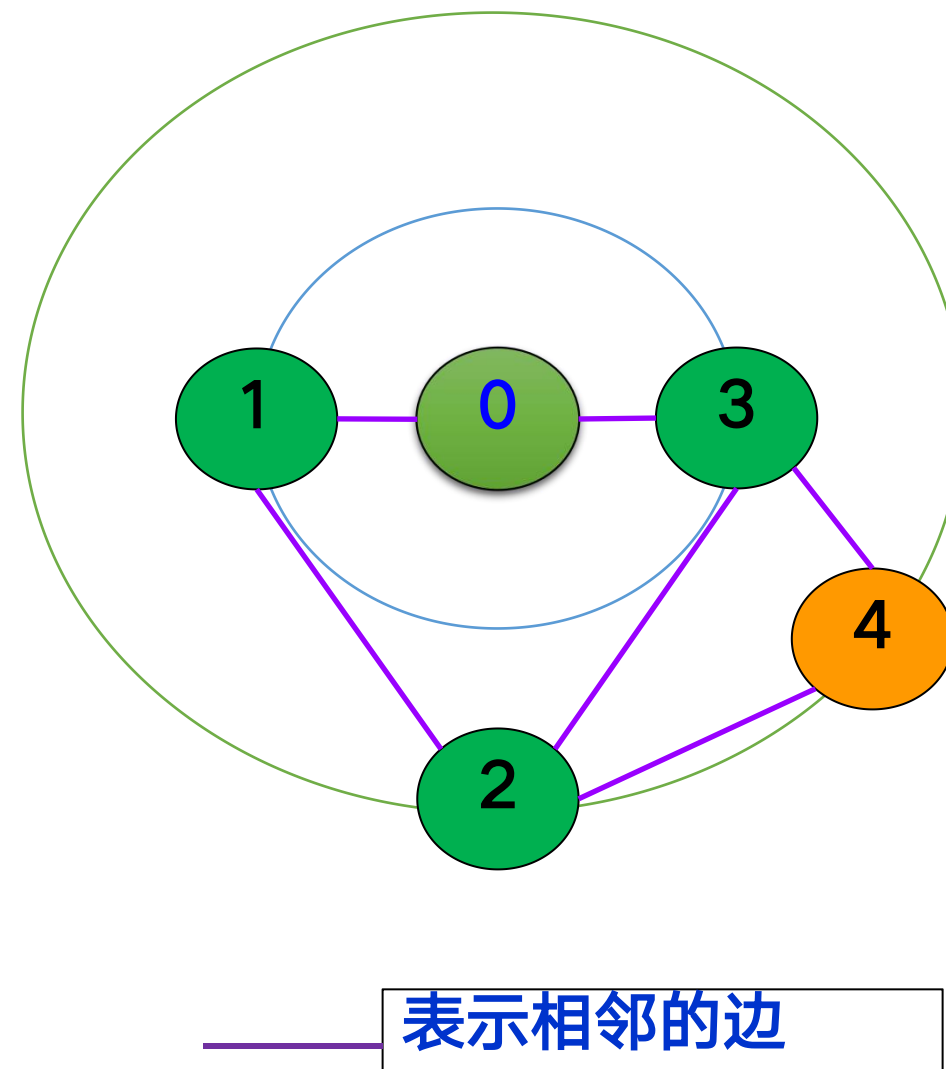
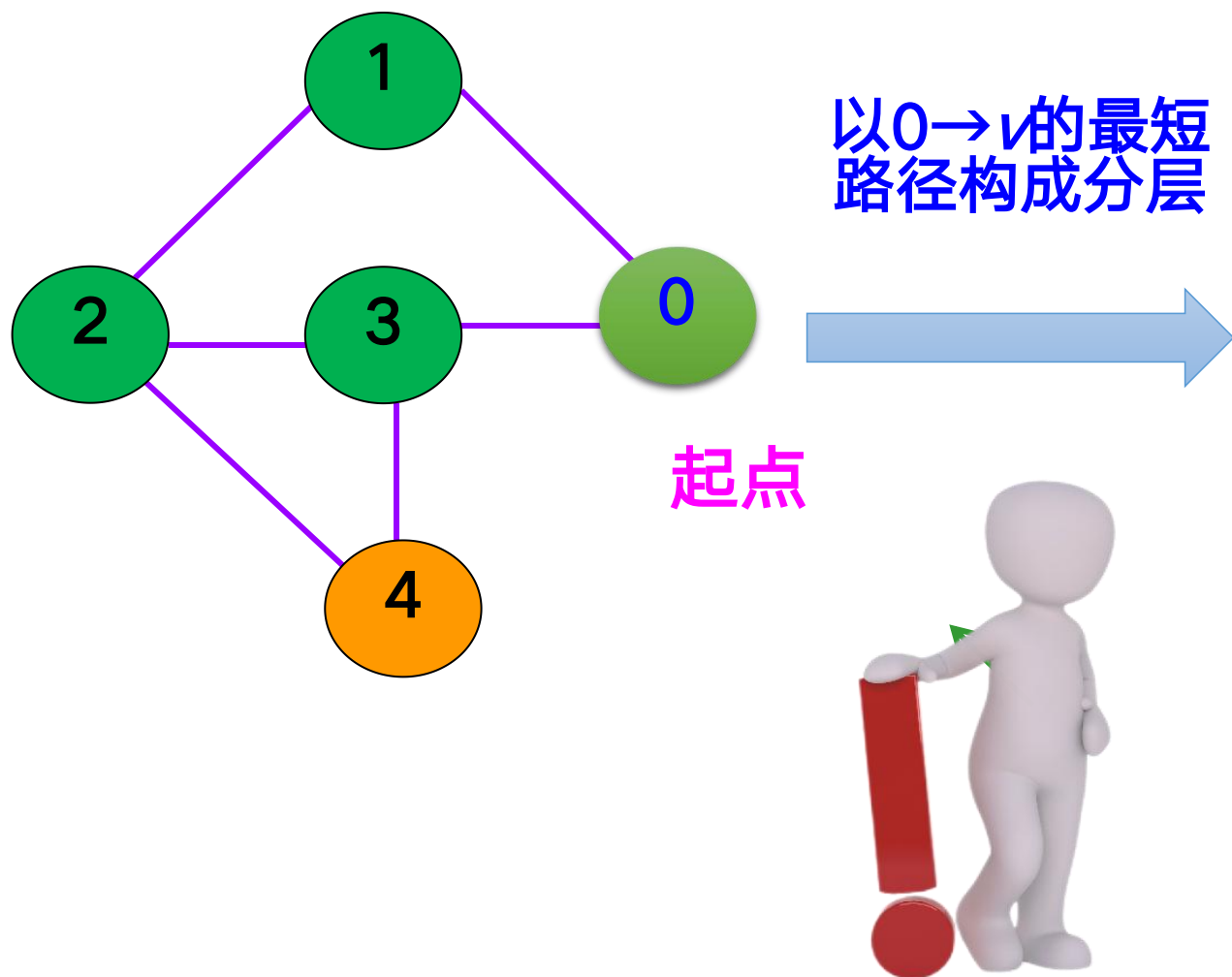
算法设计

- 采用DFS或者BFS算法
- 入口作为初始顶点
- 结束条件为找到出口
- visited改为二维数组

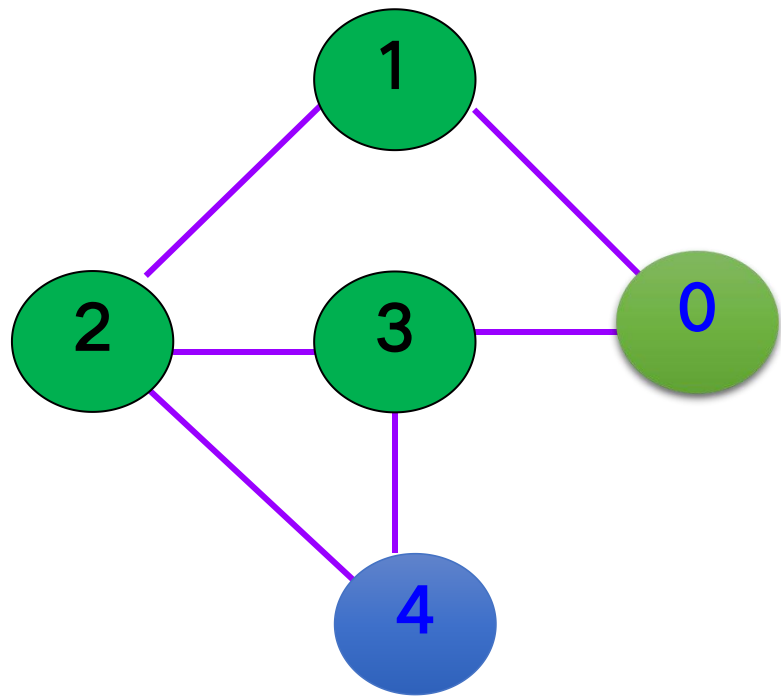
具体算法请你实现！ ✂



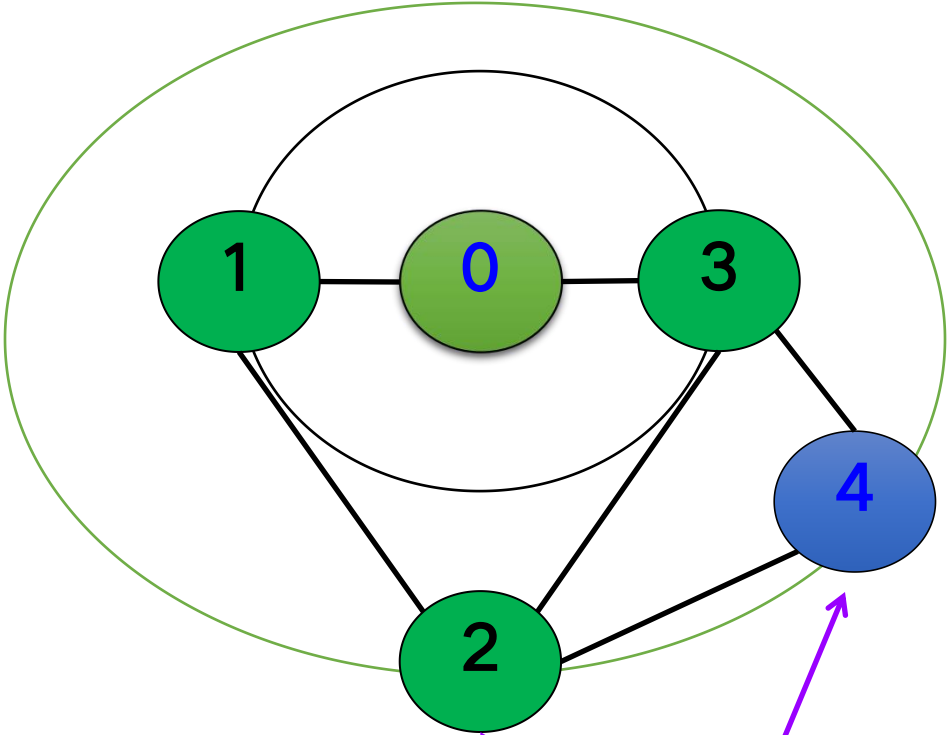
4.DFS和BFS求解路径上的差别



深度优先遍历可能找到0⇒4的一条路径：



路径：0 → 1 → 2 → 4

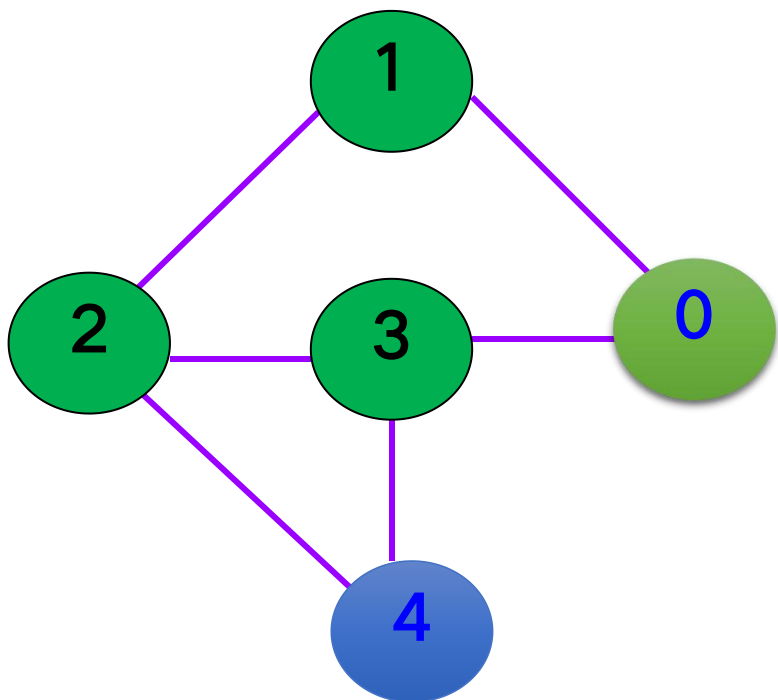


路径上的顶点可能在同一层

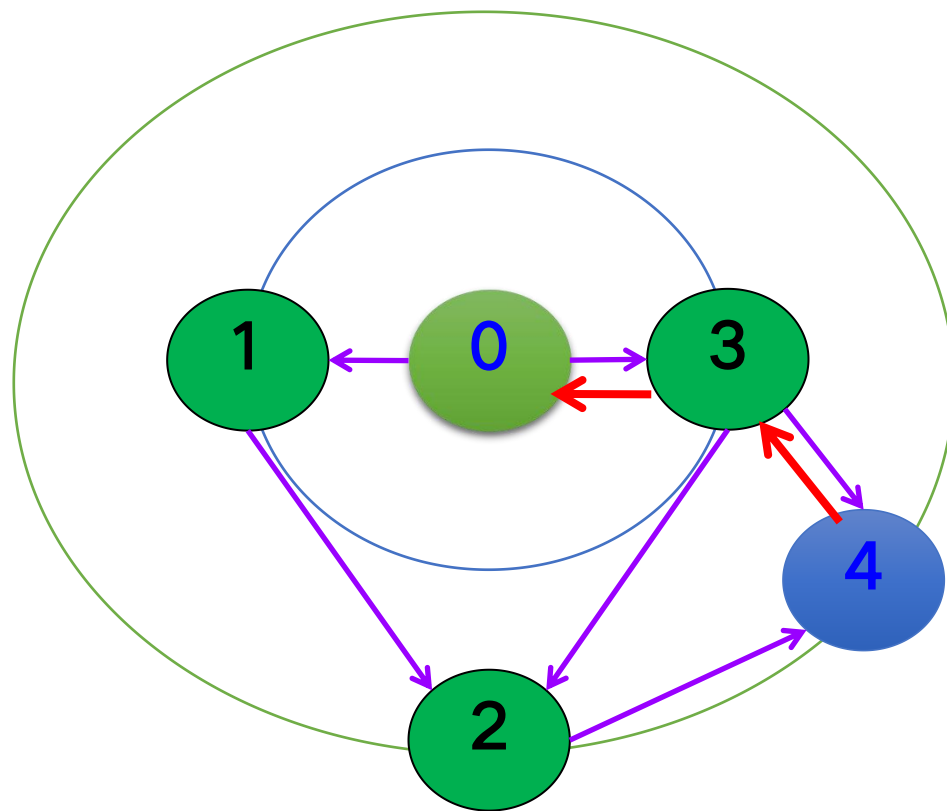


不一定是最短路径

广度优先遍历找到 $0 \Rightarrow 4$ 的路径同一层只能有一个顶点。



逆路径: $4 \rightarrow 3 \rightarrow 0$



路径: 每一层只有一个顶点

↓
一定是最短路径

结论

以路径上经过的边数来衡量路径长度（若不带权图）

- 广度优先遍历找到的路径一定是最短路径，而深度优先遍历则不一定。
- 深度优先遍历能找所有路径，而广度优先遍历难以实现。

数据结构算法的多维性

同一问题的多种解法。

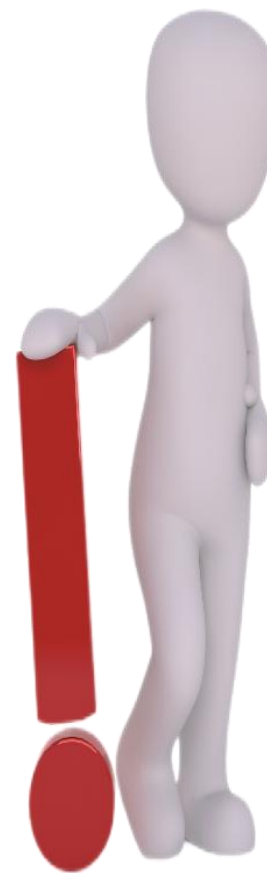
用栈方法求解

用队列方法求解

迷宫问题

用图搜索方法求解

用递归方法求解



提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

▶ 8.4 生成树和最小生成树

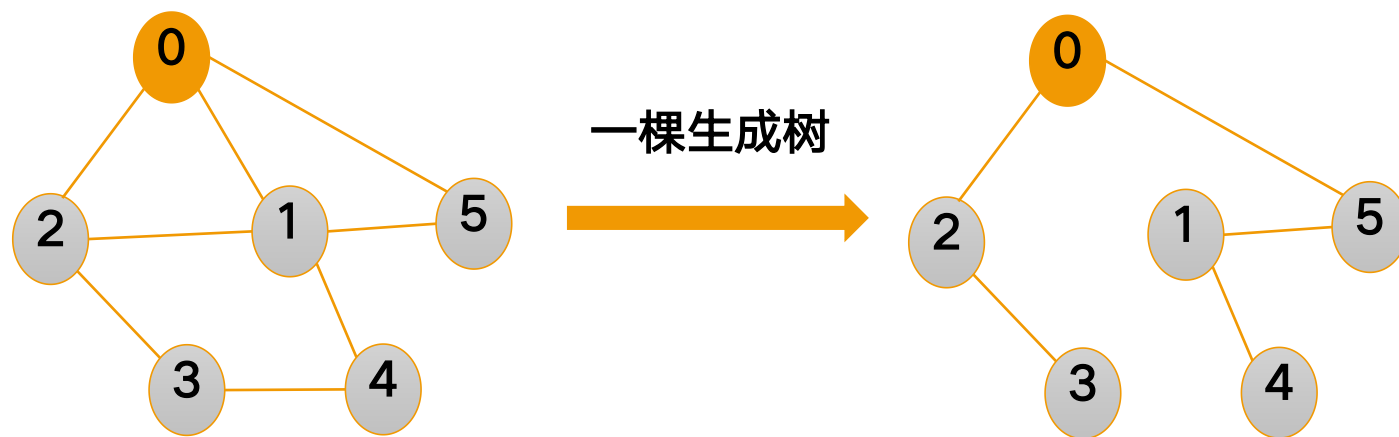
8.5 最短路径

8.6 拓扑排序

8.7 AOE网与关键路径

8.4.1 生成树的概念

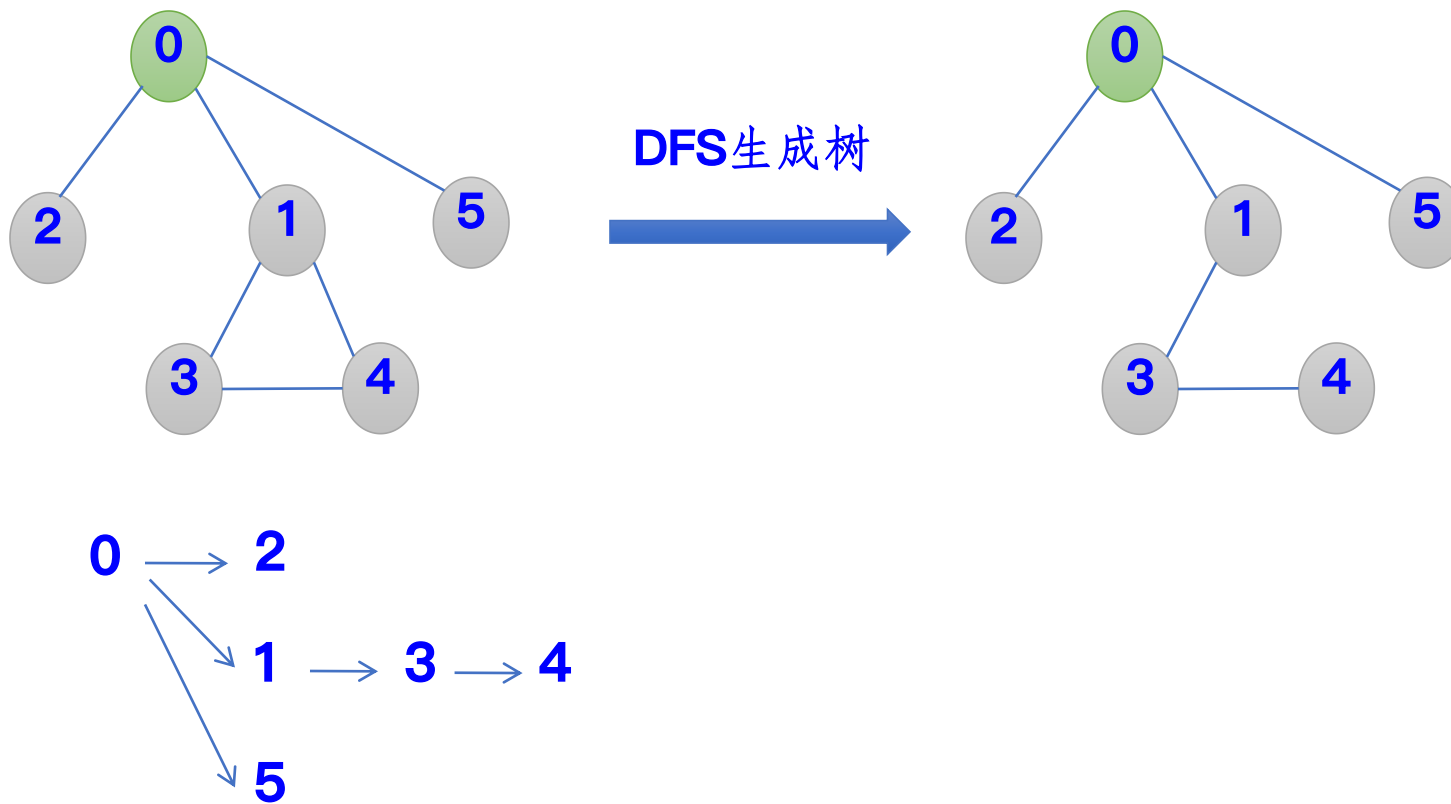
- 一个**连通图**的生成树是一个**极小连通子图**，它含有图中全部 n 个顶点和构成一棵树的 $(n-1)$ 条边。



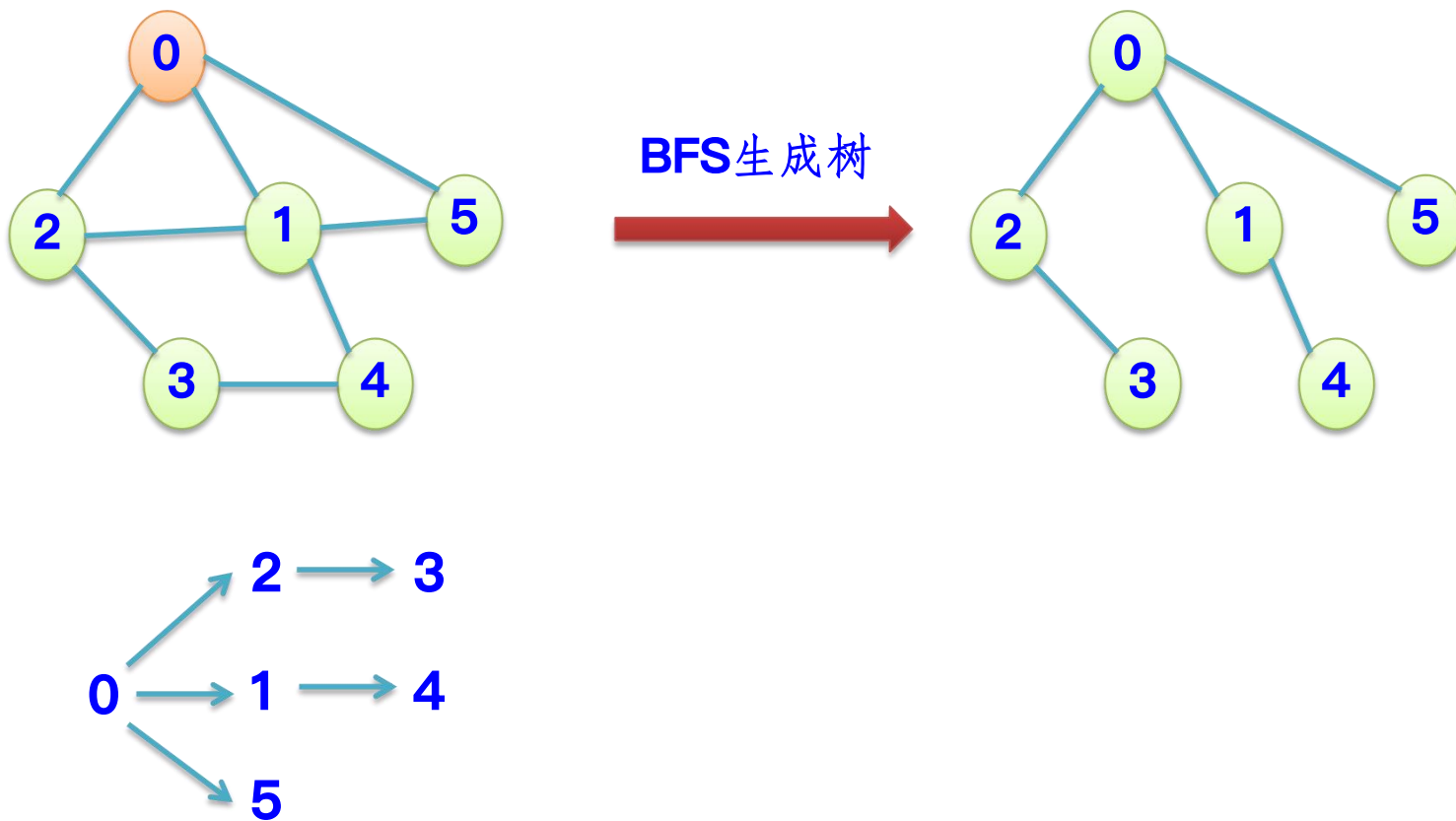
命题：如果在一棵生成树上添加一条边，必定构成一个环。

可以通过遍历方法产生生成树:

● 由深度优先遍历得到的生成树称为**深度优先生成树**。



● 由广度优先遍历得到的生成树称为**广度优先生成树**。



一个连通图的生成树不一定是唯一的！

8.4.2 非连通图和生成树

对于连通图：仅需调用遍历过程（DFS或BFS）一次，从图中任一顶点出发，便可以遍历图中的各个顶点，产生相应的生成树。



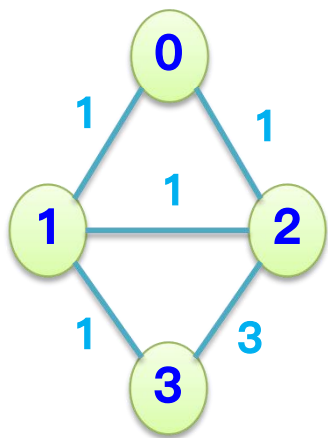
对于非连通图：需多次调用遍历过程。每个连通分量中的顶点集和遍历时走过的边一起构成一棵生成树。所有连通分量的生成树组成非连通图的生成森林。



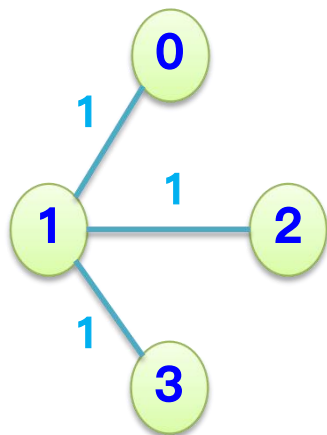
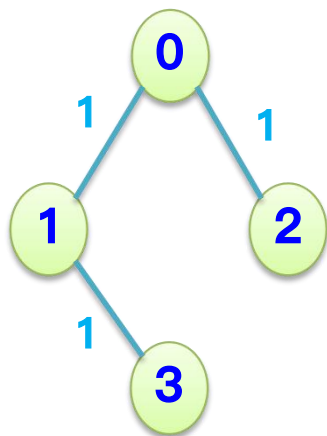
重点：求带权连通图的最小生成树

最小生成树的概念

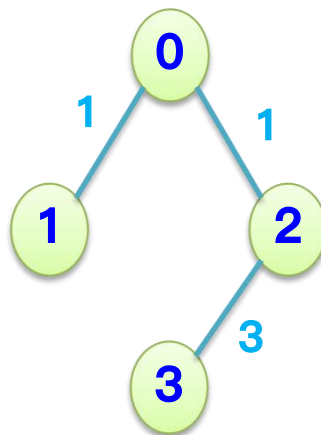
- 对于带权连通图 G （每条边上的权均为大于零的实数），可能有多棵不同生成树。
- 每棵生成树的所有边的权值之和可能不同。
- 其中权值之和最小的生成树称为图的最小生成树。



图G



都是图G的**最小生成树**



不是图G的最小生成树

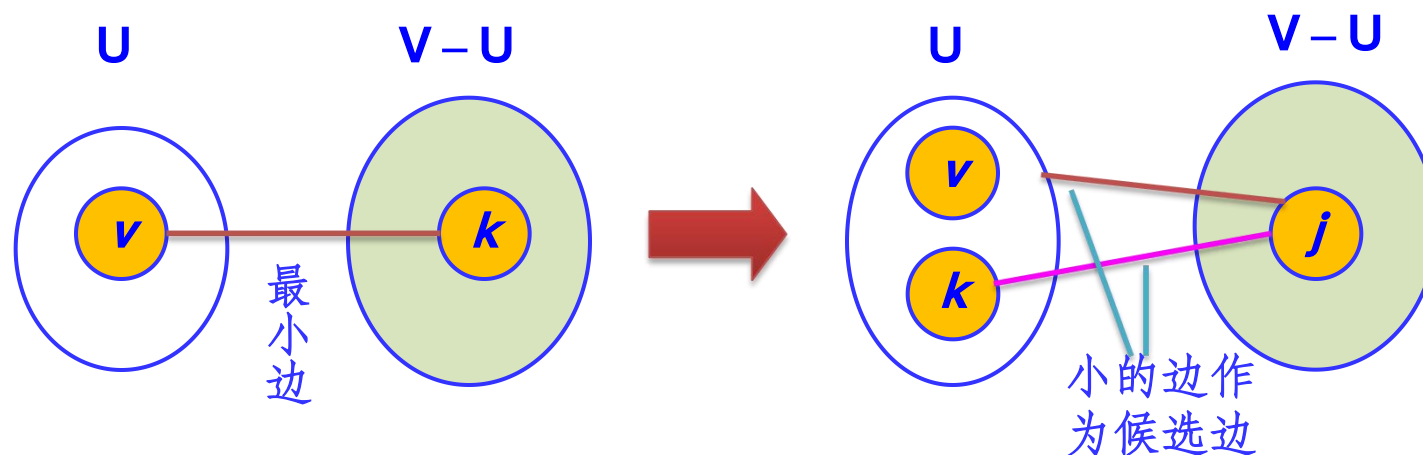
最小生成树不为1，但是最小生成树之和相同

8.4.3 普里姆算法

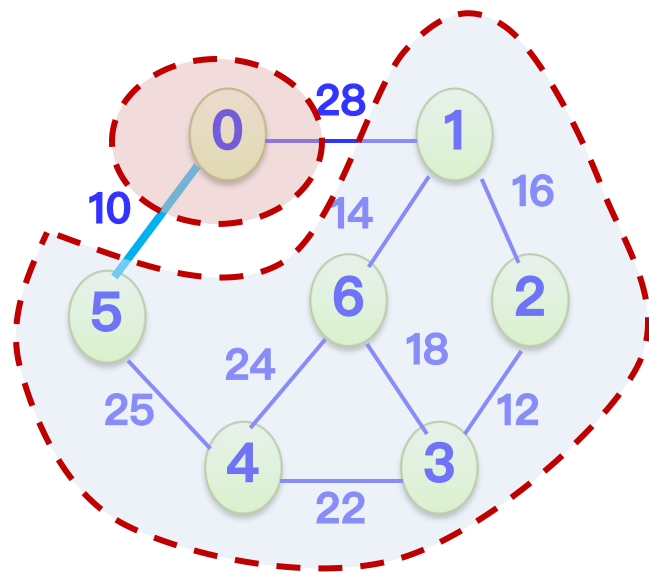
普里姆 (Prim) 算法是一种构造性算法，用于构造最小生成树。
过程如下：



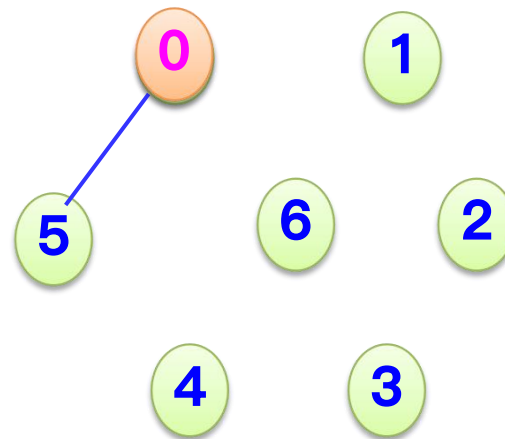
- (1) 初始化 $U=\{v\}$ 。 v 到其他顶点的所有边为候选边；
- (2) 重复以下步骤 $n-1$ 次，使得其他 $n-1$ 个顶点被加入到 U 中：
 - ① 从候选边中挑选权值最小的边输出，设该边在 $V-U$ 中的顶点是 k ，将 k 加入 U 中；
 - ② 考察当前 $V-U$ 中的所有顶点 j ，修改候选边：若 (j, k) 的权值小于原来和顶点 k 关联的候选边，则用 (k, j) 取代后者作为候选边。



Prim算法示例演示 (起点0)



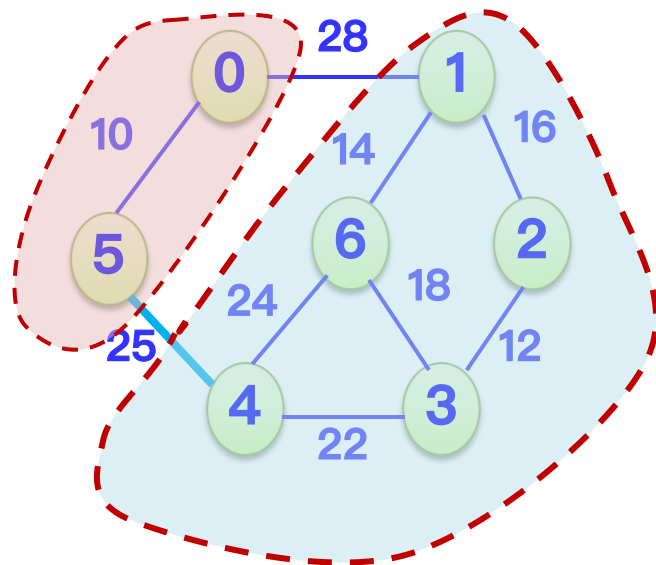
图G



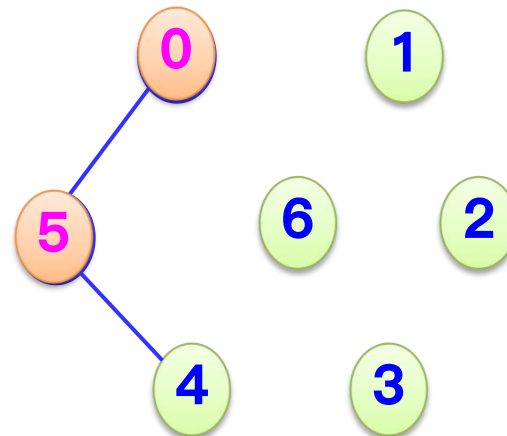
$U=\{0\}$

普里姆算法求解最小生成树的过程

Prim算法示例演示 (起点0)



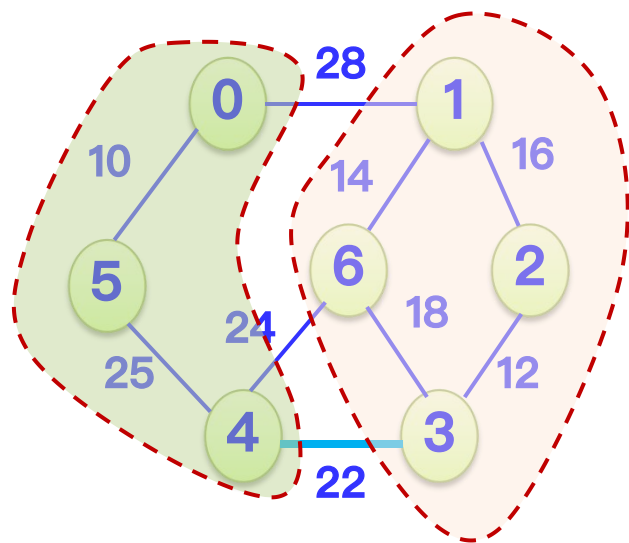
图G



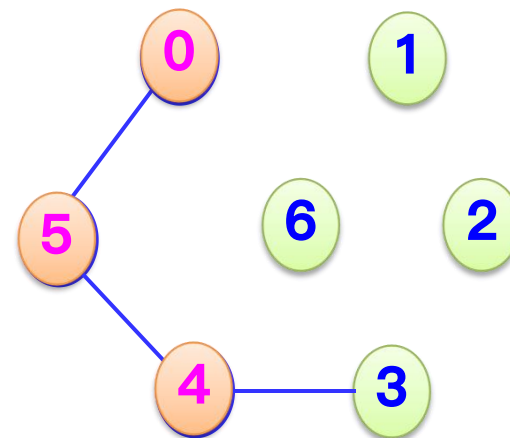
$U=\{0, 5\}$

普里姆算法求解最小生成树的过程

Prim算法示例演示 (起点0)



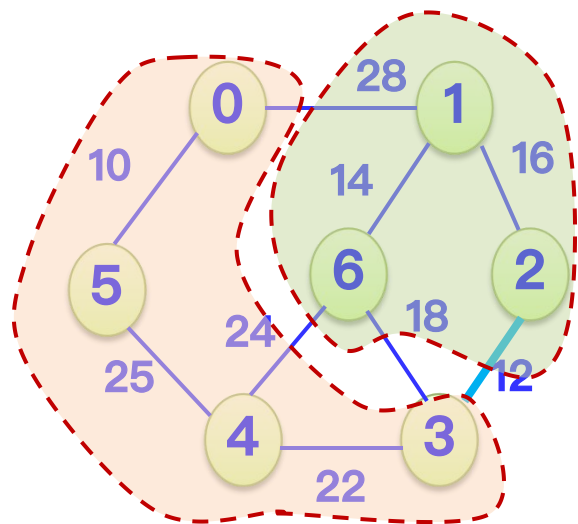
图G



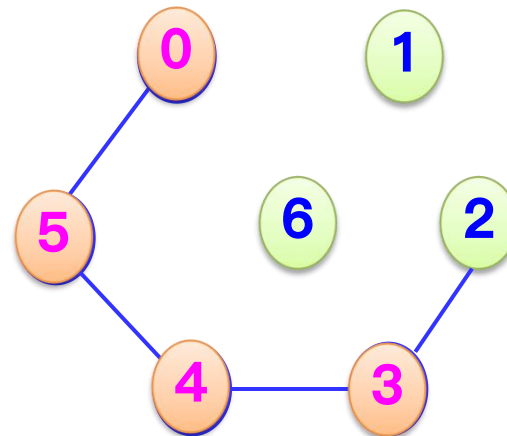
$U=\{0, 5, 4\}$

普里姆算法求解最小生成树的过程

Prim算法示例演示 (起点0)



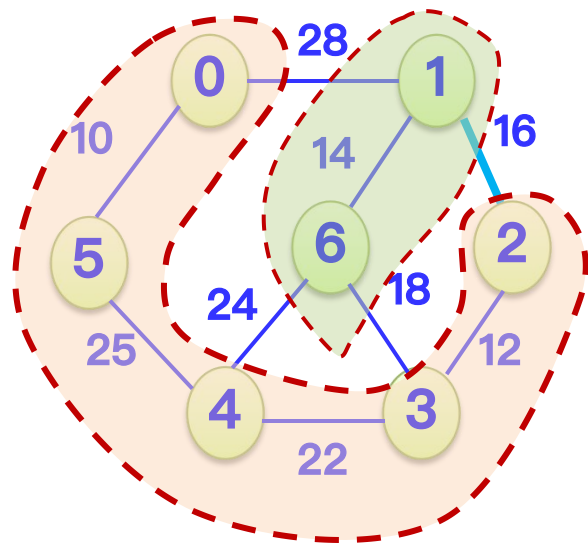
图G



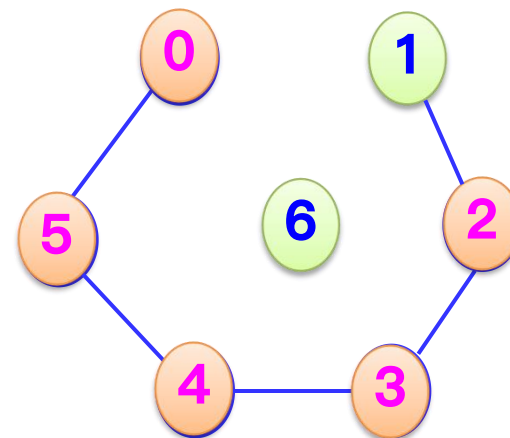
$U=\{0, 5, 4, 3\}$

普里姆算法求解最小生成树的过程

Prim算法示例演示 (起点0)



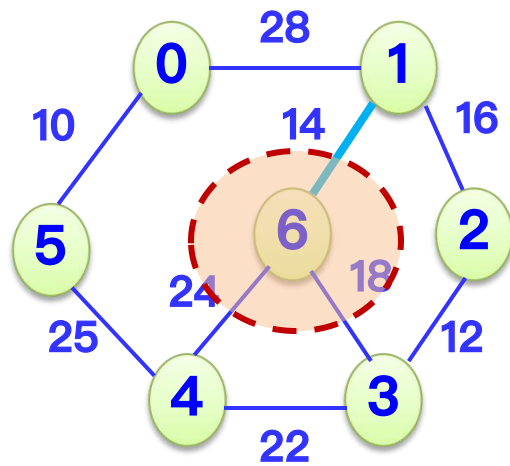
图G



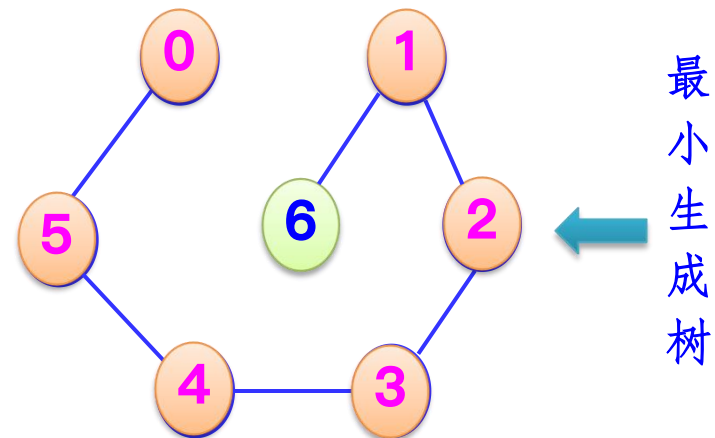
$U=\{0, 5, 4, 3, 2\}$

普里姆算法求解最小生成树的过程

Prim算法示例演示 (起点0)



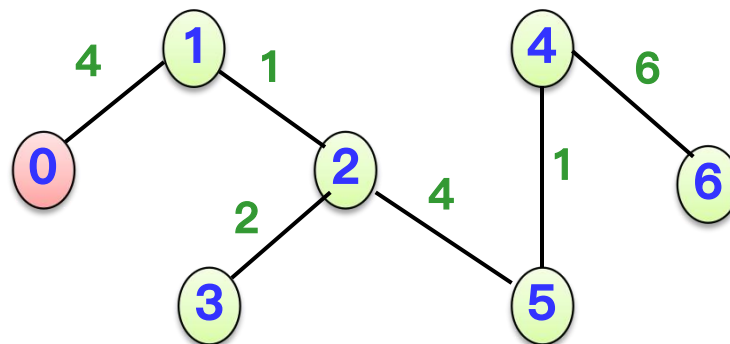
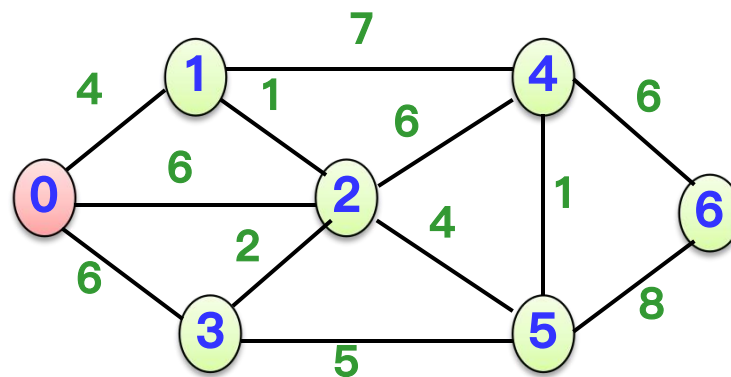
图G



$U = \{0, 5, 4, 3, 2, 1, 6\}$

普里姆算法求解最小生成树的过程

用prim算法求解最小生成树，起点0



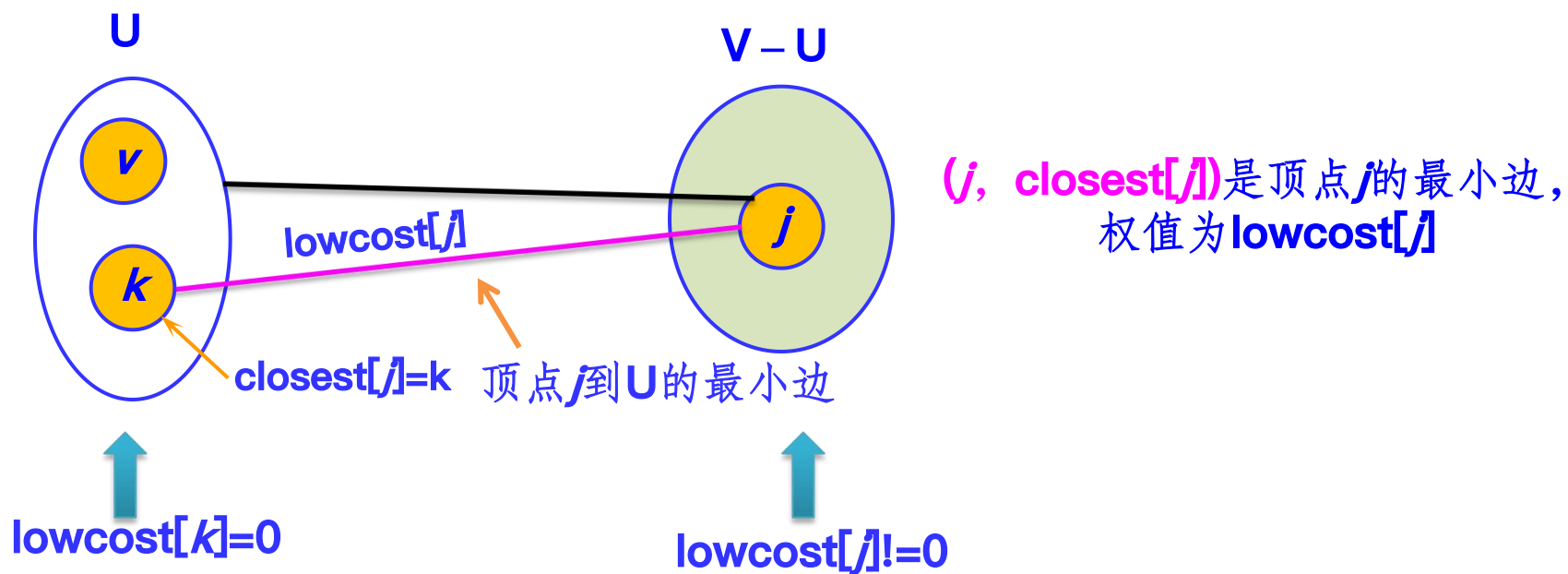
$$4 + 1 + 2 + 4 + 1 + 6 = 18$$

算法设计（解决4个问题）：

- 如何求 U 、 $V-U$ 两个顶点集之间的最小边？（只求一条）

只考虑 $V-U$ 中顶点 j 到 U 顶点集的最小边（无向图），比较来找最小边

- 如何存储顶点 j 到 U 顶点集的最小边？



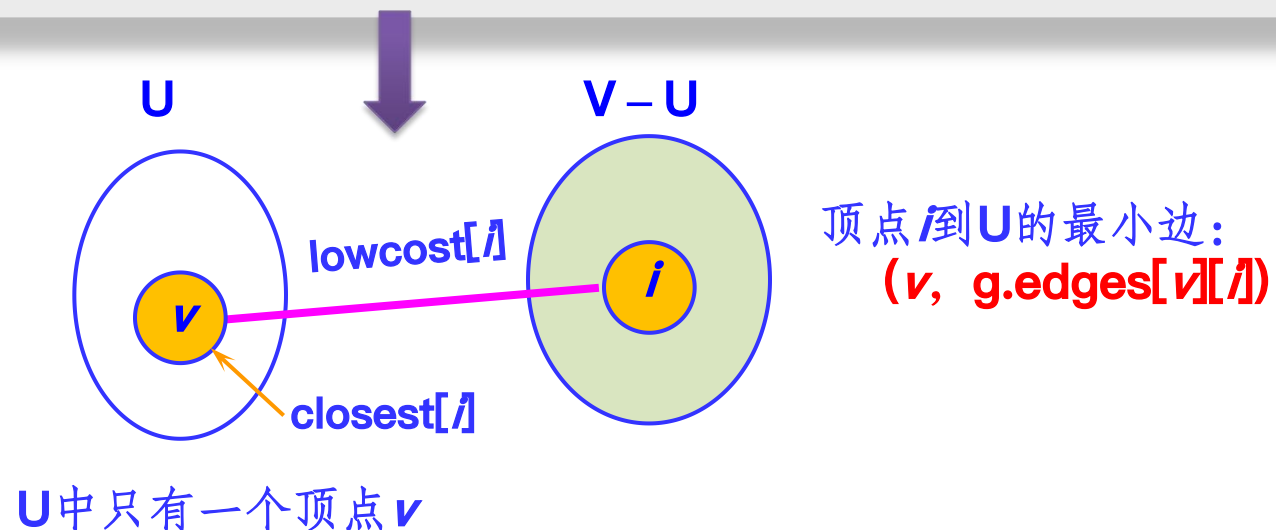
- 一个顶点属于哪个集合？

- 图采用哪种存储结构更合适？

邻接矩阵

普里姆 (Prim) 算法如下:

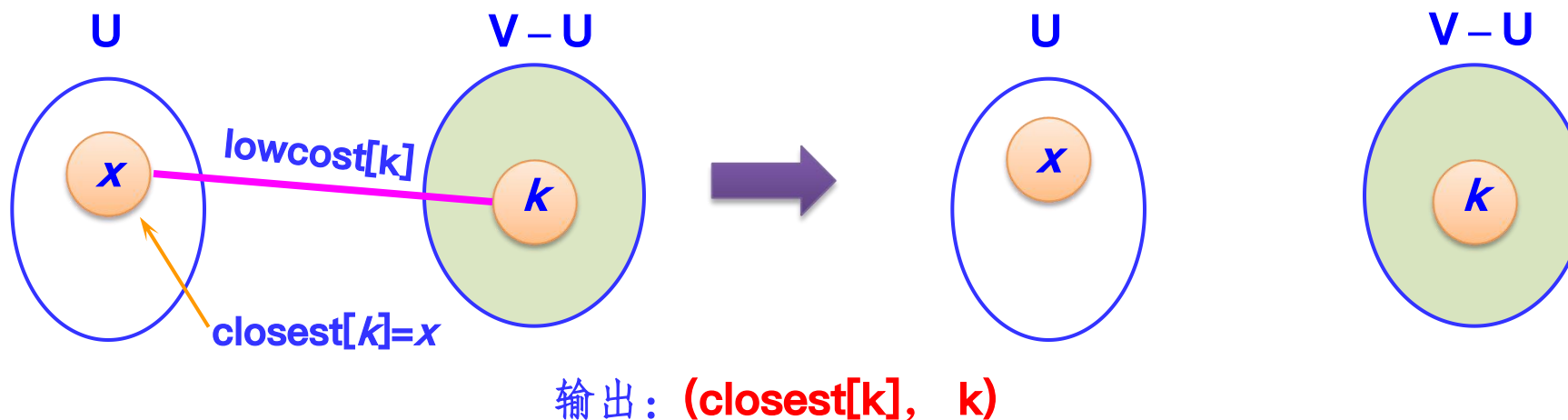
```
#define INF 32767 //INF表示 $\infty$ 
void Prim(MatGraph g, int v) //g表示带权的连通图的邻接矩阵, 起点是v
{ int lowcost[MAXV]; //开辟两个新的临时数组, 来存放权值和顶点
  int min;
  int closest[MAXV], i, j, k;
  for (i=0; i<g.n; i++) //给lowcost[]和closest[]置初值
  { lowcost[i]=g.edges[v][i]; // 顶点v到i之间的距离, 没有边为INF
    closest[i]=v; //起点为v
  }
}
```

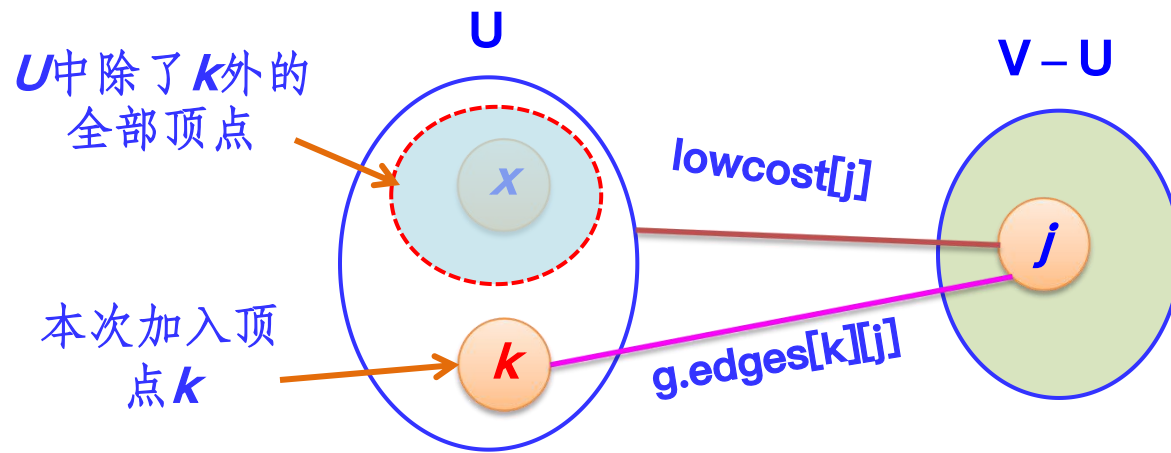


```

for (i=1;i<g.n;i++)           //扫描n-1个顶点，输出(n-1)条边
{ min=INF;
  for (j=0;j<g.n;j++)        //在(V-U)中找出离U最近的顶点k
    if (lowcost[j]!=0 && lowcost[j]<min) //保证j在V-U集合中，同时找最小候选边
    { min=lowcost[j];
      k=j;                     //k记录最近顶点编号
    } //循环完之后就能找到两个集合之间的最小边
  printf(" 边(%d, %d)权为:%d\n", closest[k], k, min); //输出边和权值
  lowcost[k]=0;             //标记k已经加入U

```





```
for (j=0;j<g.n;j++) //修改数组lowcost和closest
{
    if (lowcost[j]!=0 && g.edges[k][j]<lowcost[j])
    {
        lowcost[j]=g.edges[k][j];
        closest[j]=k;
    }
}
```

修改U和
V-U之间的
候选边，
即调整

仅仅考虑V-U中的顶点

普里姆算法思路

局部最优 + 调整 = 全局最优

↑
贪心算法思想

↑
最优结果

普里姆算法分析

Prim()算法中有两重**for**循环，所以时间复杂度为 $O(n^2)$ 。

8.4.4 克鲁斯卡尔算法

克鲁斯卡尔 (Kruskal) 算法也是一种求带权无向图的最小生成树的构造性算法。



按权值的递增次序选择合适的边来构造最小生成树的方法。

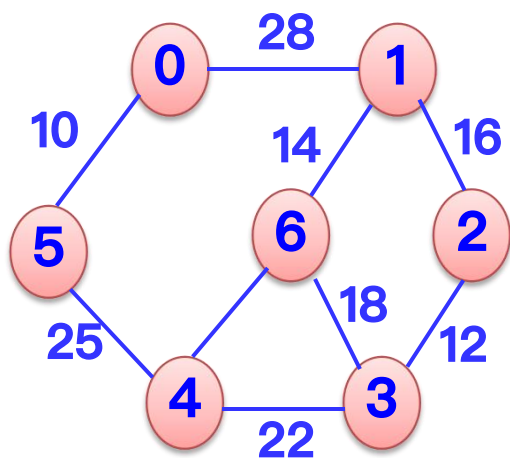
克鲁斯卡尔 (Kruskal) 算法过程: 构造最小生成树 (U,TE)

(1) 置**U**的初值等于**V** (即包含有**G**中的全部顶点), **TE**的初值为空集 (即图**T**中每一个顶点都构成一个连通分量)。

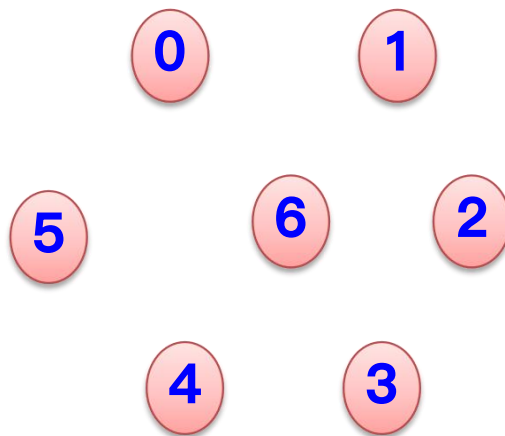
(2) 将图**G**中的边按权值**从小到大**的顺序依次选取:

① 若选取的边未使生成树**T**形成回路, 则加入**TE**;

② 否则舍弃, 直到**TE**中包含 **$(n-1)$** 条边为止。



有**条件**地加入
 $(n-1)$ 条边



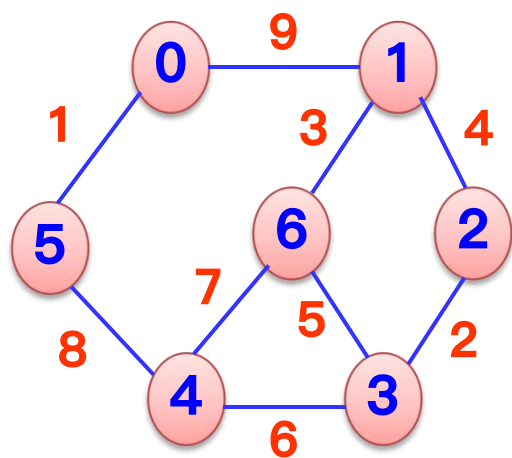
TE={}

Kruskal算法示例的演示



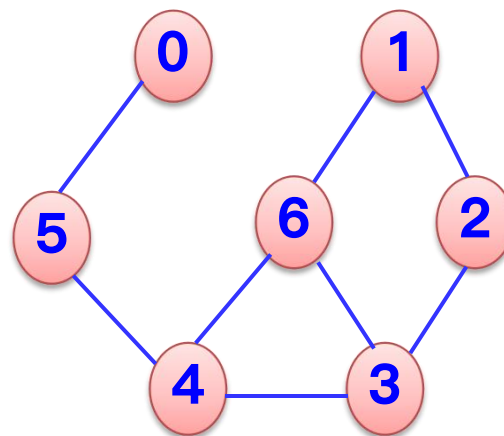
克鲁斯卡尔算法求解最小生成树的过程

Kruskal算法示例的演示



操作
取8号边

选取了 $n-1$ 条边



最小生成树

克鲁斯卡尔算法求解最小生成树的过程

算法设计（解决3个问题）：

● 图采用哪种存储结构更合适？

邻接矩阵

● 边的排序问题？

这里采用直接插入排序算法

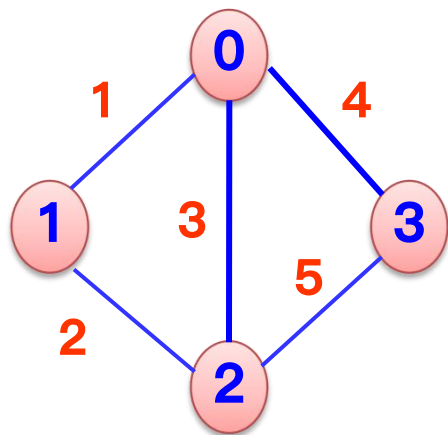
● 如何解决加入一条边后是否出现回路？

采用连通分量编号或顶点集合编号

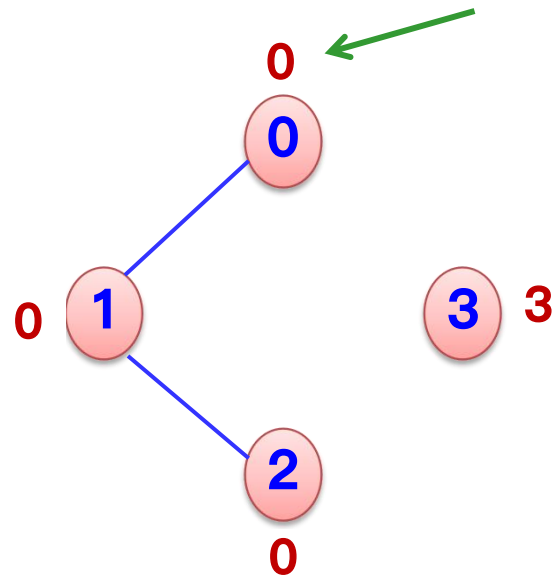
Kruskal算法如何解决出现回路的问题演示

初始连通分量编号为顶点编号

$vset[0]$: 连通分量编号



操作
取3号边



3号边的两个顶点的 $vset$ 值相同，不能添加！

在实现克鲁斯卡尔算法Kruskal()时，用数组E存放图G中的所有边，其类型如下：

```
typedef struct
{ int u;          //边的起始顶点
  int v;          //边的终止顶点
  int w;          //边的权值
} Edge;
```

```
Edge E[MAXV];
```


克鲁斯卡尔 (Kruskal) 算法如下:

```
void Kruskal(MatGraph g) //邻接矩阵g
{ int i, j, u1, v1, sn1, sn2, k;
  int vset[MAXV];
  Edge E[MaxSize];           //存放所有边
  k=0;                        //E数组的下标从0开始计
  for (i=0;i<g.n;i++)        //由g产生的边集E
    for (j=0;j<g.n;j++)
      if (g.edges[i][j]!=0 && g.edges[i][j]!=INF)
        { E[k].u=i; E[k].v=j; E[k].w=g.edges[i][j];
          k++;
        }
  InsertSort(E, g.e);        //按权值递增排序
  for (i=0;i<g.n;i++)        //初始化连通分量编号为顶点号
    vset[i]=i;
```

```

k=1;                //k表示当前构造生成树的第几条边，从1开始
j=0;                //E中边的下标，初值为0
while (k<g.n)        //生成的边数小于n时循环，选n-1条边
{
    u1=E[j].u;v1=E[j].v;    //取一条边的头尾顶点
    sn1=vset[u1];
    sn2=vset[v1];          //分别得到两个顶点所属的集合编号
    if (sn1!=sn2)           //两顶点属于不同的集合
    { printf(" (%d, %d):%d\n", u1, v1, E[j].w);
      k++;                  //生成边数增1
      for (i=0;i<g.n;i++) //两个集合统一编号
          if (vset[i]==sn2) //集合编号为sn2的改为sn1
              vset[i]=sn1;
    }
    j++;                  //扫描下一条边
}
}

```

上述算法不是最优的



改进：堆排序

Kruskal算法的时间复杂度为 $O(e \log_2 e)$

```
void Kruskal(MatGraph g)           //改进的Kruskal算法
{ int i,j,k,u1,v1,sn1,sn2;
  UFSTree t[MaxSize];
  Edge E[MaxSize];
  k=1;                             //e数组的下标从1开始计
  for (i=0;i<g.n;i++)              //由g产生的边集E
    for (j=0;j<=i;j++)
      if (g.edges[i][j]!=0 && g.edges[i][j]!=INF)
      { E[k].u=i;E[k].v=j;E[k].w=g.edges[i][j];
        k++;
      }

  HeapSort(E,g.e);                 //采用堆排序对E数组按权值递增排序
```

```

MAKE_SET(t,g.n);           //初始化并查集树t
k=1;                        //k表示当前构造生成树的第几条边,初值为1
j=1;                        //E中边的下标从1开始

while (k<g.n)                //生成的边数小于n时循环
{
    u1=E[j].u;
    v1=E[j].v;                //取一条边的头尾顶点编号u1和v2
    sn1=FIND_SET(t,u1);
    sn2=FIND_SET(t,v1);      //分别得到两个顶点所属的集合编号
    if (sn1!=sn2)             //两顶点属不同的集合,是最小生成树的边
    {
        printf(" (%d,%d):%d\n",u1,v1,E[j].w);
        k++;                  //生成边数增1
        UNION(t,u1,v1);      //将u1和v1两个顶点合并
    }
    j++;                      //扫描下一条边
}
}

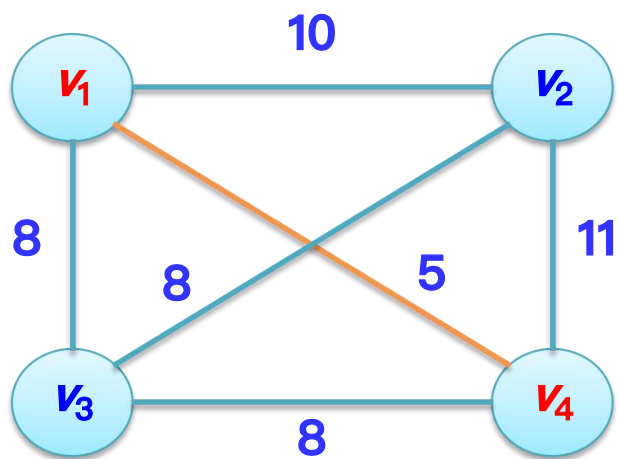
```

上述Kruskal算法的时间复杂度为 $O(e \log_2 e)$ 。

【例8-12】 求下面带权图的最小（代价）生成树时，可能是克鲁斯卡（kruskal）算法第二次选中但不是普里姆（Prim）算法（从 v_4 开始）第2次选中的边是（ ）。

A. (v_1, v_3) B. (v_1, v_4) C. (v_2, v_3) D. (v_3, v_4)

注：2015年全国考研题



kruskal:

1: (v_1, v_4) , 2: (v_1, v_3) 或 (v_3, v_4)
或 (v_2, v_3) , ...

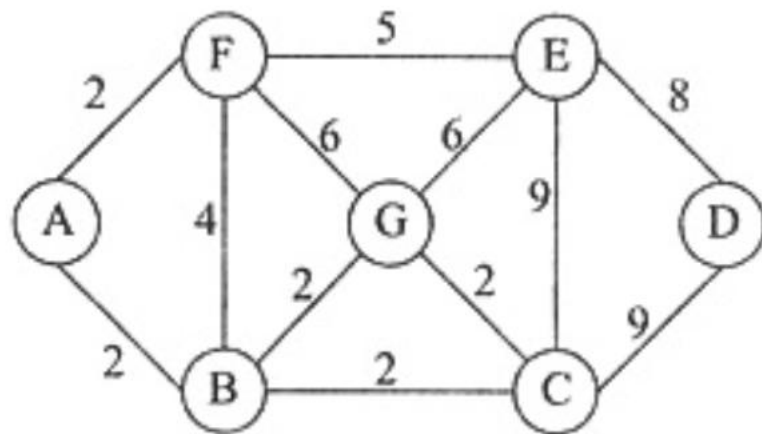
Prim（从 v_4 开始）:

1: (v_1, v_4) , 2: (v_1, v_3) 或 (v_3, v_4) ,
不可能是 (v_2, v_3)

答案为C

【习题】 如下图所示带权图G,其最小生成树各边权的总和为多少?

A.14 B.19 C.21 D.26

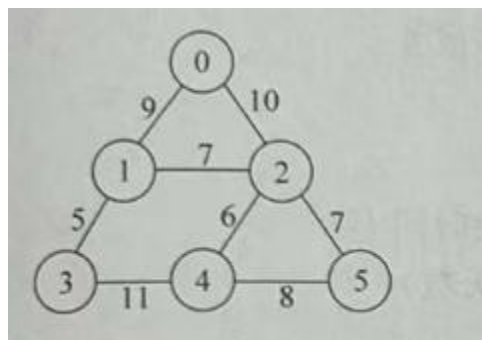


【习题】 给定如图 所示的带权无向图 G 。

(1) 画出该图的邻接表存储结构。

(2) 根据该图的邻接表存储结构，从顶点 0 出发，调用 DFS 和 BFS 算法遍历该图，给出相应的遍历序列。

(3) 给出采用 Kruskal 算法构造最小生成树的过程。





数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

➤ 第三章

- 栈：先入后出

顺序栈 四要素：

栈空条件： $S \rightarrow TOP == -1$
栈满条件： $S \rightarrow TOP == \text{MaxSize} - 1$
进栈e操作：先 $Top++$ ；后存放
退栈操作：先取值，后 $Top--$

链栈

四要素：

栈空条件： $S \rightarrow \text{Next} == \text{NULL}$
栈满条件（不考虑）
进栈e操作：放在头结点之后
退栈操作：取出首结点的值

- 共享栈
- 简单表达式求值：中缀表达式转后缀表达式；后缀表达式求值
- 队列：先入先出

➤ 第三章

• 队列：先入先出

顺序队

四要素：
队空条件：front==rear
队满条件：rear ==MaxSize - 1
进队e操作：rear++; data[rear]=e
元素e出队：front++; e=data[front]

• 环形队列：（防止假溢出）

rear++: rear=(rear+1)%MaxSize

front++: front=(front+1)%MaxSize

队满条件：(rear+1)%MaxSize == front

链队

四要素：

队空条件：q->front==NULL或者q->rear==NULL

队满条件：不考虑

进队e操作：将包含e的结点插入到单链表表尾

退队操作：删除单链表首数据结点

1.若一个栈采用数组 s 存放其元素，且初始时栈顶指针为0，则以下元素 x 进栈的 正确操作是 () A

- | | |
|-----------------------|-----------------------|
| A. $top++; s[top]=x;$ | B. $s[top]=x; top++;$ |
| C. $top--; s[top]=x;$ | D. $s[top]=x; top--;$ |

2.若一个栈采用数组 s 存放其元素，MaxSize为 n ，且初始时栈顶指针为 n ，则以下元素 x 进栈的 正确操作是 () C

- | | |
|-----------------------|-----------------------|
| A. $top++; s[top]=x;$ | B. $s[top]=x; top++;$ |
| C. $top--; s[top]=x;$ | D. $s[top]=x; top--;$ |

3.设有一个栈，元素依次进栈的顺序为 1、2、3、4、5。下列 () 是不可能的出栈序列。 C

- | | | | |
|--------------|--------------|--------------|--------------|
| A. 1,2,3,4,5 | B. 5,4,3,2,1 | C. 3,1,2,4,5 | D. 4,3,5,2,1 |
|--------------|--------------|--------------|--------------|



复习



4. 表达式 $a * (b + c) - d$ 的后缀表达式为: $abc + * d -$

5. 后缀表达式 $20 \ 15 \ 2 \ + \ - \ 27 \ 9 \ / \ *$ 运算后的值为_____。 9

6. 判断一个循环队列 q (m 为最多元素, $q \rightarrow \text{front}$ 和 $q \rightarrow \text{rear}$ 分别指向队头和队尾) 为满的条件为 ()。 C

A. $q \rightarrow \text{front} == q \rightarrow \text{rear}$

B. $q \rightarrow \text{front} != q \rightarrow \text{rear}$

C. $q \rightarrow \text{front} == (q \rightarrow \text{rear} + 1) \% m$

D. $q \rightarrow \text{front} != (q \rightarrow \text{rear} + 1) \% m$

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

8.4 生成树和最小生成树

▶ 8.5 最短路径

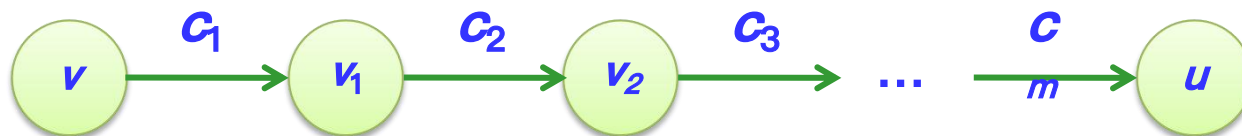
8.6 拓扑排序

8.7 AOE网与关键路径

8.5 最短路径

8.5.1 路径的概念

考虑带权有向图，把一条路径（仅仅考虑简单路径）上所经边的权值之和定义为该路径的**路径长度**或称**带权路径长度**。

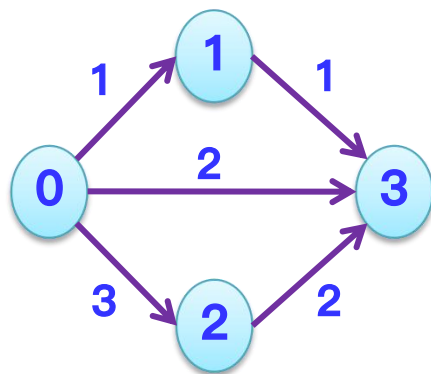


$$\text{路径长度} = c_1 + c_2 + \dots + c_m$$

路径: $(v, v_1, v_2, \dots, u)$

从源点到终点可能不止一条路径，把路径长度最短的那条路径称为**最短路径**。

带权有向图



顶点0到3:

- 最短路径: $0 \rightarrow 1 \rightarrow 3$; $0 \rightarrow 3$
- 最短路径长度: 2

最短路径不一定唯一，但是最短路径长度唯一

如何求图中的最短路径？

- **Dijkstra**（狄克斯特拉）算法
- **Floyd**算法

8.5.2 从一个顶点到其余各顶点的最短路径

问题描述： 给定一个带权有向图 G 与源点 v ，求从 v 到 G 中其他顶点的最短路径，并限定各边上的权值大于或等于0。

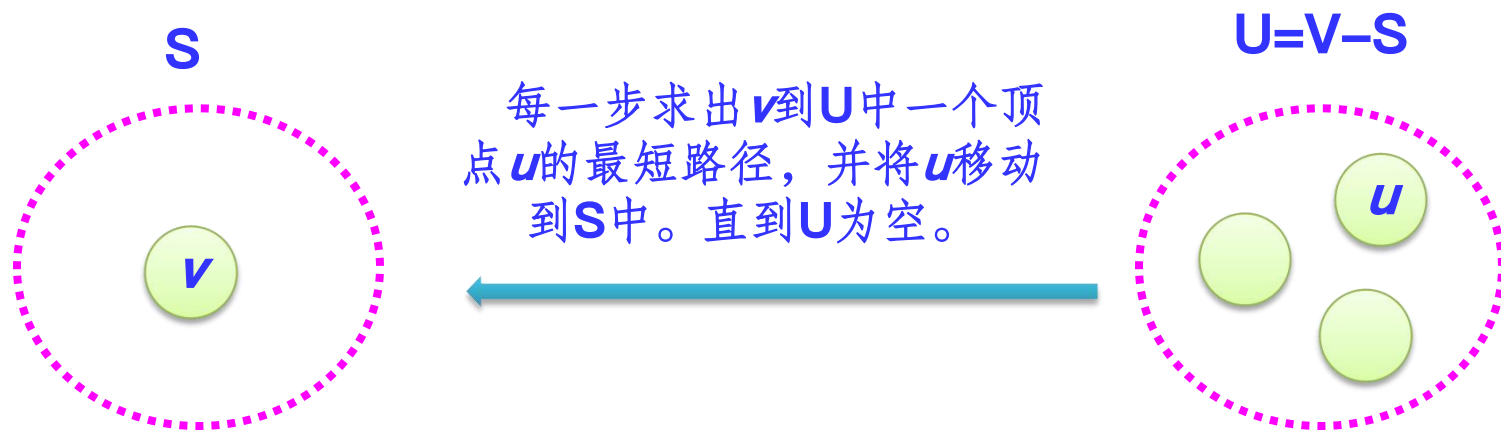
单源最短路径问题： **Dijkstra**（狄克斯特拉） 算法



狄克斯特拉 (Dijkstra) 求解思路

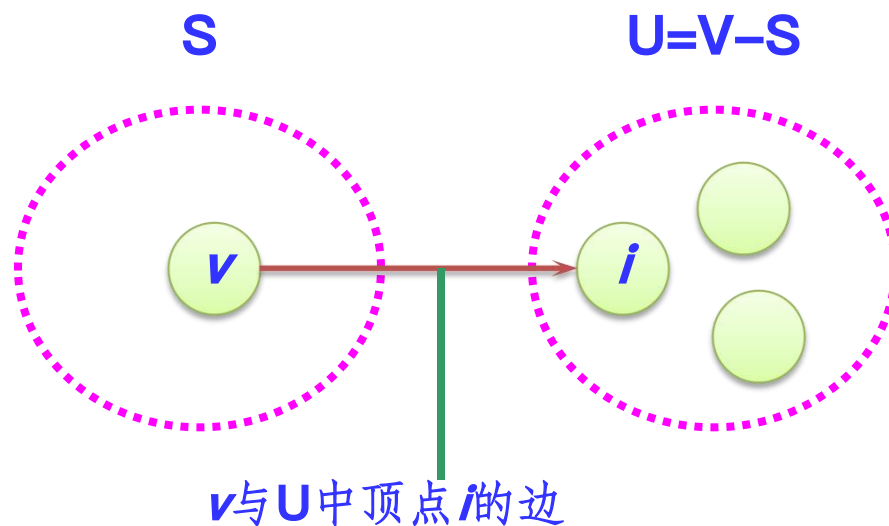
设 $G=(V, E)$ 是一个带权有向图，把图中顶点集合 V 分成两组：

- **第1组**为已求出最短路径的顶点集合（用 S 表示，初始时 S 中只有一个源点，以后每求得一条最短路径 $v, \dots, u$ ，就将 u 加入到集合 S 中，直到全部顶点都加入到 S 中，算法就结束了）。
- **第2组**为其余未求出最短路径的顶点集合（用 U 表示）。

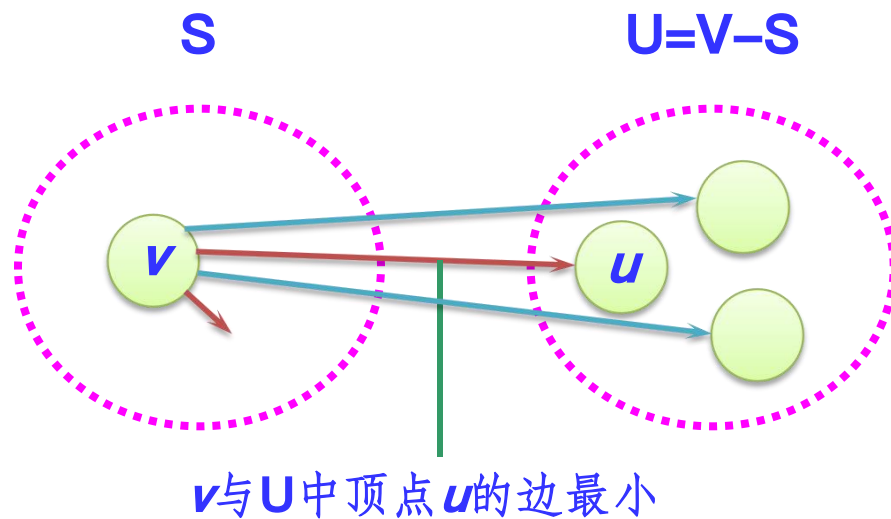


狄克斯特拉算法的过程

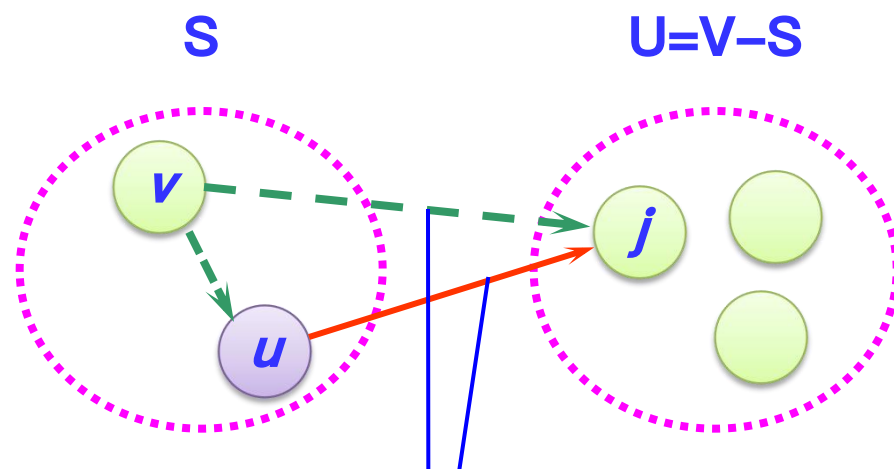
(1) 初始化: S 只包含源点即 $S=\{v\}$, v 的最短路径为 0。 U 包含除 v 外的其他顶点, U 中顶点 i 距离为边上的权值 (若 v 与 i 有边 $\langle v, i \rangle$) 或 ∞ (若 i 不是 v 的出边邻接点)。



(2) 从 U 中选取一个距离 v 最小的顶点 u , 把 u 加入 S 中 (该选定的距离就是 $v \rightarrow u$ 的最短路径长度)。

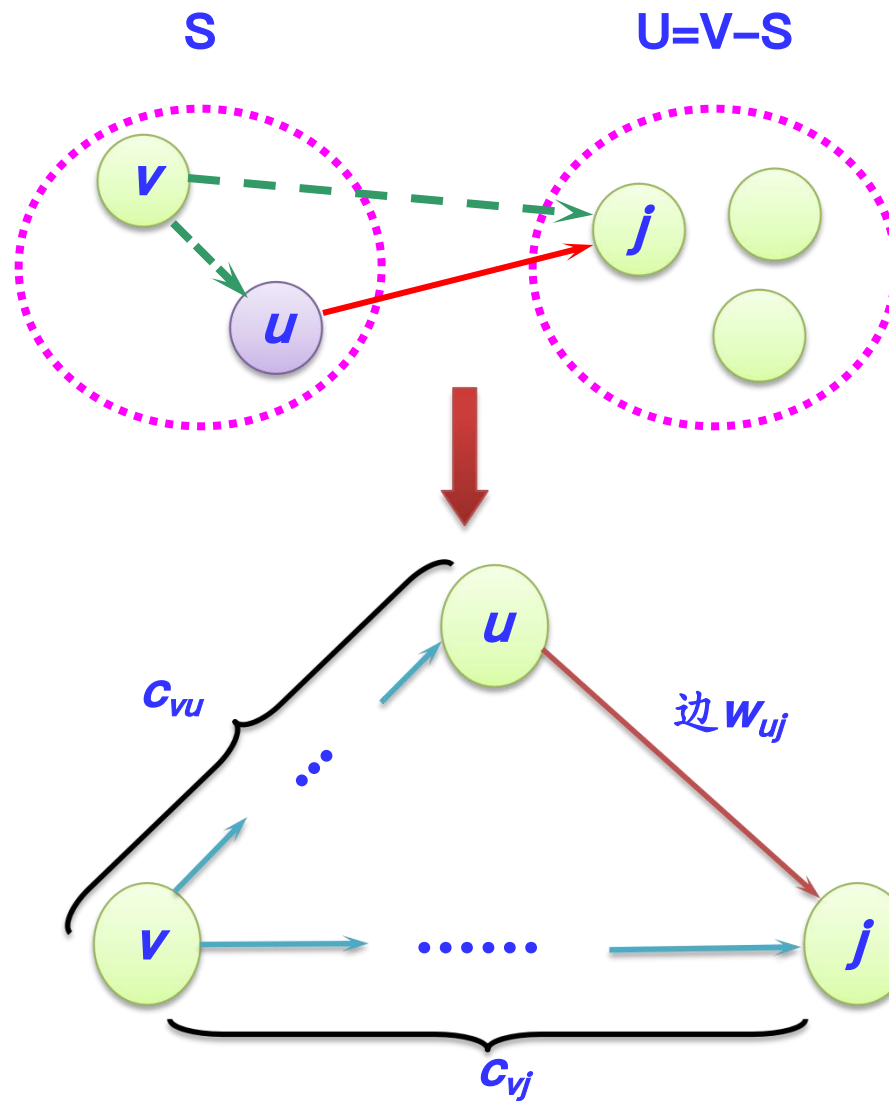


(3) 以 u 为新考虑的中间点，修改 U 中各顶点 j 的最短路径长度：若从源点 $v \rightarrow j$ ($j \in U$) 的最短路径长度（经过顶点 u ）比原来最短路径长度（不经过顶点 u ）短，则修改顶点 j 的最短路径长度。



两条路径进行比较：
若经过 u 的最短路径长度更短，则修正

修改方式

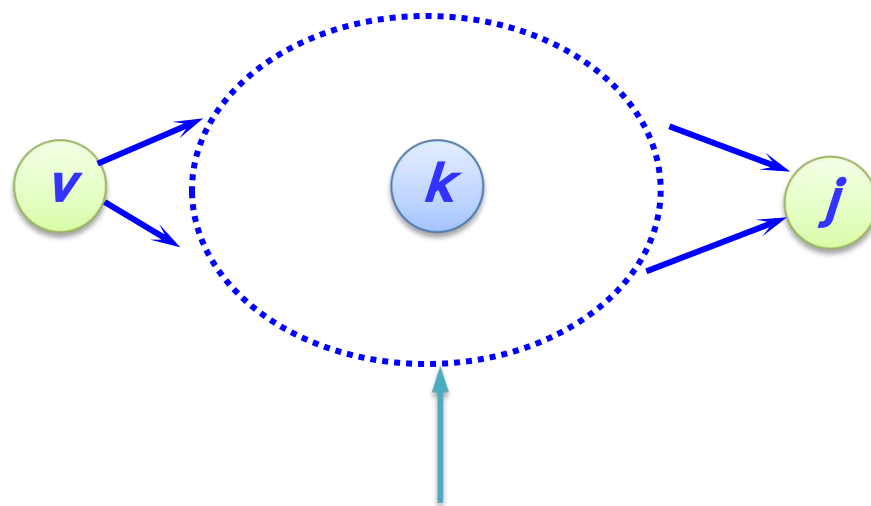


$v \Rightarrow j$ 的路径:

- 不经过顶点 u
- 经过顶点 u

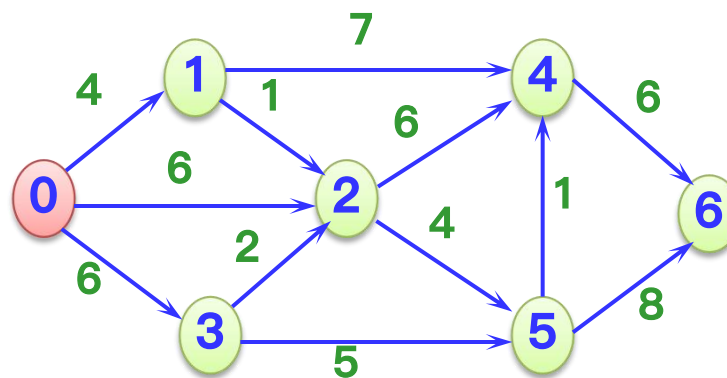
顶点 $v \Rightarrow j$ 的最短路径长度 = $\text{MIN}(c_{vk} + w_{kj}, c_{vj})$

(4) 重复步骤 (2) 和 (3) 直到所有顶点都包含在 **S** 中。



考虑中间其他所有顶点 k ，通过比较
得到 $v \Rightarrow j$ 的最短路径

Dijkstra算法示例演示



S

U

dist[]

path[]

{0}

{1,2,3,4,5,6}

0 1 2 3 4 5 6
 {0, 4, 6, 6, ∞, ∞, ∞}

0 1 2 3 4 5 6
 {0, 0, 0, 0, -1, -1, -1}

最小的顶点: 1

{0,1}

{2,3,4,5,6}

{0, 4, 5, 6, 11, ∞, ∞}

{0, 0, 1, 0, 1, -1, -1}

最小的顶点: 2

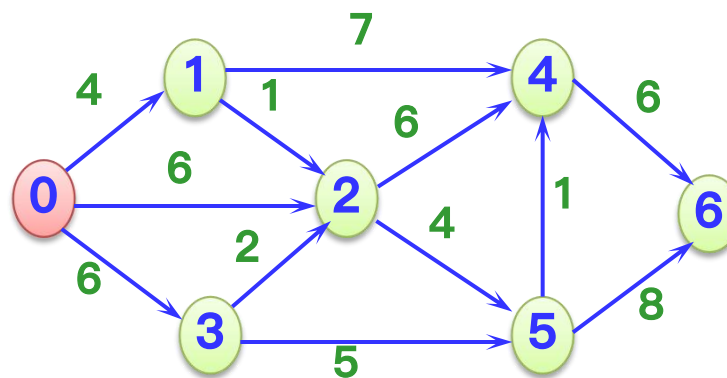
{0,1,2}

{3,4,5,6}

{0, 4, 5, 6, 11, 9, ∞}

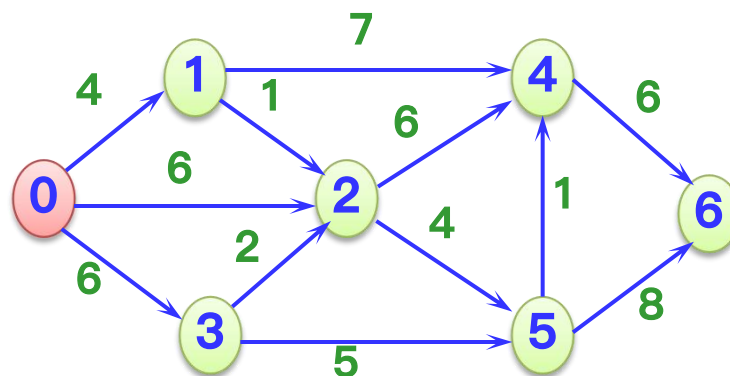
{0, 0, 1, 0, 1, 2, -1}

Dijkstra算法示例演示



S	U	dist[]	path[]
		0 1 2 3 4 5 6	0 1 2 3 4 5 6
{0,1,2}	{3,4,5,6}	{0, 4, 5, <u>6</u> , 11, 9, ∞}	{0, 0, 1, 0, 1, 2, -1}
		↓ 最小的顶点: 3	
{0,1,2,3}	{4,5,6}	{0, 4, 5, 6, <u>11</u> , 9, ∞}	{0, 0, 1, 0, 1, 2, -1}
		↓ 最小的顶点: 5	
{0,1,2,3,5}	{4,6}	{0, 4, 5, 6, 10, 9, <u>17</u> }	{0, 0, 1, 0, 5, 2, 5}

Dijkstra算法示例演示



S	U	dist[]	path[]
		0 1 2 3 4 5 6	0 1 2 3 4 5 6
{0,1,2,3,5}	{4,6}	{0, 4, 5, 6, <u>10</u> , 9, <u>17</u> }	{0, 0, 1, 0, 5, 2, 5}
		最小的顶点: 4	
{0,1,2,3,5,4}	{6}	{0, 4, 5, 6, 10, 9, <u>16</u> }	{0, 0, 1, 0, 5, 2, 4}
		最小的顶点: 6	
{0,1,2,3,5,4,6}	{}	{0, 4, 5, 6, 10, 9, 16}	{0, 0, 1, 0, 5, 2, 4}
最终结果			

利用dist和path求最短路径长度和最短路径

① 求0 → 6的最短路径长度:

0 1 2 3 4 5 6
dist={0, 4, 5, 6, 10, 9, 16}

从顶点0 → 6的最短路径长度为16

② 求0 → 6的最短路径:

0 1 2 3 4 5 6
path={0, 0, 1, 0, 5, 2, 4}

path[6]=4

path[4]=5

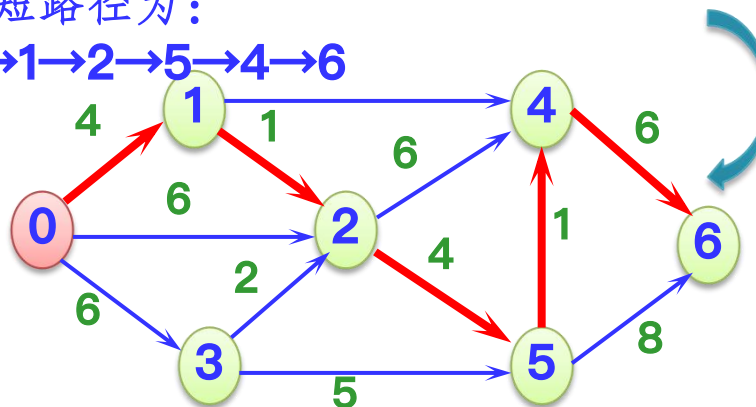
path[5]=2

path[2]=1

path[1]=0到源点

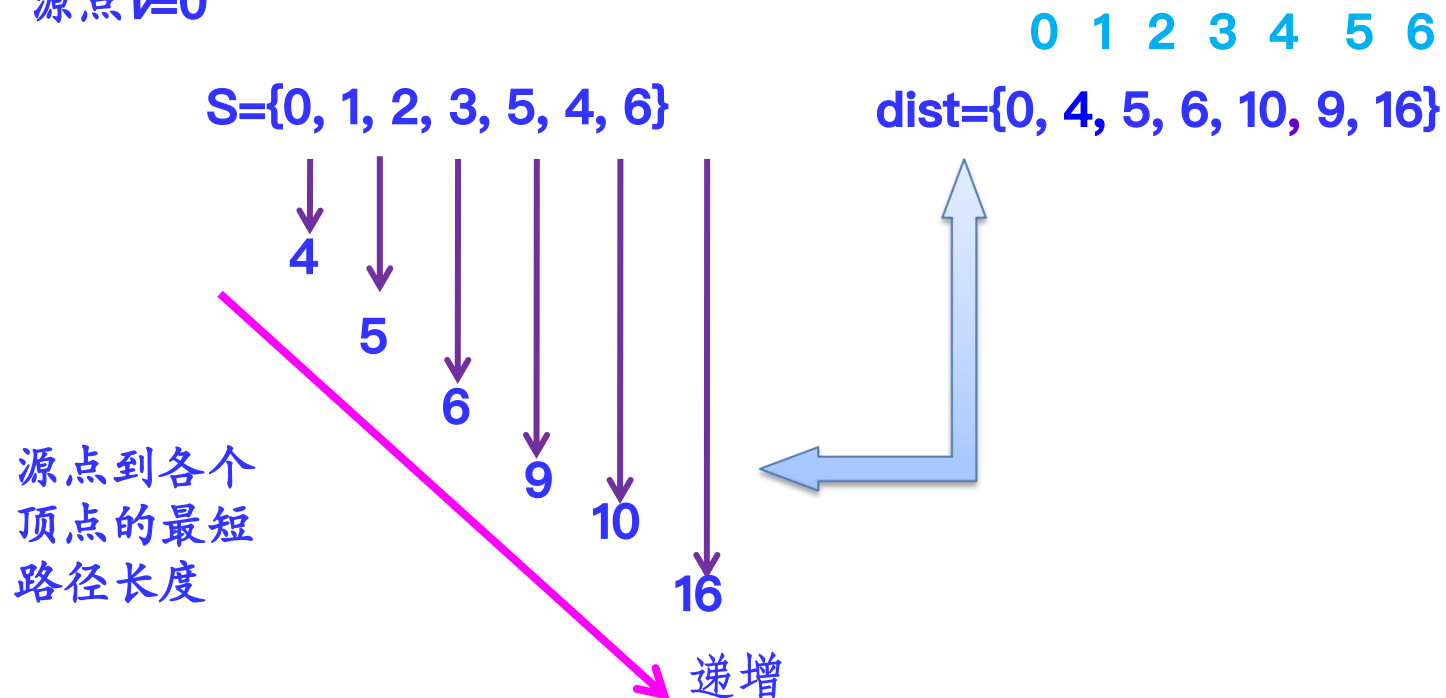
最短路径为:

0→1→2→5→4→6



观察求解结果

源点 $v=0$

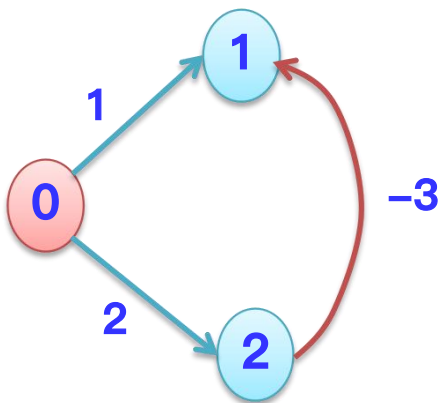


结论:

- ① 按顶点进入S的先后顺序, 最短路径长度越来越长。
- ② 一个顶点一旦进入S后, 其最短路径长度不再改变 (调整)。

Dijkstra算法不适合负权值的情况

用一个反例说明



$v=0$, $S=\{0\}$, v 到1的距离为1, v 到2的距离为2



选取顶点1, $S=\{0,1\}$, v 到1的距离为1:

- 以后不再调整
- 实际上 v 到1的最小距离为 $2-3=-1$

算法设计 (解决2个问题)

● 如何存放最短路径长度:

用一维数组 **dist[]** 存储!

源点 v 默认, **dist[]** 表示源点 $\Rightarrow$ 顶点 j 的最短路径长度。如 **dist[2]=12** 表示源点 $\Rightarrow$ 顶点 **2** 的最短路径长度为 **12**。

● 如何存放最短路径:

从源点到其他顶点的最短路径有 $n-1$ 条, 一条最短路径用一个一维数组表示, 如从顶点 **0** $\Rightarrow$ **5** 的最短路径为 **0、2、3、5**, 表示为

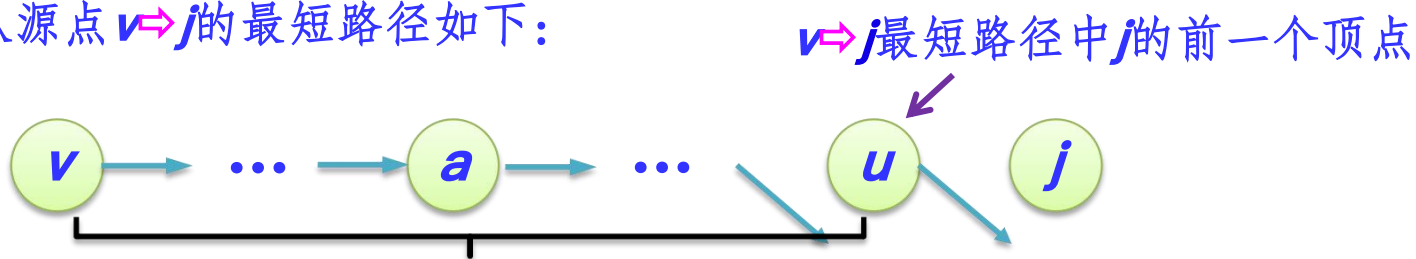
path[5]={0,2,3,5}。

所有 $n-1$ 条最短路径可以用二维数组 **path[][]** 存储。

?

改进的方法是采用一维数组 **path** 来保存:

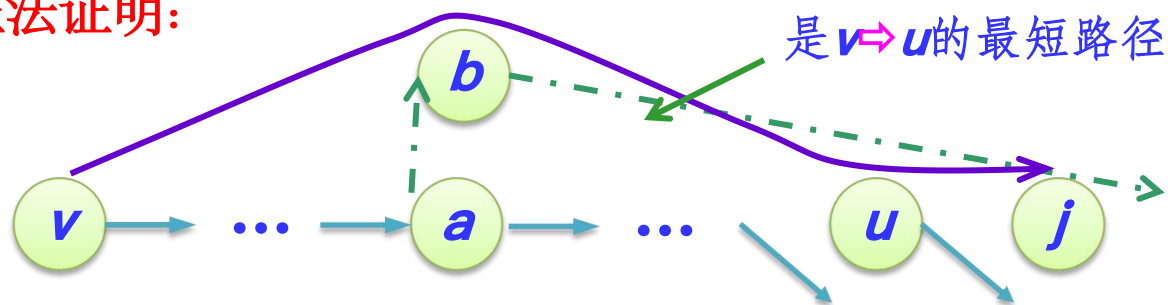
若从源点 $v \rightarrow j$ 的最短路径如下:



则

一定是从源点 $v \rightarrow u$ 的最短路径 ?

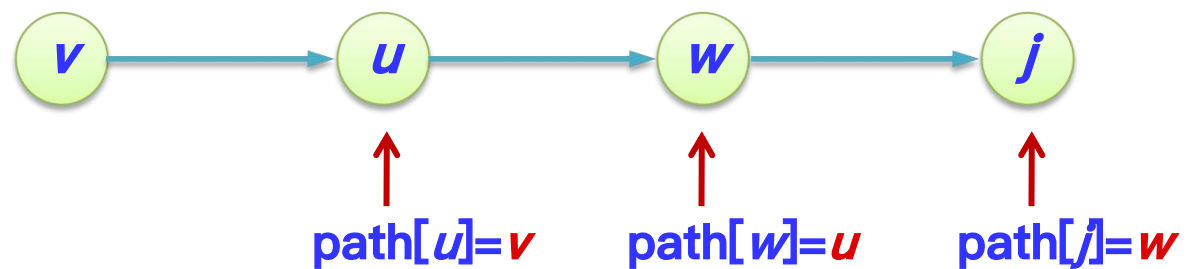
反证法证明:



而通过 b 的路径更短, 则 $v \rightarrow \dots a \rightarrow \dots u \rightarrow j$ 不是最短路径

与假设矛盾, 问题得到证明。

$v \Rightarrow j$ 的最短路径:



- 从 $\text{path}[j]$ 推出的逆路径: j, w, u, v
- 对应的最短路径为: $v \rightarrow u \rightarrow w \rightarrow j$

狄克斯特拉算法如下 (v为源点编号) :

```
void Dijkstra(MatGraph g, int v)
{ int dist[MAXV], path[MAXV];
  int s[MAXV];
  int mindis,i,j,u;
  for (i=0;i<g.n;i++)
  { dist[i]=g.edges[v][i]; //距离初始化
    s[i]=0; //s[]置空
    if (g.edges[v][i]<INF) //路径初始化
      path[i]=v; //顶点v到i有边时
    else
      path[i]=-1; //顶点v到i没边时
  }
  s[v]=1; //源点v放入S中
```

dist和path
数组初始化

```

for (i=0;i<g.n;i++)                                //循环n-1次
{ mindis=INF;
  for (j=0;j<g.n;j++)
    if (s[j]==0 && dist[j]<mindis)
    { u=j;
      mindis=dist[j];
    }
  s[u]=1;                                           //顶点u加入S中
  for (j=0;j<g.n;j++)                               //修改不在s中的顶点的距离
  { if (s[j]==0)
    { if (g.edges[u][j]<INF &&
        dist[u]+g.edges[u][j]<dist[j])
      { dist[j]=dist[u]+g.edges[u][j];
        path[j]=u;
      }
    }
  }
}
Dispath(dist,path,s,g.n,v);                         //输出最短路径
}

```

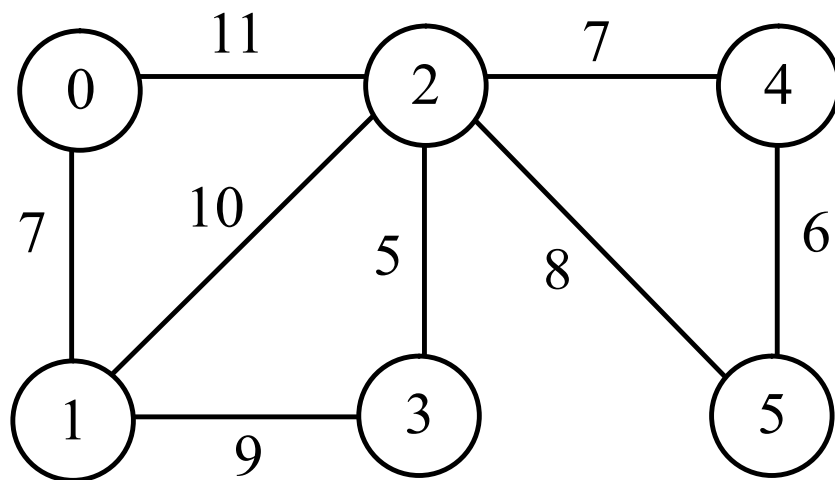
找最小路径长度顶点 **u**

调整

狄克斯特拉算法的时间复杂度为 $O(n^2)$ 。

习题

有一个无向带权图如图所示，采用**Dijkstra**算法求顶点**0**到其他顶点的最短路径和最短路径长度，要求给出求解过程（即给出求最短路径中各步骤的**S**、**dist**和**path**值）。



8.5.3 每对顶点之间的最短路径

问题描述： 对于一个各边权值均大于零的有向图，对每一对顶点 $i \neq j$ ，求出顶点 i 与顶点 j 之间的最短路径和最短路径长度。

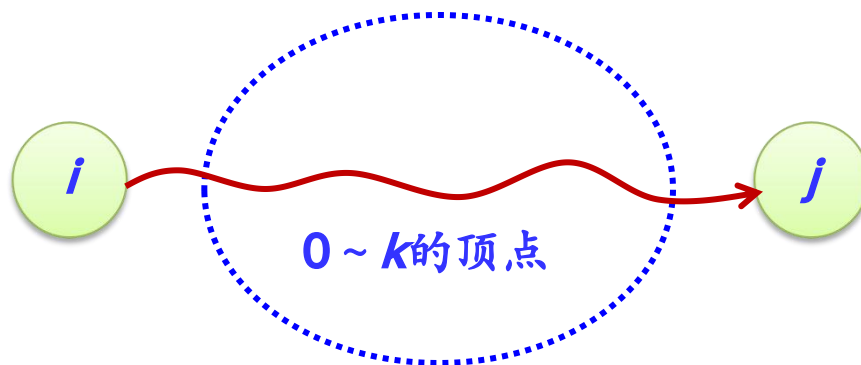
多源最短路径问题： **Floyd**算法



1936~2001

算法：迭代（递推）思路

假设有向图 $G=(V, E)$ 采用邻接矩阵存储。设置一个二维数组 A 用于存放当前顶点之间的最短路径长度，分量 $A[i][j]$ 表示当前顶点 $i \rightarrow j$ 的最短路径长度。



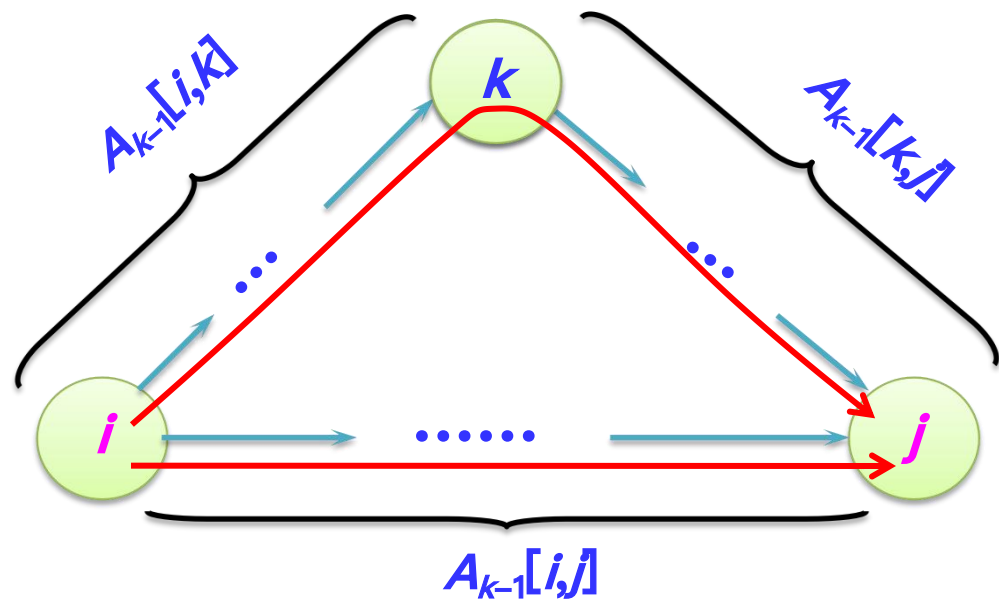
按顶点0, 1, 2, ...的顺序依次考虑

递推产生一个矩阵序列：

$$A_0 \Rightarrow A_1 \Rightarrow \cdots \Rightarrow A_k \Rightarrow \cdots \Rightarrow A_{n-1}.$$

$A_k[i][j]$: $i \rightarrow j$ 的路径上所经过的顶点编号不大于 k 的最短路径长度。

- ① 初始时, 有 $A_1[i,j]=g.edges[i,j]$ 。
- ② 若 A_{k-1} 已经求出。
- ③ 考虑顶点 k : 求从 $i \Rightarrow j$ 的最短路径经过编号为 k 顶点的情况。



$$A_k[i,j] = \text{MIN} \{ A_{k-1}[i,j], A_{k-1}[i,k] + A_{k-1}[k,j] \}$$



$$A_1[i,j] = g.edges[i,j]$$

$$A_k[i,j] = \text{MIN} \{ A_{k-1}[i,j], A_{k-1}[i,k] + A_{k-1}[k,j] \} \quad 0 \leq k \leq n-1$$

算法设计 (解决2个问题)

(1) 用二维数组 A 存储最短路径长度:

- $A_k[i][j]$ 表示考虑顶点 $0 \sim k$ 后得出的 $i \rightarrow j$ 的最短路径长度。
- $A_{n-1}[i][j]$ 表示最终的 $i \rightarrow j$ 的最短路径长度。

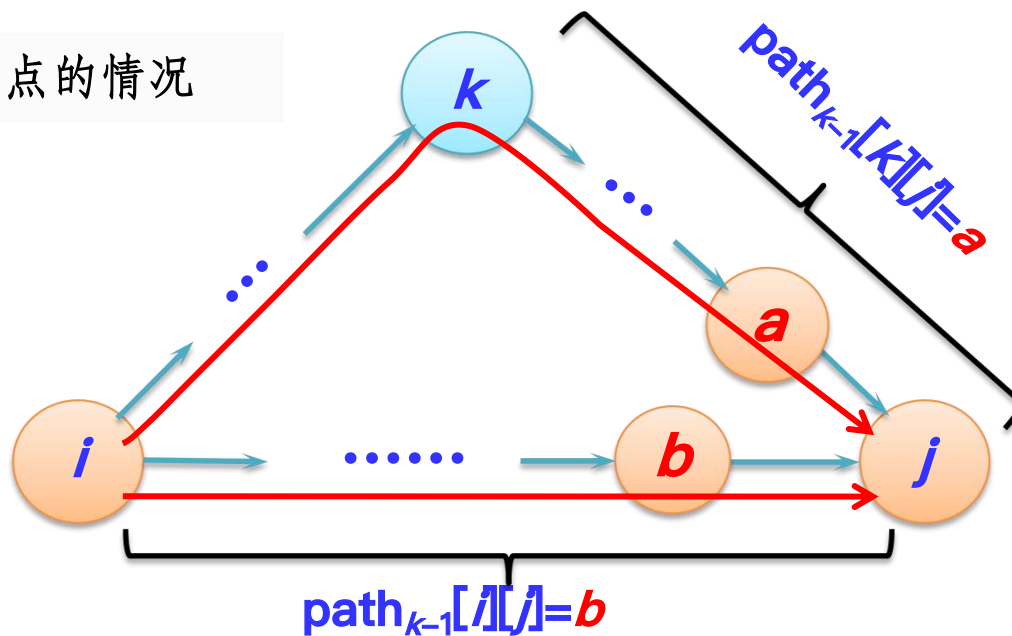
(2) 用二维数组 $path$ 存放最短路径:

- $path_k[i][j]$ 表示考虑顶点 $0 \sim k$ 后得出的 $i \rightarrow j$ 的最短路径。
- $path_{n-1}[i][j]$ 表示最终 $i \rightarrow j$ 的最短路径。

如何用path存放最短路径?

$\text{path}_x[i][j]$ 表示考虑过 $0 \sim x$ 的顶点得到 $i \rightarrow j$ 的最短路径, 存放该路径上顶点 j 的前一个顶点编号。

① 考虑过 $0 \sim k-1$ 顶点的情况

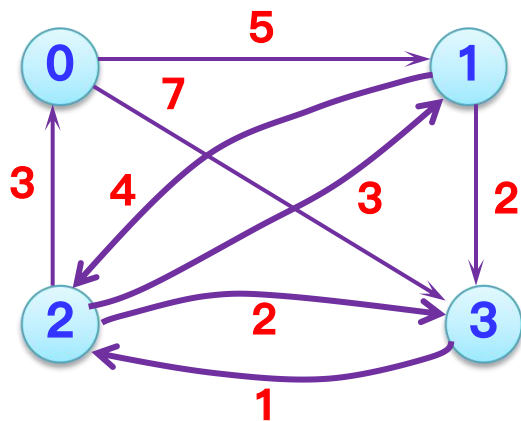


② 现在考虑顶点 k

若经过顶点 k 的路径更短: $\text{path}_k[i][j] = a = \text{path}_{k-1}[k][j]$

否则: $\text{path}_k[i][j] = b = \text{path}_{k-1}[i][j]$ 不改变

Floyd算法示例演示



A_1

	0	1	2	3
0	0	5	∞	7
1	∞	0	4	2
2	3	3	0	2
3	∞	∞	1	0



$$\begin{bmatrix} 0 & 5 & \infty & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$


path₁

求path

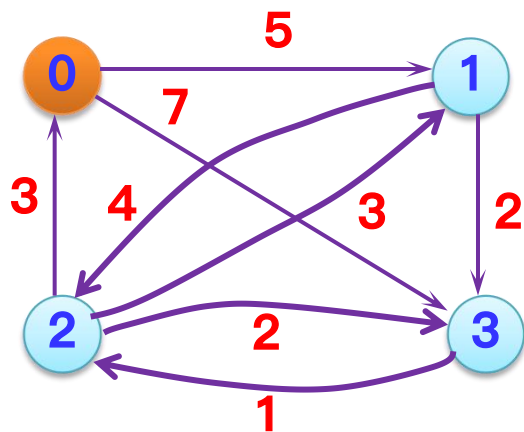
∞ 和 $i \rightarrow j$: -1

(i,j) 有边: i



	0	1	2	3
0	-1	0	-1	0
1	-1	-1	1	1
2	2	2	-1	2
3	-1	-1	3	-1

Floyd算法示例演示



A_0

	0	1	2	3
0	0	5	∞	7
1	∞	0	4	2
2	3	3	0	2
3	∞	∞	1	0

考虑顶点**0**:

● 没有任何路径修改

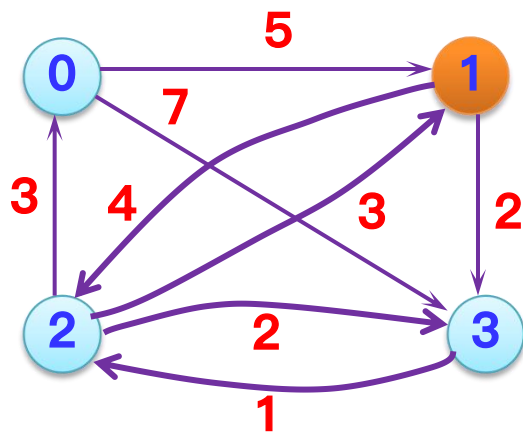
$A_0 = A_1$, $path_0 = path_1$



$path_0$

	0	1	2	3
0	-1	0	-1	0
1	-1	-1	1	1
2	2	2	-1	2
3	-1	-1	3	-1

Floyd算法示例演示



考虑顶点**1**:

- $0 \rightarrow 2$: 由无路径改为 $0 \rightarrow 1 \rightarrow 2$, 长度为**9**, $\text{path}[0][2]$ 改为**1**



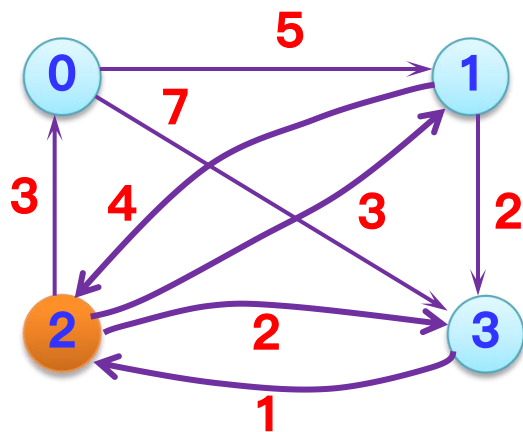
A_1

	0	1	2	3
0	0	5	9	7
1	∞	0	4	2
2	3	3	0	2
3	∞	∞	1	0

path_1

	0	1	2	3
0	-1	0	1	0
1	-1	-1	1	1
2	2	2	-1	2
3	-1	-1	3	-1

Floyd算法示例演示



考虑顶点**2**:

- **1→0**: 由无路径改为**1→2→0**, 长度为**7**,
path[1][0]改为**2**
- **3→0**: 由无路径改为**3→2→0**, 长度为**4**,
path[3][0]改为**2**
- **3→1**: 由无路径改为**3→2→1**, 长度为**4**,
path[3][1]改为**2**

A_2

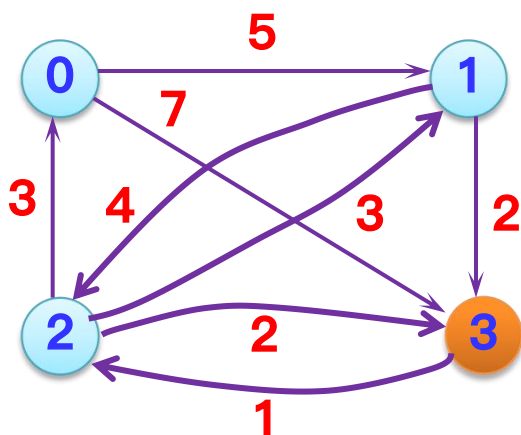
	0	1	2	3
0	0	5	9	7
1	7	0	4	2
2	3	3	0	2
3	4	4	1	0



$path_2$

	0	1	2	3
0	-1	0	1	0
1	2	-1	1	1
2	2	2	-1	2
3	2	2	3	-1

Floyd算法示例演示



考虑顶点**3**:

- $0 \rightarrow 2$: 由 $0 \rightarrow 1 \rightarrow 2$ 改为 $0 \rightarrow 3 \rightarrow 2$, 长度为**8**, $\text{path}[0][2]$ 改为**3**
- $1 \rightarrow 0$: 由 $1 \rightarrow 2 \rightarrow 0$ 改为 $1 \rightarrow 3 \rightarrow 2 \rightarrow 0$, 长度为**6**, $\text{path}[1][0]$ 改为**2**
- $1 \rightarrow 2$: 由 $1 \rightarrow 2$ 改为 $1 \rightarrow 3 \rightarrow 2$, 长度为**3**, $\text{path}[1][2]$ 改为**3**

A_3

	0	1	2	3
0	0	5	8	7
1	6	0	3	2
2	3	3	0	2
3	4	4	1	0



path_3

	0	1	2	3
0	-1	0	3	0
1	2	-1	3	1
2	2	2	-1	2
3	2	2	3	-1

求最终结果

A_3

	0	1	2	3
0	0	5	8	7
1	6	0	3	2
2	3	3	0	2
3	4	4	1	0

$path_3$

	0	1	2	3
0	-1	0	3	0
1	2	-1	3	1
2	2	2	-1	2
3	2	2	3	-1

① 求最短路径长度:

由 A_3 数组可以直接得到两个顶点之间的最短路径长度。

如 $A_3[1][0]=6$

说明顶点1到0的最短路径长度为6。

A_3

	0	1	2	3
0	0	5	8	7
1	6	0	3	2
2	3	3	0	2
3	4	4	1	0

 $path_3$

	0	1	2	3
0	-1	0	3	0
1	2	-1	3	1
2	2	2	-1	2
3	2	2	3	-1

② 求最短路径:

求顶点1 $\Rightarrow$ 0的最短路径:

$path_3[1][0]=2$ 0的前驱顶点是2

$path_3[1][2]=3$ 2的前驱顶点是3

$path_3[1][3]=1$ 3的前驱顶点是1

查找的顶点序列为0、2、3、1，则顶点1 $\Rightarrow$ 0的最短路径为1 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 0。

弗洛伊德算法如下:

```
void Floyd(MatGraph g) //求每对顶点之间的最短路径
```

```
{ int A[MAXVEX][MAXVEX];    //建立A数组
```

```
  int path[MAXVEX][MAXVEX]; //建立path数组
```

```
  int i, j, k;
```

```
  for (i=0;i<g.n;i++)
```

```
  for (j=0;j<g.n;j++)
```

```
  { A[i][j]=g.edges[i][j];
```

```
    if (i!=j && g.edges[i][j]<INF)
```

```
        path[i][j]=i;    //i和j顶点之间有一条边时
```

```
    else                                //i和j顶点之间没有一条边时
```

```
        path[i][j]=-1;
```

```
  }
```

A和path数组初始化

```
for (k=0;k<g.n;k++)           //求Ak[i][j]
{
    for (i=0;i<g.n;i++)
    {
        for (j=0;j<g.n;j++)
        {
            if (A[i][j]>A[i][k]+A[k][j])    //找到更短路径
            {
                A[i][j]=A[i][k]+A[k][j];    //修改路径长度
                path[i][j]=path[k][j];      //修改最短路径为经过顶点k
            }
        }
    }
}
```

调整

本算法的时间复杂度为 $O(n^3)$ 。

算法比较

算法	用途	时间复杂度	特点
Dijkstra	单源最短路径	$O(n^2)$	不适合负权
Floyd	多源最短路径	$O(n^3)$	不适合负权回路

数据结构经典算法的启示

用Dijkstra求所有顶点之间的最短路径



共享前面路径比较所得到的信息 **A**

Floyd算法

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

8.4 生成树和最小生成树

8.5 最短路径

▶ 8.6 拓扑排序

8.7 AOE网与关键路径

8.6 拓扑排序

1、什么是拓扑排序

设 $G=(V, E)$ 是一个具有 n 个顶点的有向图， V 中顶点序列 $v_1, v_2, \dots, v_n$ 称为一个**拓扑序列**，当且仅当该顶点序列满足下列条件：

若 $\langle i, j \rangle$ 是图中的边（或从顶点 $i \Rightarrow j$ 有一条路径）：



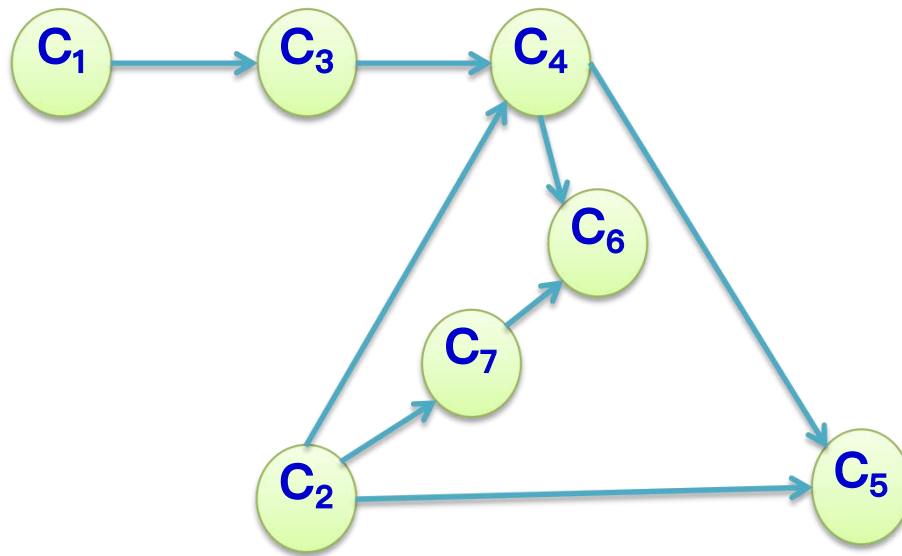
则在拓扑序列中顶点 i 必须排在顶点 j 之前。

在一个有向图中找一个拓扑序列的过程称为**拓扑排序**。

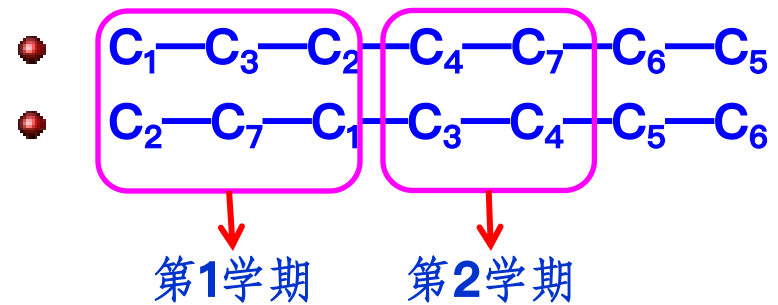
例如，计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业，假设这些课程的名称与相应代号有如下关系：

课程代号	课程名称	先修课程
C ₁	高等数学	无
C ₂	程序设计	无
C ₃	离散数学	C ₁
C ₄	数据结构	C ₂ , C ₃
C ₅	编译原理	C ₂ , C ₄
C ₆	操作系统	C ₄ , C ₇
C ₇	计算机组成原理	C ₂

课程之间的先后关系可用有向图表示:



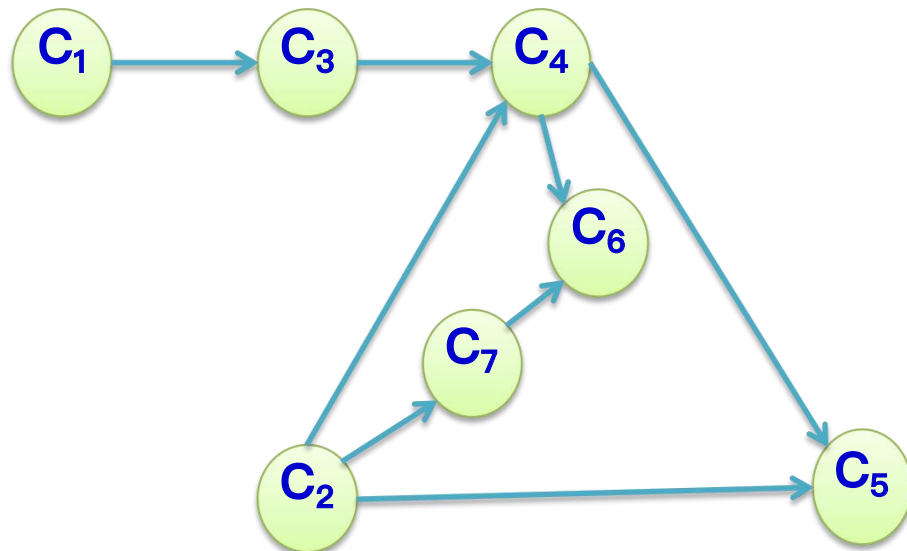
拓扑序列:



2、拓扑排序步骤

- (1) 从有向图中选择一个没有前驱（即入度为0）的顶点并且输出它。
- (2) 从图中删去该顶点，并且删去从该顶点发出的全部有向边。
- (3) 重复上述两步，直到剩余的图中不再存在没有前驱的顶点为止。

拓扑排序演示



产生一个拓扑序列:

C_1 C_3 C_2 C_7 C_4 C_6 C_5

排序完成

一个有向图拓扑排序的结果:

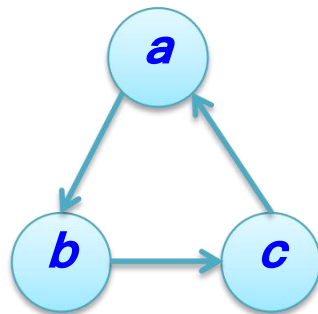
- 产生的拓扑序列包含全部的顶点
- 产生的拓扑序列不包含全部的顶点

⇒ 该图能够进行拓扑排序

⇒ 该图不能进行拓扑排序



图中存在回路





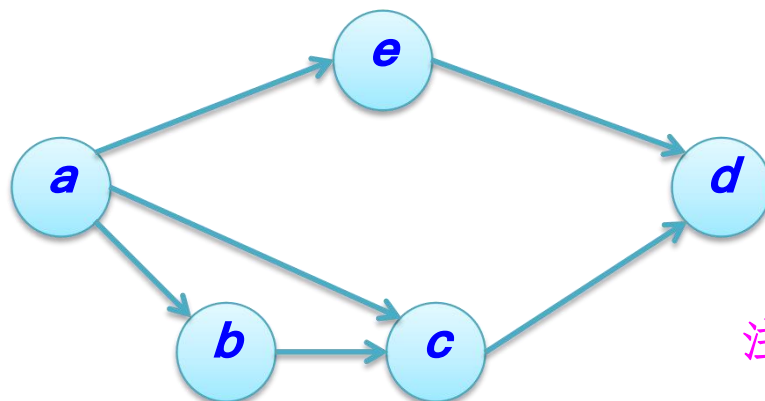
对如图所示的图进行拓扑排序，可以得到不同的拓扑序列个数是（ ）。

A. 4

B. 3

C. 2

D. 1



注：2010年全国考研题

解：不同的拓扑序列有：*aebcd*、*abced*、*abecd*。答案为**B**。

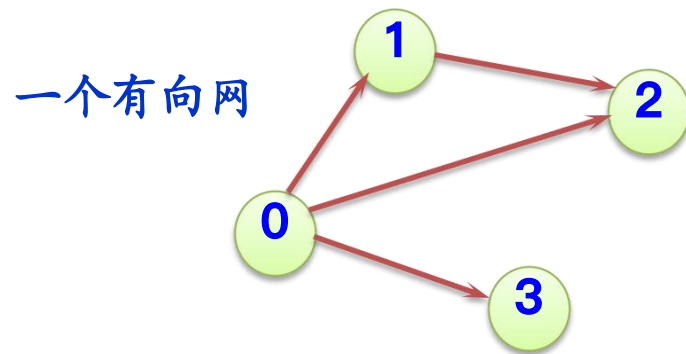
3、拓扑排序算法设计

算法设计需要解决的问题

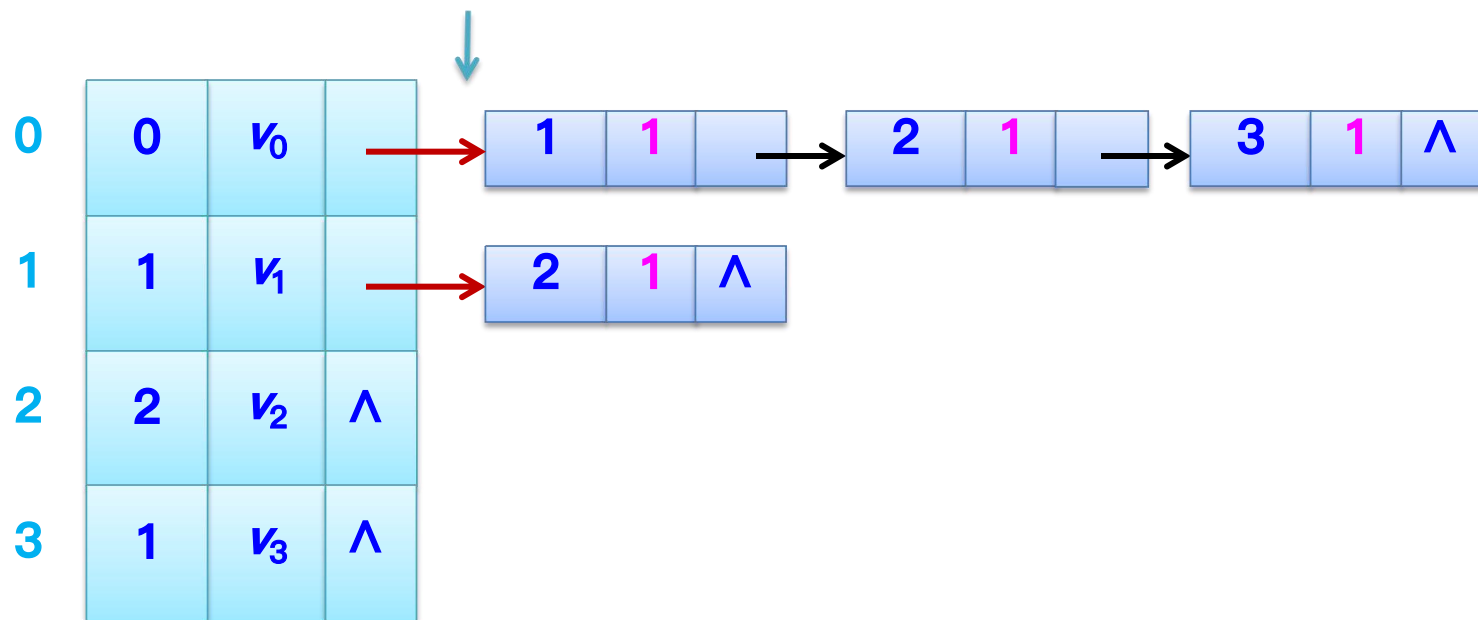
- 如何标志没有前驱（即入度为0）的顶点
- 如何删除一个顶点及其从该顶点发出的全部有向边
- 多个没有前驱（即入度为0）顶点的处理顺序

} 图存储结构中增加
每个顶点的入度

用栈保存这些顶点



顶点入度



将邻接表定义中的VNode类型修改如下:

```
typedef struct    //表头结点类型
{ Vertex data;    //顶点信息
  int count;      //存放顶点入度
  ArcNode *firstarc; //指向第一条边
} VNode;
```

用于找入度为0
的顶点



拓扑排序算法如下:

修改后的含 n 个顶点的邻接表

```
void TopSort(AdjGraph *G) //拓扑排序算法
{ int i, j;
  int St[MAXV], top=-1; //栈St的指针为top
  ArcNode *p;

  for (i=0;i<G->n;i++) //入度置初值0
    G->adjlist[i].count=0;

  for (i=0;i<G->n;i++) //求所有顶点的入度
  { p=G->adjlist[i].firstarc;
    while (p!=NULL)
    { G->adjlist[p->adjvex].count++;
      p=p->nextarc;
    }
  }
}
```



```

for (i=0;i<G->n;i++)           //将入度为0的顶点进栈
{
    if (G->adjlist[i].count==0)
    {
        top++;
        St[top]=i;
    }

while (top>-1)                 //栈不空循环
{
    i=St[top];top--;           //出栈一个顶点i
    printf("%d ",i);          //输出该顶点

    p=G->adjlist[i].firstarc;   //找第一个邻接点
    while (p!=NULL)             //将顶点i的出边邻接点的入度减1
    {
        j=p->adjvex;
        G->adjlist[j].count--;
        if (G->adjlist[j].count==0) //将入度为0的邻接点进栈
        {
            top++;
            St[top]=j;
        }

        p=p->nextarc;          //找下一个邻接点
    }
}

```

产生一个拓扑序列，时间复杂度为 $O(n+e)$ 。

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构和基本运算算法

8.3 图的遍历

8.4 生成树和最小生成树

8.5 最短路径

8.6 拓扑排序

▶ 8.7 AOE网与关键路径

8.7 AOE网与关键路径

8.7.1 相关概念

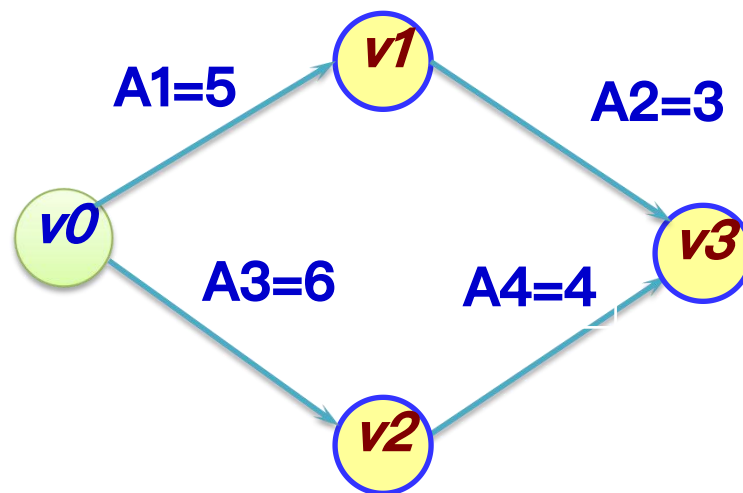
- 用一个带权有向图（**DAG**）描述工程的预计进度。
- 顶点表示事件，有向边表示活动，边 e 的权 $d(e)$ 表示完成活动 e 所需的时间（比如天数）。
- 图中入度为0的顶点表示工程的开始事件（如开工仪式），出度为0的顶点表示工程结束事件。



AOE网 (Activity On Edge)

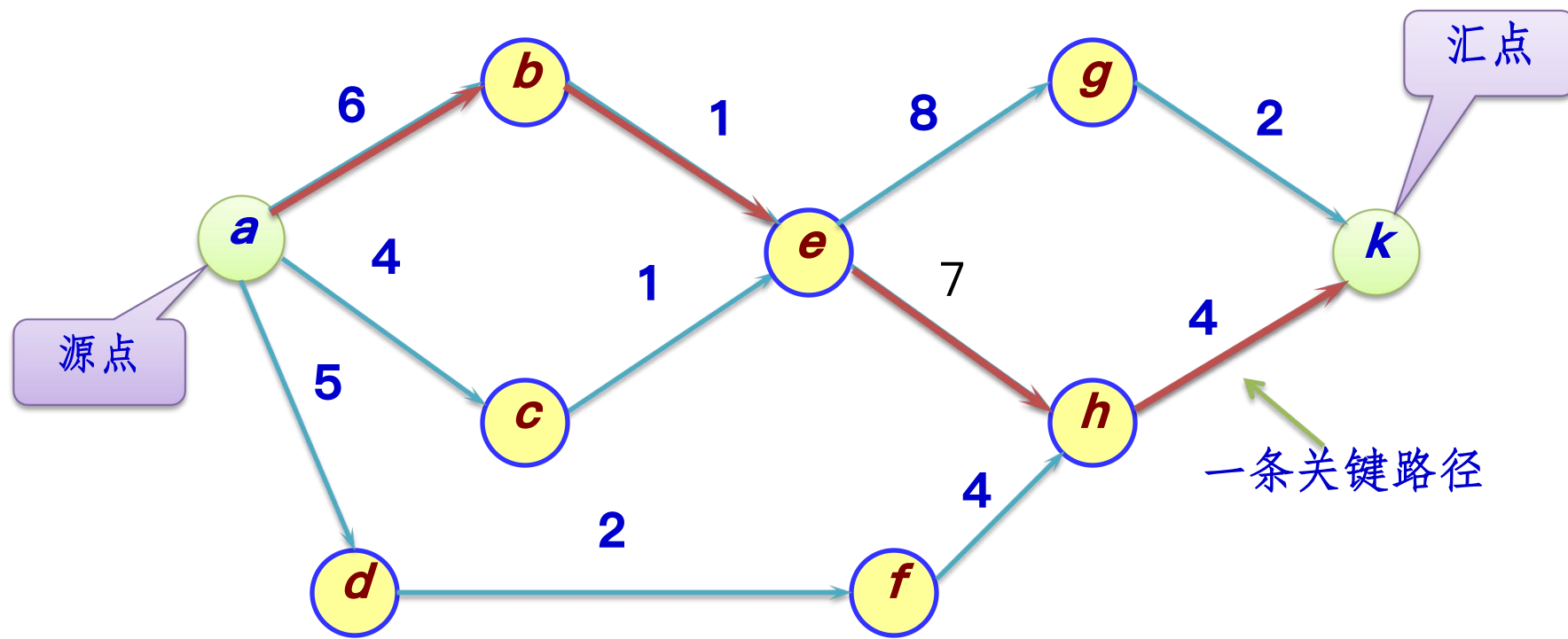
带权有向无环图

- 从AOE网中源点到汇点的最长路径，具有最大长度的路径叫**关键路径**。
- 关键路径是由**关键活动**构成的，关键路径可能不唯一。



1. 如图所示的AOE网，完成的工期至少需要多久？
2. 为缩短完成工程所需的时间，应当加快哪些活动？

关键路径演示



这里给出的求解方法:

求一个AOE的关键路径



求AOE中的关键活动

需要计算如下有关的量:

事件的最早发生时间 $ee(v)$

事件的最迟发生时间 $le(v)$

活动的最早开始时间 $e(a)$

活动的最迟开始时间 $l(a)$

某个活动的 $l(a)-e(a)=0$, 则为关键活动

顶点相关

边相关

8.7.2 求AOE网的关键活动

(1) 事件的最早开始和最迟开始时间

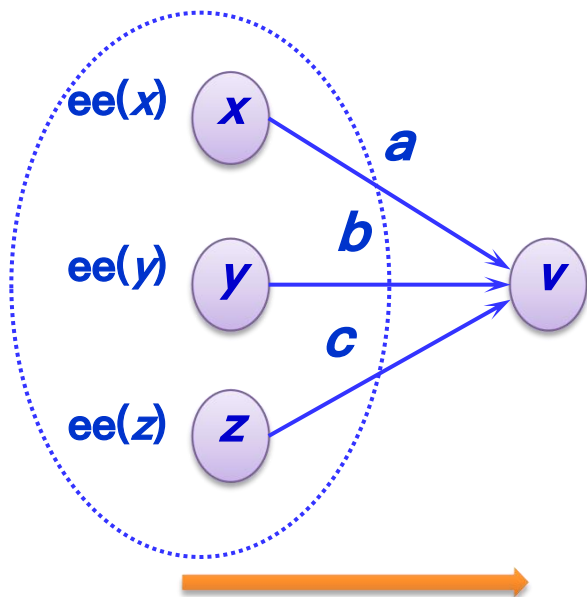
事件 v 的最早开始时间：规定源点事件的最早开始时间为0。定义图中任一事件 v 的最早开始时间（early event） $ee(v)$ 等于源点到 v 所有路径长度的最大值：

$$ee(v)=0$$

当 v 为源点时

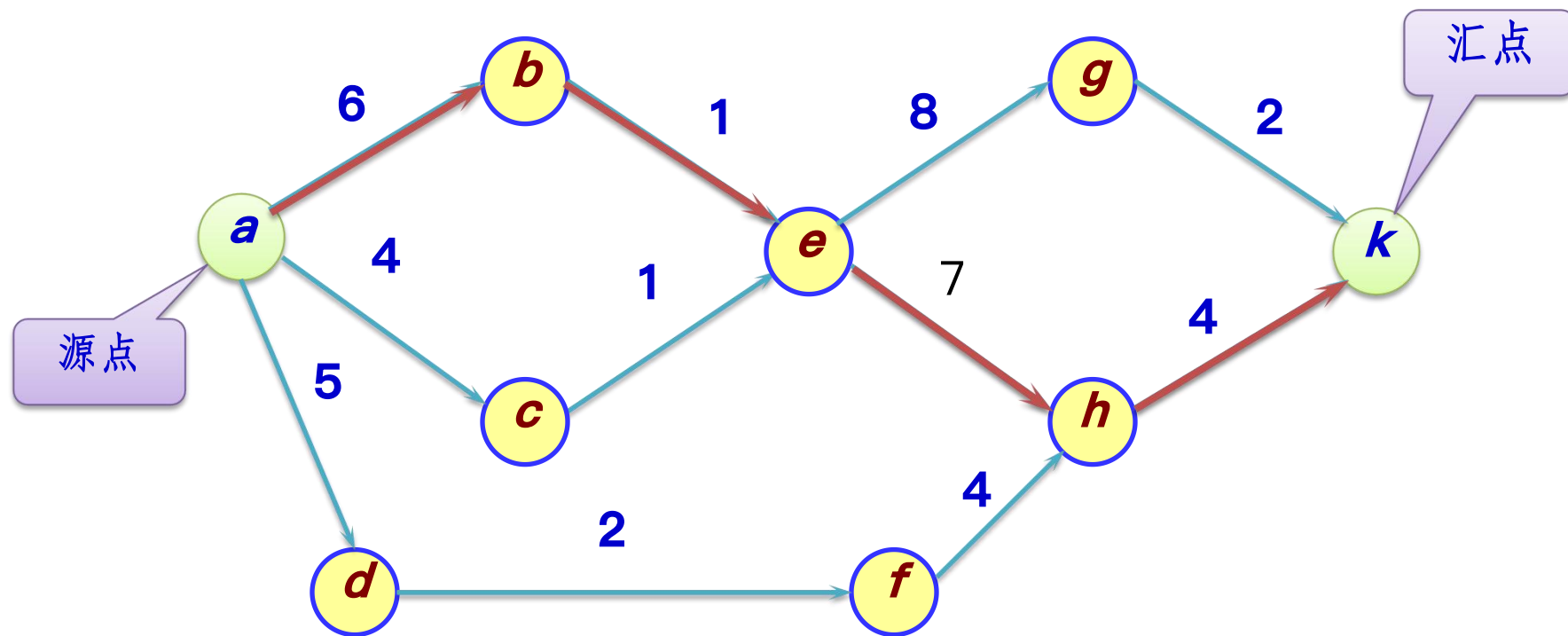
$$ee(v)=\text{MAX}\{ee(x)+a, ee(y)+b, ee(z)+c\}$$

否则



$ee(v)=?$

一个事件最早开始的时间，
取决于前面的事件都完成了的时间



	a	b	c	d	e	f	g	h	k
ee(k)	0	6	4	5	7	7	15	14	18

$$ee(e) = \text{MAX}\{ee(b)+1, ee(c)+1\}$$

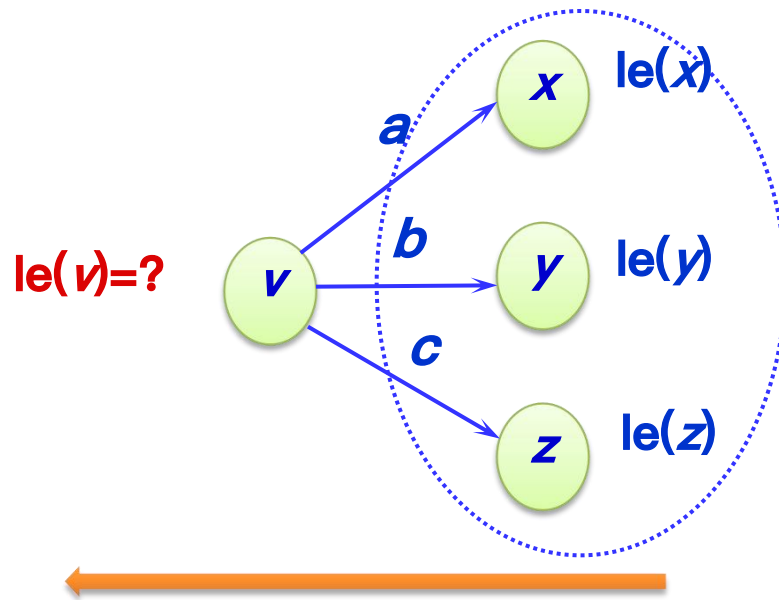
事件 v 的最迟开始时间：定义在不影响整个工程进度的前提下，事件 v 必须发生的时间称为 v 的最迟开始时间（late event），记作 $le(v)$ 。 $le(v)$ 应等于 $ee(y)$ 与 v 到汇点的最长路径长度之差：

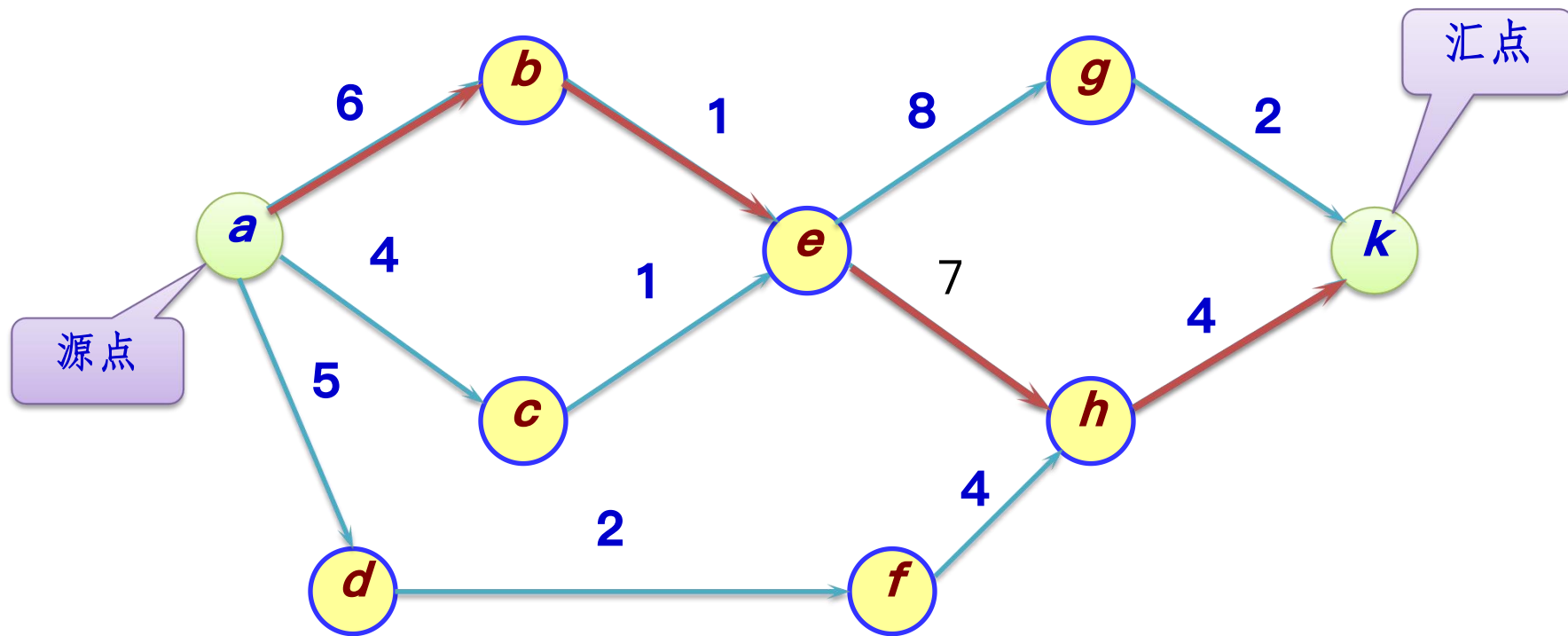
$$le(v) = ee(v)$$

当 v 为汇点时

$$le(v) = \text{MIN}\{le(x) - a, le(y) - b, le(z) - c\}$$

否则





	a	b	c	d	e	f	g	h	k
ee(k)	0	6	4	5	7	7	15	14	18

	a	b	c	d	e	f	g	h	k
le(k)	0	6	6	8	7	10	16	14	18

$$le(h) = \text{MIN}\{le(k) - 4\} = 18 - 4 = 14$$

$$le(e) = \text{MIN}\{le(g) - 8, le(h) - 7\} = 7$$

(2) 活动的最早开始时间和最迟开始时间

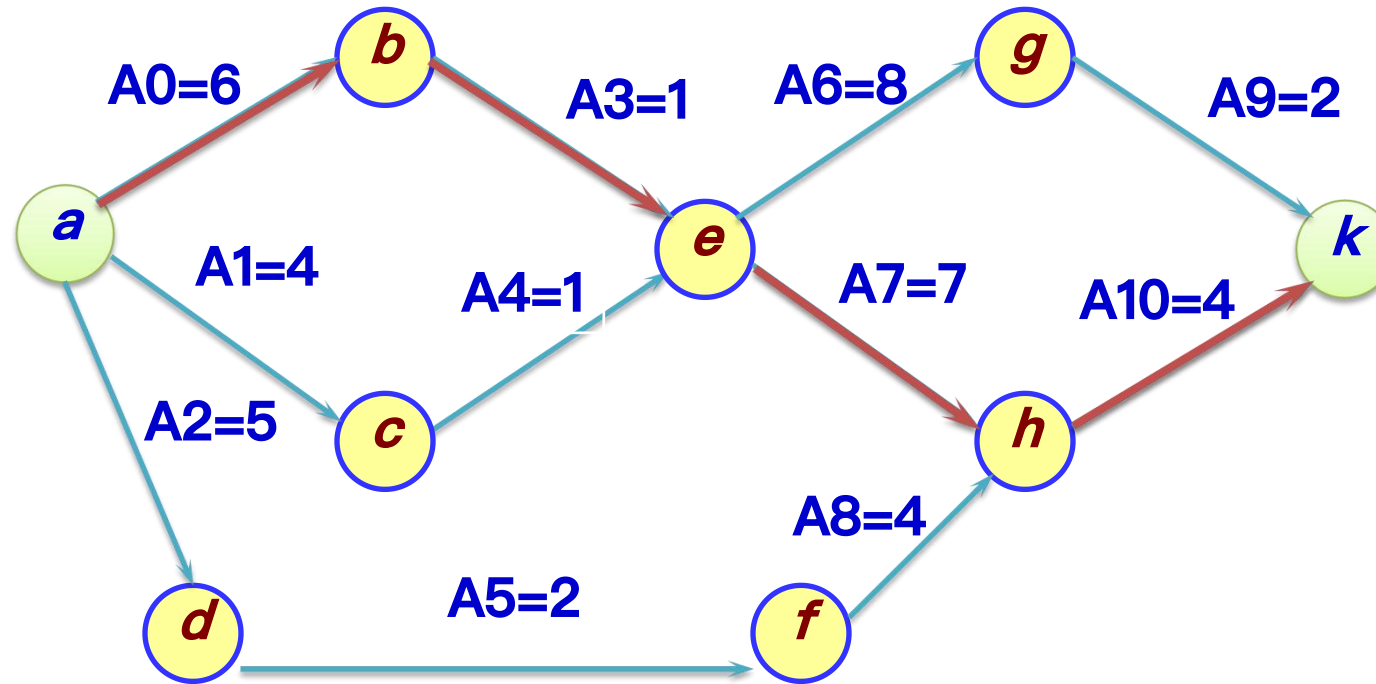


活动 a 的最早开始时间 $e(a)$ 指该活动起点 x 事件的最早开始时间，即：

$$e(a) = ee(x)$$

活动 a 的最迟开始时间 $l(a)$ 指该活动终点 y 事件的最迟开始时间与该活动所需时间之差，即：

$$l(a) = le(y) - c$$



ee

a	b	c	d	e	f	g	h	k
0	6	4	5	7	7	15	14	18

le

a	b	c	d	e	f	g	h	k
0	6	6	8	7	10	16	14	18

e

A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
0	0	0	6	4	5	7	7	7	15	14

$$e(a) = ee(x)$$

l

A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
0	2	3	6	6	8	8	7	10	16	14

$$l(k) = le(y) - c$$

(3) 求关键活动

对于每个活动 a ，求出 $d(a)=l(a)-e(a)$ ，若 $d(a)$ 为 0，则称活动 a 为 **关键活动**。

对关键活动来说，不存在富余时间。



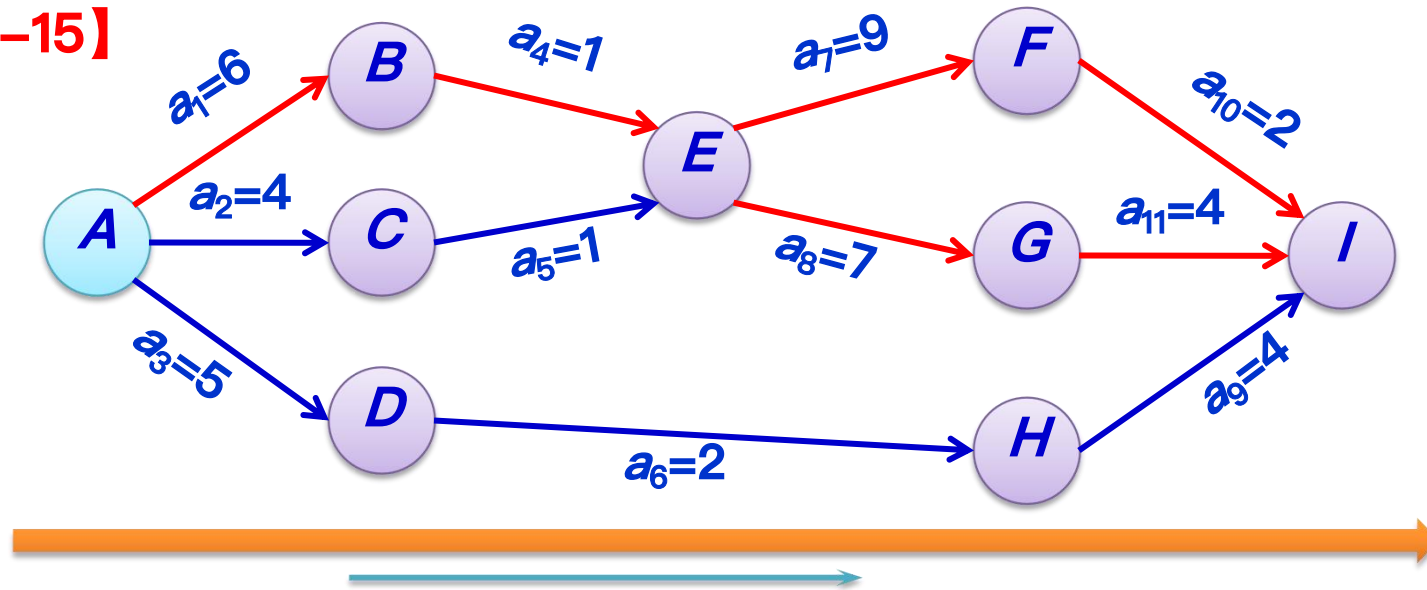
老师布置的作业要求一个星期交，而小明恰好需要一个星期做完，那么这个作业对于小明来说就是关键活动。



老师布置的作业要求一个星期交，而小英只需要**3**天做完，那么这个作业对于小英来说就不是关键活动，她有**4**天的富裕时间。

一个不存在富裕时间的活动必须赶紧完成，否则会影响整个工期

【例8-15】



ee

A	B	C	D	E	F	G	H	I
0	6	4	5	7	16	14	7	18

le

A	B	C	D	E	F	G	H	I
0	6	6	12	7	16	14	14	18

e

a1	a2	a3	a4	a5	a6	a7	a8	a9	a10	a11
0	0	0	6	4	5	7	7	7	16	14

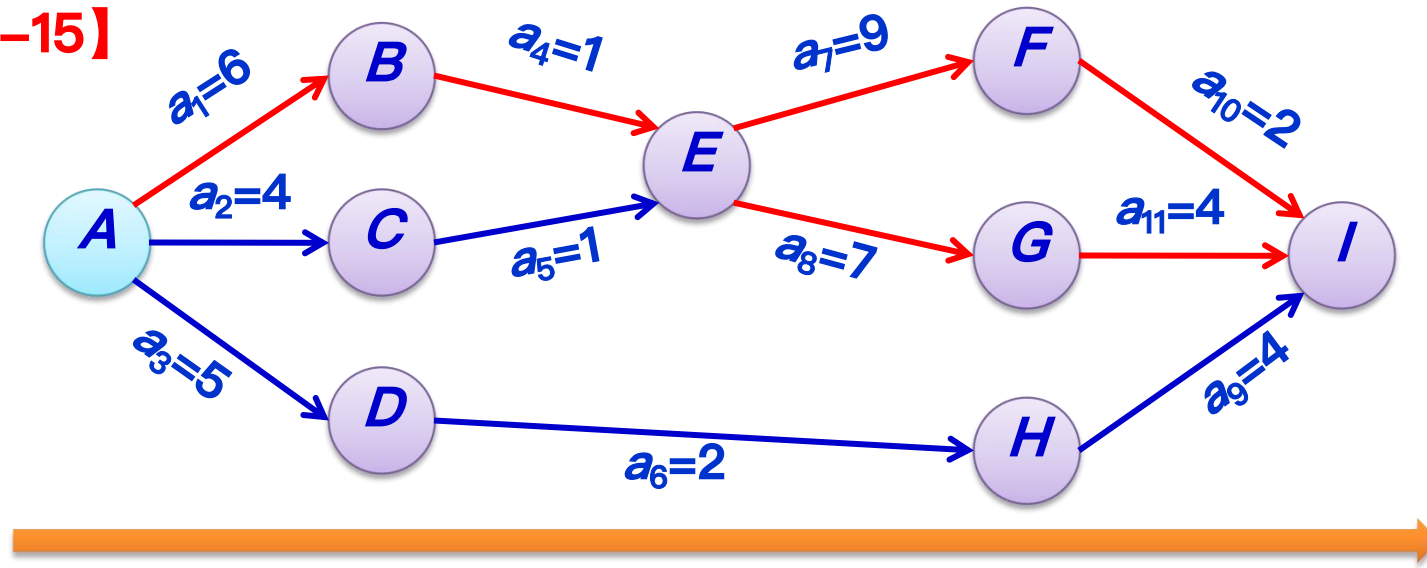
$$e(a) = ee(x)$$

l

a1	a2	a3	a4	a5	a6	a7	a8	a9	a10	a11
0	2	7	6	6	12	7	7	14	16	14

$$l(k) = le(y) - c$$

【例8-15】



先进行拓扑排序，假设拓扑序列为：*ABCDEFGHI*

计算各事件的 $ee(v)$ 如下：

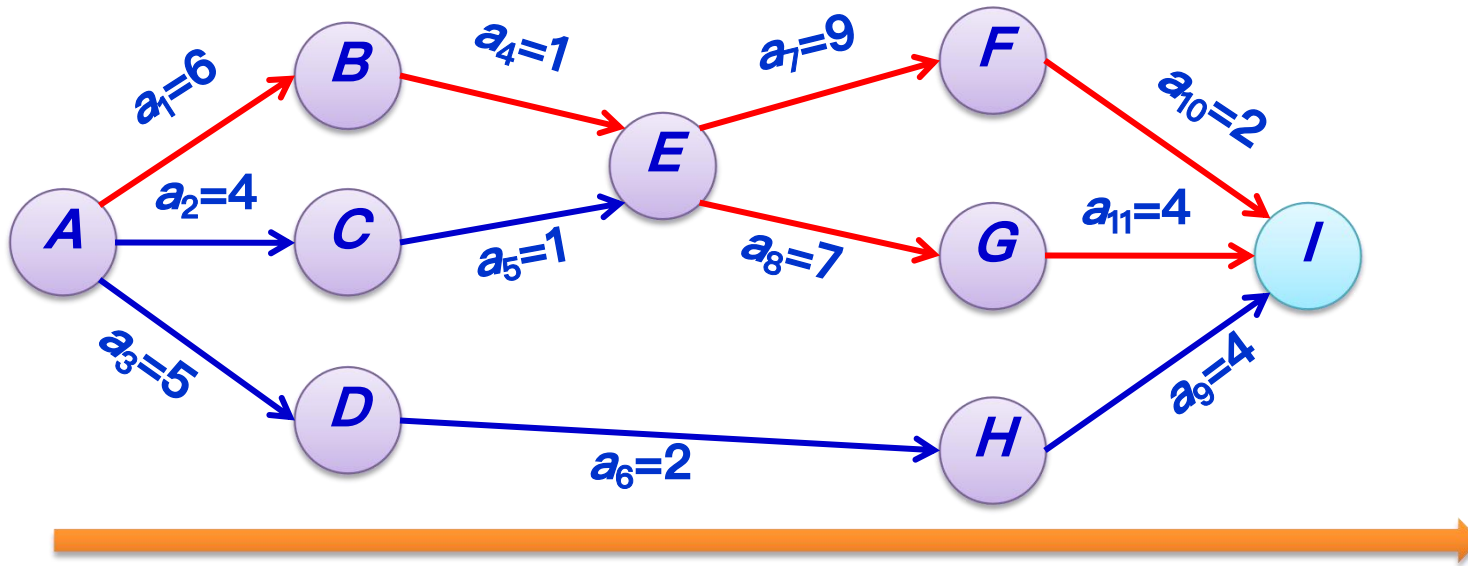
$$ee(A)=0$$

$$ee(B)=ee(A)+c(a_1)=6$$

$$ee(C)=ee(A)+c(a_2)=4$$

$$ee(D)=ee(A)+c(a_3)=5$$

$$ee(E)=\text{MAX}(ee(B)+c(a_4), ee(C)+c(a_5))=\text{MAX}\{7, 5\}=7$$

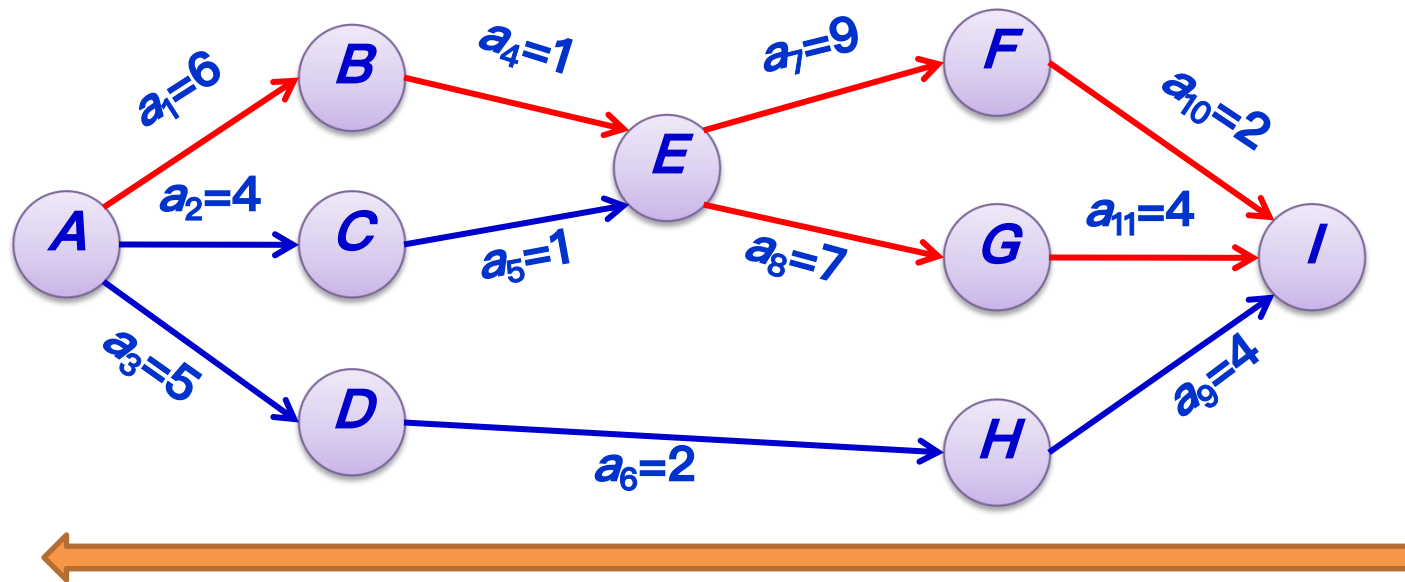


$$ee(F) = ee(E) + c(a_7) = 16$$

$$ee(G) = ee(E) + c(a_8) = 14$$

$$ee(H) = ee(D) + c(a_6) = 7$$

$$\begin{aligned}
 ee(I) &= \text{MAX}\{ee(F) + c(a_{10}), ee(G) + c(a_{11}), ee(H) + c(a_9)\} \\
 &= \text{MAX}(18, 18, 11) = 18
 \end{aligned}$$



拓扑序列为 **ABCDEFGHI**，按拓扑逆序 **IHG FEDCBA** 计算各事件的 **le(v)** 如

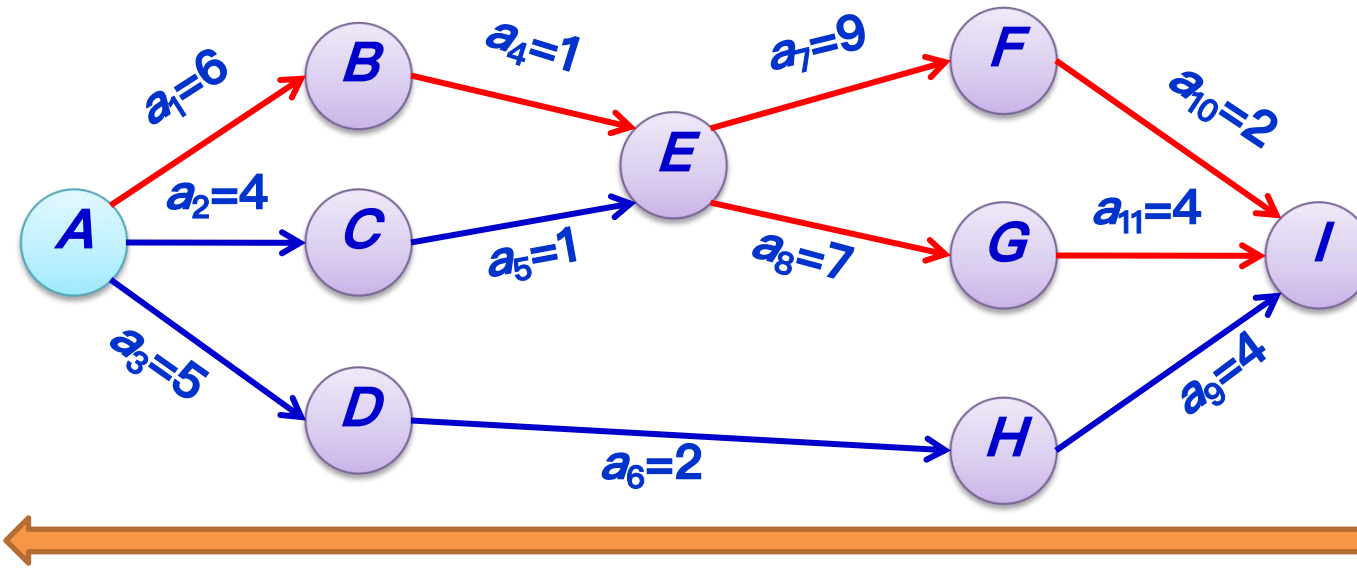
下：

$$\text{le}(\text{I}) = \text{ee}(\text{I}) = 18$$

$$\text{le}(\text{H}) = \text{le}(\text{I}) - c(a_9) = 14$$

$$\text{le}(\text{G}) = \text{le}(\text{I}) - c(a_{11}) = 14$$

$$\text{le}(\text{F}) = \text{le}(\text{I}) - c(a_{10}) = 16$$



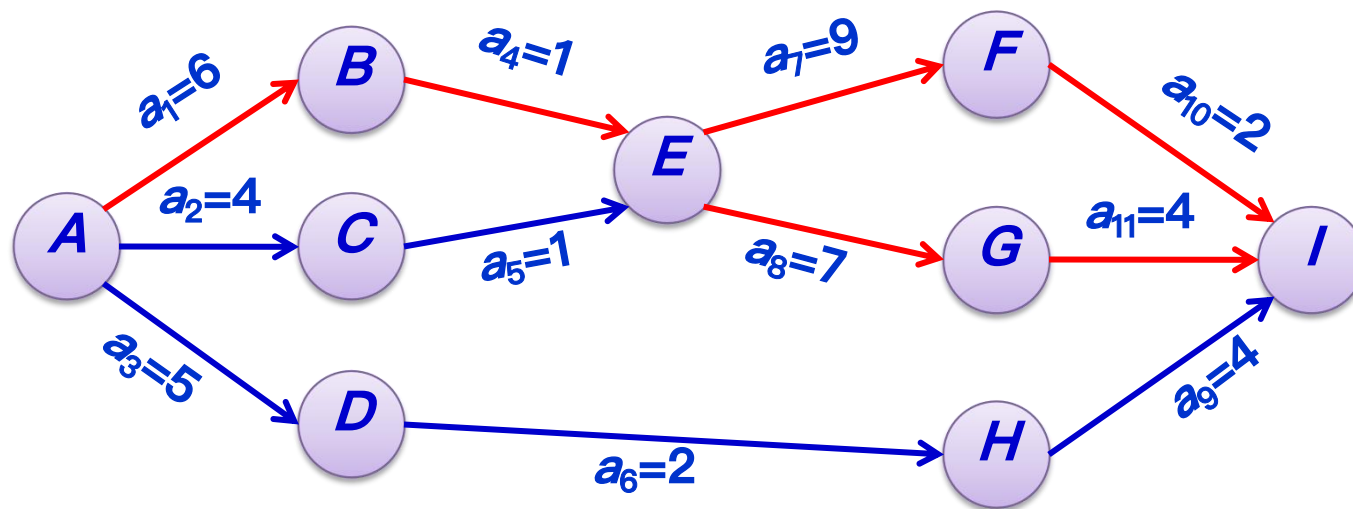
$$le(E) = \min(le(F) - c(a_7), le(G) - c(a_8)) = \{7, 7\} = 7$$

$$le(D) = le(H) - c(a_6) = 12$$

$$le(C) = le(E) - c(a_5) = 6$$

$$le(B) = le(E) - c(a_4) = 6$$

$$le(A) = \min(le(B) - c(a_1), le(C) - c(a_2), le(D) - c(a_3)) = \{0, 2, 7\} = 0$$



计算各活动的 $e(a)$ 、 $l(a)$ 和 $d(a)$ 如下：

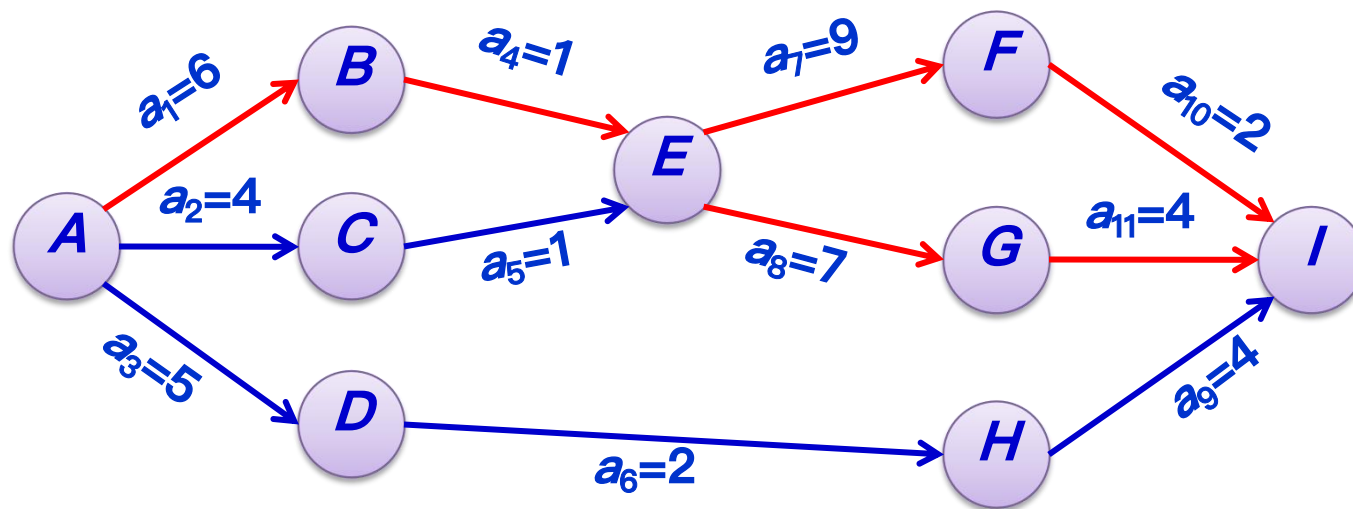
活动 a_1 : $e(a_1)=ee(A)=0$, $l(a_1)=le(B)-6=0$, $d(a_1)=0$

活动 a_2 : $e(a_2)=ee(A)=0$, $l(a_2)=le(C)-4=2$, $d(a_2)=2$

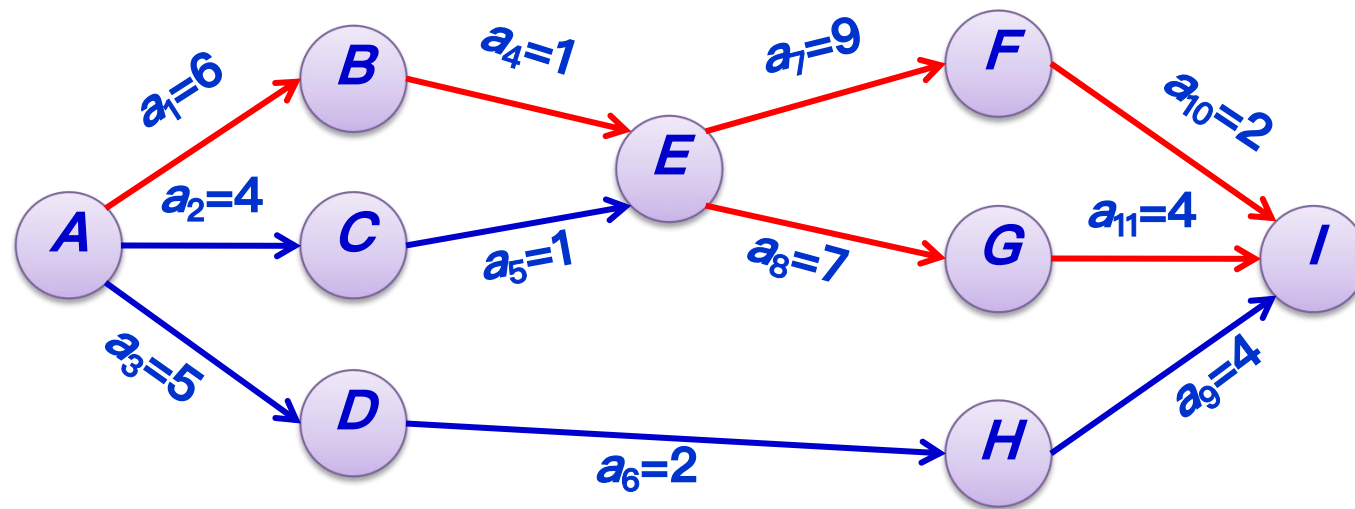
活动 a_3 : $e(a_3)=ee(A)=0$, $l(a_3)=le(D)-5=7$, $d(a_3)=7$

活动 a_4 : $e(a_4)=ee(B)=6$, $l(a_4)=le(E)-1=6$, $d(a_4)=0$

活动 a_5 : $e(a_5)=ee(C)=4$, $l(a_5)=le(E)-1=6$, $d(a_5)=2$



活动 a_6 :	$e(a_6)=ee(D)=5,$	$l(a_6)=le(H)-2=12,$	$d(a_6)=7$
活动 a_7 :	$e(a_7)=ee(E)=7,$	$l(a_7)=le(F)-9=7,$	$d(a_7)=0$
活动 a_8 :	$e(a_8)=ee(E)=7,$	$l(a_8)=le(G)-7=7,$	$d(a_8)=0$
活动 a_9 :	$e(a_9)=ee(H)=7,$	$l(a_9)=le(I)-4=14,$	$d(a_9)=3$
活动 a_{10} :	$e(a_{10})=ee(F)=16,$	$l(a_{10})=le(I)-2=16,$	$d(a_{10})=0$
活动 a_{11} :	$e(a_{11})=ee(G)=14,$	$l(a_{11})=le(I)-4=14,$	$d(a_{11})=0$



由此可知，关键活动有 a_{11} 、 a_{10} 、 a_8 、 a_7 、 a_4 、 a_1 ，因此关键路径有两条： $A-B-E-F-I$ 和 $A-B-E-G-I$ 。

示例

以下对于AOE网的叙述中，错误的是（ ）。

- A.在AOE网中可能存在多条关键路径 ✓
- B.关键活动不按期完成就会影响整个工程的完成时间 ✓
- C.任何一个关键活动提前完成，整个工程也将提前完成 ✗
- D.所有关键活动都提前完成，整个工程也将提前完成 ✓

第8章-小结 (1)

② 图两种存储方法的特点

示例

以下关于图的存储结构的叙述中正确的是_____。

- A. 一个图的邻接矩阵表示唯一，邻接表表示唯一
- B. 一个图的邻接矩阵表示唯一，邻接表表示可能不唯一
- C. 一个图的邻接矩阵表示可能不唯一，邻接表表示唯一
- D. 一个图的邻接矩阵表示可能不唯一，邻接表表示可能不唯一

- 一个图的邻接矩阵表示唯一
- 邻接表表示可能不唯一（一个顶点相邻的所有顶点构成一个单链表，其中相邻顶点的节点顺序可以任意）

B

② 图两种存储方法的特点

示例

13. 含有 n ($n \geq 2$) 个顶点的无向图的邻接矩阵必然是一个_____。

A. 对称矩阵

B. 零矩阵

C. 上三角矩阵

D. 对角矩阵

● 答案：A

示例

以下关于图的存储结构的叙述中正确的是（ ）。

- A. 邻接矩阵占用的存储空间大小只与图中顶点数有关，而与边数无关
- B. 邻接矩阵占用的存储空间大小只与图中边数有关，而与顶点数无关
- C. 邻接表占用的存储空间大小只与图中顶点数有关，而与边数无关
- D. 邻接表占用的存储空间大小只与图中边数有关，而与顶点数无关

● 用邻接矩阵存储时，占用的存储空间大小为 $O(n^2)$ ；用邻接表存储时，占用的存储空间大小为 $O(n+e)$

A

示例

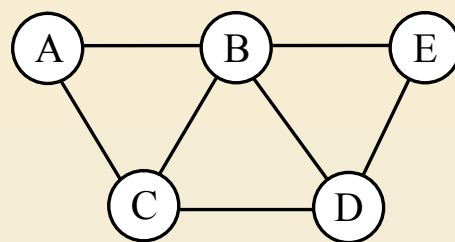
对如图所示的无向图，从顶点A出发得到的广度优先序列可能是_____。

A.ABECD

B.ACBDE

C.ACDBE

D.ABDEC



● 答案：B



若一个具有 n 个顶点和 e 条边的无向图是一个森林 ($n > e$)，则该森林必有 () 棵树。

A. e

B. n

C. $n - e$

D. 1

- 设该森林有 m 棵树，结点个数分别为 $n_1, n_2, \dots, n_m$
- 总结点数 $n = n_1 + n_2 + \dots + n_m$
- 第 i 棵树的边数至少 $= n_i - 1$ (可以看成自己的生成树)
- 总边数 $= (n_1 - 1) + (n_2 - 1) + \dots + (n_m - 1) = n - m = e$, 所以 $m = n - e$

C



对某个带权连通图构造最小生成树，以下说法中正确的是（ ）。

I. 该图的所有最小生成树的总代价一定是唯一的 ✓

II. 该图的最小生成树是唯一的 ✗

III. 用Prim算法从不同顶点开始构造的所有最小生成树一定相同 ✗

IV. 使用Prim和Kruskal算法得到的最小生成树总不相同 ✗

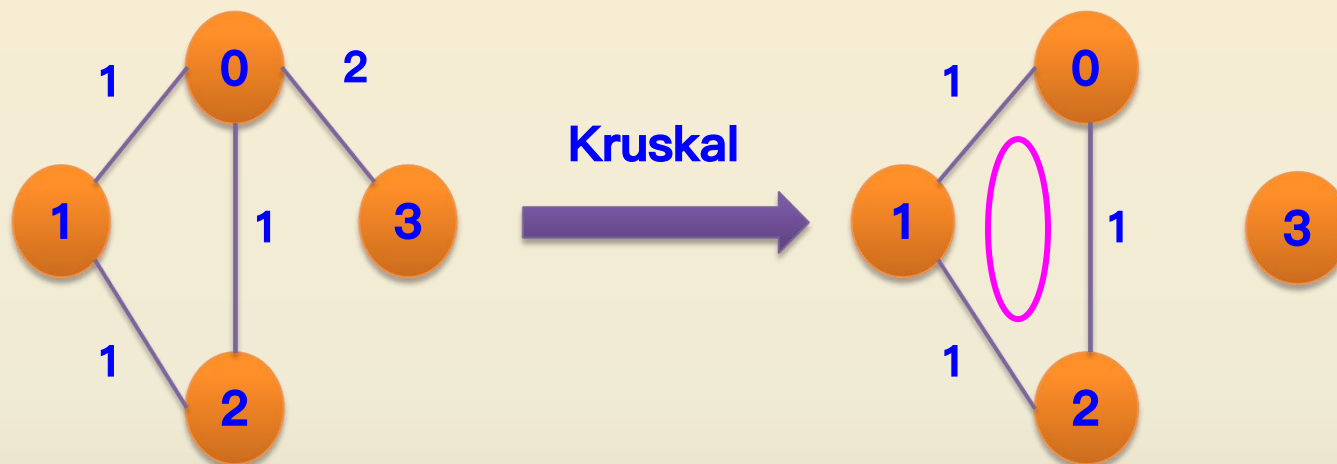
A. 仅 I B. 仅 II C. 仅 I、III 仅 II、IV

A



一个带权连通图中所有权值最小的边一定会出现在所有的最小生成树中?

不一定!



示例

Dijkstra算法是（ ）方法求出图中从源点到其余顶点最短路径的。

- A.按长度递减的顺序求出图的某顶点到其余顶点的最短路径
- B.按长度递增的顺序求出图的某顶点到其余顶点的最短路径 ✓
- C.通过深度优先遍历求出图中某顶点到其余顶点的最短路径
- D.通过广度优先遍历求出图中某顶点到其余顶点的最短路径

加入S集合的顶点:



- 最短路径不再改变
- 越后加入的顶点，**dist**越长